

University of Trento

Master's Degree in Data Science

~ · ~

Academic Year 2025–2026

Introducing an LLM-Based Approach for Guideline-Grounded Discharge Recommendations in Italian Emergency Departments

Supervisor
Professor Jacopo Staiano

Co-Supervisor
Professor Andrea Passerini

Candidate
Damiano Zaccaron
247952

Final examination date: October 15, 2026

Abstract

Qui va l'abstract

Contents

1 Introduction **1**
- 1.1 Current Technologies and the Limits of Large Language Models . . . 2
- 1.2 Objectives and Scope . . . 4
- 1.3 Research Questions and Contributions . . . 6
  - 1.3.1 Research Questions . . . 6
  - 1.3.2 Contributions . . . 7
- 1.4 Thesis Outline . . . 8

2 Related Works **9**
- 2.1 LLMs for Clinical Documentation . . . 9
- 2.2 Retrieval-Augmented Generation in Medicine . . . 10
- 2.3 Retrieval from Patient Descriptions . . . 12
- 2.4 Machine Translation in Clinical Pipelines . . . 13
- 2.5 Evaluating Clinical Text Generation . . . 14

3 Theoretical Framework **17**
- 3.1 Transformer-Based Language Models . . . 17
  - 3.1.1 The Transformer Architecture . . . 17
  - 3.1.2 Encoder Models . . . 19
  - 3.1.3 Large Language Models . . . 19
- 3.2 Retrieval-Augmented Generation . . . 21
  - 3.2.1 Formulation . . . 21
  - 3.2.2 Advantages over Parametric Knowledge . . . 22
  - 3.2.3 Advanced Retrieval Pipelines . . . 22
  - 3.2.4 Limitations . . . 23
- 3.3 Document Processing for Retrieval . . . 23
  - 3.3.1 Text Extraction from PDF . . . 24
  - 3.3.2 Sentence Segmentation . . . 24
  - 3.3.3 Chunking . . . 24
- 3.4 Clinical Information Extraction . . . 25
  - 3.4.1 Named Entity Recognition . . . 25
  - 3.4.2 Negation Detection . . . 25
- 3.5 Text Representations . . . 26
  - 3.5.1 Sparse Representations . . . 26
  - 3.5.2 Dense Representations . . . 27
- 3.6 Vector Similarity Search . . . 28
  - 3.6.1 Similarity Functions . . . 28
  - 3.6.2 Exact and Approximate Search . . . 29
- 3.7 Reranking . . . 30
  - 3.7.1 Score Aggregation . . . 30
  - 3.7.2 Cross-Encoders . . . 31

4 System Architecture **33**
4.1 Design Principles . . . 34
4.1.1 Traceability . . . 34
4.1.2 Structural Rather Than Vocabulary-Based Rules . . . 35
4.1.3 Privacy . . . 35
4.2 Guideline Ingestion Pipeline . . . 35
4.2.1 Parsing and Region Separation . . . 36
4.2.2 Table Processing . . . 36
4.2.3 Prose Cleaning . . . 37
4.2.4 Segmentation and Chunking . . . 38
4.2.5 Indexing and Stored Data . . . 38
4.3 Query Processing and Generation Pipeline . . . 38
4.3.1 Input Acquisition and Anonymisation . . . 39
4.3.2 Translation and Section Parsing . . . 39
4.3.3 Query Construction . . . 40
4.3.4 Text Retrieval and Reranking . . . 40
4.3.5 Table Retrieval . . . 41
4.3.6 Prompt Routing and Assembly . . . 41
4.3.7 Generation and Output . . . 42
4.4 Deployment Configurations . . . 42

5 The Guideline Corpus **45**
5.1 Requirements and Design Principles . . . 45
5.1.1 Requirements . . . 46
5.1.2 Design Principles . . . 47
5.2 Defining the Scope . . . 48
5.2.1 The Reference Textbook . . . 48
5.2.2 Title Terms . . . 49
5.2.3 Terms from the References . . . 50
5.2.4 The EM Model . . . 51
5.3 Searching PubMed . . . 52
5.3.1 Query and Filters . . . 52
5.3.2 The European Filter . . . 53
5.3.3 Keeping Every Hit . . . 54
5.3.4 Search Statistics . . . 54
5.4 Selecting Guidelines . . . 55
5.4.1 The Selection Call . . . 55
5.4.2 Long Lists and Model Choice . . . 56
5.4.3 The Fallback Tier . . . 56
5.4.4 Consolidation . . . 57
5.5 Retrieving Full Texts . . . 58
5.5.1 Automatic Retrieval . . . 58
5.5.2 Validation . . . 58
5.5.3 Browser-Assisted Retrieval . . . 59
5.5.4 Legal Basis . . . 60
5.6 The Resulting Corpus . . . 60
5.6.1 Size and Composition . . . 60
5.6.2 Cost and Running Time . . . 60
5.7 Evaluation . . . 61

5.7.1 Precision . . . 62
5.7.2 Recall . . . 62
5.7.3 Coverage of Clinical Notes . . . 63
5.7.4 Summary and Limitations . . . 63

6 System Implementation 65
6.1 Codebase and Configuration . . . 65
6.1.1 Repository Layout . . . 65
6.1.2 Configuration . . . 66
6.1.3 Development and Runtime Environment . . . 67
6.2 Guideline Ingestion . . . 67
6.2.1 PDF Parsing . . . 68
6.2.2 Cleaning . . . 68
6.2.3 Sentence Segmentation and Chunking . . . 69
6.2.4 Table Extraction . . . 69
6.2.5 Embedding . . . 70
6.2.6 Vector Indexes and Lexical Resources . . . 71
6.3 Input Processing . . . 72
6.3.1 Reading the Report . . . 73
6.3.2 Anonymisation . . . 74
6.3.3 Separation of the Physician's Advice . . . 74
6.3.4 Translation . . . 75
6.3.5 Field Parsing and Routing . . . 76
6.4 Query Construction and Retrieval . . . 76
6.4.1 Rule-Based Query Construction . . . 78
6.4.2 LLM-Based Queries . . . 80
6.4.3 Dense Retrieval . . . 81
6.4.4 Reranking and Selection . . . 81
6.4.5 Table Retrieval . . . 83
6.5 Generation . . . 83
6.5.1 Prompt Assembly . . . 84
6.5.2 Home Discharge Template . . . 84
6.5.3 Admission Template . . . 85
6.5.4 Generation Back Ends . . . 86
6.5.5 Output Translation . . . 86
6.6 User Interface . . . 86
6.6.1 Model Loading and Input . . . 87
6.6.2 Options and Processing . . . 88
6.6.3 Output . . . 89
6.6.4 Index Maintenance . . . 90

7 Evaluation 91
7.1 Evaluation Design . . . 91
7.1.1 The Clinical Notes . . . 91
7.1.2 Deployment Configurations . . . 92
7.1.3 Protocol and Scoring . . . 93
7.1.4 Comparing Settings . . . 94
7.2 Translation . . . 95
7.3 Retrieval . . . 96

7.3.1 The Retrieval Judge . . . 96
7.3.2 Embedding Model . . . 97
7.3.3 Retrieval Depth . . . 97
7.3.4 Cross-Encoder . . . 98
7.4 Generation Results . . . 98
7.4.1 Overall Results . . . 99
7.4.2 Generation Models and Queries . . . 100
7.4.3 Wrong and Harmful Statements . . . 102
7.4.4 Cross-Encoder . . . 102
7.5 Latency and Cost . . . 102
7.6 Discussion . . . 104
7.6.1 Comparison with the Physicians' Advice . . . 104
7.6.2 From Frontier to Hosted Models . . . 105
7.6.3 Deployment Recommendations . . . 105

8 Limitations **107**
8.1 Guideline Corpus . . . 107
8.2 System Design and Deployment . . . 108
8.3 Threats to Validity . . . 109
8.4 Ethical and Regulatory Considerations . . . 110
8.5 Future Work . . . 111

9 Conclusions **113**

Appendix A: Guideline Corpus **117**
A.1 Prompts . . . 117
A.2 Tables . . . 121

Appendix B: Experimental Materials **123**
B.1 Prompts . . . 123
B.2 Storage of Tables . . . 129
B.3 Table of Costs . . . 130

Appendix C: Output and Evaluation **131**
C.1 Clinical Note Template . . . 131
C.2 Queries . . . 132
C.3 Output Comparison with the Physician's Advice . . . 134
C.4 Scores per Note . . . 136

Bibliography **137**

List of Figures **145**

List of Tables **147**

Chapter 1

Introduction

In Italy, every patient who accesses an Emergency Department (ED) receives a written document summarising their visit. This is referred to as the *verbale di Pronto Soccorso*. The document, signed by the attending physician, is intended to serve both clinical and medico-legal purposes: it records what happened during the visit, and provides a reference for any future care or legal proceedings.

While no national legislation is in place to determine how it must be written, its structure is largely standardised in practice and, in its digital form, formalised by the national HL7 CDA2 implementation[1]. After the patient's demographic and triage data, the medical history (*anamnesi*) reports the reason for the visit and the history and symptomps of the present illness, followed by the patient's relevant past conditions and ongoing therapies. The objective examination (*esame obiettivo*) records the physical findings at presentation, while the clinical diary (*diario clinico*) logs, step by step, every test, consultation and treatment performed during the stay, along with the medical staff involved at each step. The document then states the diagnosis and a prognosis, the latter expressed either as an expected number of days to recovery or as hospitalisation. Finally, it closes with a set of discharge recommendations.
The recommendations section differs from the rest of the document in both purpose and audience. in fact, while the preceding sections record what happened in the ED and are read mainly by clinicians, the recommendations describe what should happen *next* and are addressed to two readers at once: the patient, or their caregivers, and the general practitioner who will take over their care. This sections represents the main tool to convey written instructions to the patient, and contains instructions on what to do after leaving the Emergency Room: which therapies to continue, start or stop, which specialist visits to book, which lifestyle changes to adopt, and which warning signs should bring them back to the ED. For the general practitioner, it is the basis for organizing follow-up care.

However, EDs operate under chronic crowding and time pressure[2], which has been proven to negatively impact patient's care and occasionally lead to medical errors[3,4]. This reflects on the quality of the discharge recommendations as well. Being written under time pressure and with the physician's attention divided between multiple patients, they can be incomplete or hard to understand for a patient without medical training.
Some instructions are only given orally and risk being forgotten, particularly by elderly, distressed or otherwise impaired patients; others, deemed obvious or secondary, are left implicit and never communicated at all. What does reach the page tends to be terse and

dense with clinical abbreviations (Figure 1.1), which can be opaque to patients, and is sometimes written in fragmentary sentences. The content most often omitted is secondary guidance such as lifestyle modifications and non-pharmacological measures.

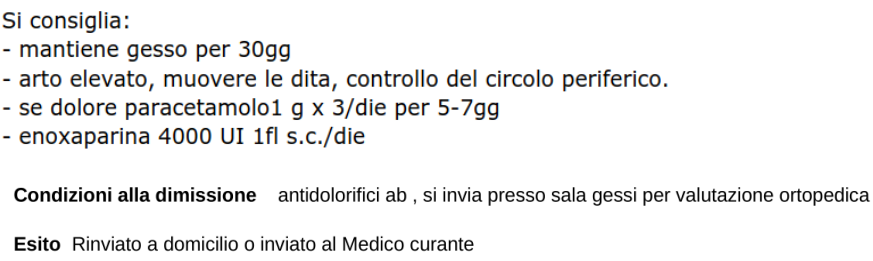


Figure 1.1. Illustrative examples of recommendation sections as typically written under time pressure. Abbreviations (s.c., die, fl, ab), brief and ambiguous instructions, misspellings, sentence fragments and generic prescriptions ("antidolorifici", painkillers).

These shortcomings have practical consequences. Patients discharged from the ED often leave with an incomplete understanding of the instructions they received, and are frequently unaware of the gaps in their own understanding[5]. Yet, it has been proven that clear written instructions are one of the main tools available to mitigate this effect[6]. Incomplete or unclear recommendations can compromise adherence to therapy and delay the recognition of complications, and they might also reflect on the physician and the hospital itself.

It is important to specify that none of this is due to the medical staff's negligence. It is a consequence of the environment in which the document is produced: the physician knows what the patient should do, but lacks the time to convey it to them in written form completely and clearly. The issue is therefore a natural candidate to be addressed with assited support, through a tool that is able to draft a complete, readable and guideline-consistent set of recommendations, which the physician then only needs to review and sign. In the next section, we will address the question of whether current language technologies are up to the task.

1.1 Current Technologies and the Limits of Large Language Models

Some software solutions to assist physicians in the production of discharge recommendations have already been implemented and are in use, with the information systems used in Italian EDs offering templates and predefined text snippets covering common conditions. These tools reduce the time spent typing, but are limited in scope, cannot adapt to the specific patient and are unable to provide assistance when the case is not covered by a template. In an environment where recommendations must be written for a wide variety of conditions on an heterogeneous patient population, these tools are of limited help, and physicians are forced to resort back to manual writing.

The arrival on the market of large language models (LLMs) did affect this picture. Since the public release of ChatGPT, at the end of 2022, general-purpose conversational models such as ChatGPT, Gemini and Claude have become more and more able to produce fluent,

well-structured text, and to rephrase technical content for a lay audience.
Many physicians have integrated the use of these models in clinical practice, largely on their own initiative: in a 2025 survey of general practitioners in the United Kingdom, one in four reported using generative AI tools in their clinical work, most commonly to write documentation, despite nearly all of them having received no formal training in their use[7,8]. The linguistic capability behind this adoption is real: in a study on inpatient discharge summaries, GPT-4 rewrote physician-authored summaries into versions with a substantially higher understandability scores than the originals[9]. For an ED physician, pasting the clinical note into a chat window and asking for a patient-friendly version of the recommendations is therefore a tempting option, and one that is often used in practice. However, the limitations of general-purpose LLMs make this approach unsafe, as discussed in the following paragraphs.

Alongside general-purpose models, a market of specialised clinical AI products has emerged and is growing rapidly[10]. Unlike general-purpose models, these products are typically designed for a single, narrowly defined task and cannot be repurposed through prompting or even fine-tuning.
To the best of our knowledge, no available product generates discharge recommendations grounded in European clinical guidelines and integrated into the workflow of an Italian ED. In practice, general-purpose LLMs remain the only readily available solution, and their limitations define the problem addressed in this thesis.

Hallucinations and omissions. LLMs are prone to *hallucinations*[11]: content that is plausible and fluently written, but supported neither by the input nor by fact. They can occur even when the model is only asked to summarise information already present in the note, and pose a serious threat when utilized in a context where accuracy is paramount. These hallucinations are hard to detect, as models often appear confident and authoritative even when the content displayed is false. This overconfidence often leads users to trust the output without verification, and to overlook errors[12].
In an evaluation of GPT-4 drafting ED encounter summaries, factual inaccuracies were discovered in 10% of cases, with 42% of the summaries containing hallucinated content and 47% omitting clinically relevant information; inaccuracies and hallucinations were concentrated in the *Plan* section, the part of the summary closest to discharge recommendations[13]. Similarly, in the discharge summary study cited above, 18 of the 100 physician reviews flagged potential safety concerns, mostly due to omissions but also to inaccuracies[9]. While these studies showed improvements between different versions of the models (namely, GPT-3.5 and GPT-4), the problem persists even in the most recent models.
To exacerbate the issue, writing recommendations is a harder task than summarising, as it may require the model to add knowledge that is not contained in the note: which therapy to prescribe, at which dose, and which symptoms should prompt a return to the Emergency Room. An invented dose or a missed contraindication in a signed document is a direct risk to patient safety, and a potential source of legal liability for the physician and the hospital.

Opaque and unversioned knowledge. When a general-purpose model writes a recommendation, the knowledge behind it comes from its parameters, learned from undisclosed training data fixed at a cutoff date. There is no way to tell which guideline, if any, a recommendation reflects, which version of it, or which healthcare system it was written for. This matters because guidelines differ across jurisdictions and change over time. A model could easily recommend a therapy based on a guideline that is not applicable to the country where the patient is being treated, or that has been superseded by a more recent version.

The only way for the physician to verify the output is to consult the guideline personally. However, this would cancel most of the time the tool was meant to save.

Unreliable source attribution. To mitigate the aforementioned issues, some LLMs are designed to offer the option to provide citations for the statements they make. In theory, this should reduce the risk of hallucinations and provide a direct link for the user to verify the content. In practice, the cited sources often do not support the statements they are attached to.
An automated evaluation of popular LLMs on medical questions found that between 50% and 90% of responses were not fully supported by the sources they cited, and some were even contradicted by them; with web search enabled, GPT-4o still left about 30% of individual statements unsupported[14]. Moreover, since sources are drawn from the open web, the user has no control over whether an answer rests on an official guideline, a review article or a discussion forum. In fact, multiple studies have shown that LLMs consistently provide citations from non-reliable sources, even in the medical domain, in more than 10% of cases[14,15], with one study reporting a rate of 16.6% AI-generated sources and 8.1% coming from social media[16].

Data protection and regulation. Clinical notes contain health data, which is identified as a special category of personal data under the General Data Protection Regulation[17]. Pasting a note into a consumer chatbot transfers it to a third-party provider, outside the control of the hospital and potentially outside the European Union, without the contractual and technical safeguards that the processing of health data requires. These aspects are discussed further in Section 8.4.

Taken together, these limitations show that the missing piece is not linguistic capability: current LLMs can already write clear, patient-friendly text. What is missing is control over where the clinical content comes from, a reliable way to verify each statement, and a deployment compatible with the handling of health data. A useful system must therefore restrict the model to an explicitly chosen, authoritative body of knowledge, make every recommendation traceable to the passage that supports it, and leave the final decision to the physician. These requirements shape the objectives of this thesis.

1.2 Objectives and Scope

The objective of this thesis is to design, implement and evaluate a system that supports ED physicians in writing discharge recommendations, and to build a comprehensive corpus of official clinical guidelines to serve as the system's knowledge base. Given the clinical note of an ED visit or its draft, written in Italian, the system produces a draft set of recommendations that is complete, readable by the patient and grounded in clinical guidelines, together with a reference for each recommendation that allows the physician to verify it before signing the document.

Following the limitations discussed in Section 1.1, the system is designed around four requirements:

1. **Grounding**: to reduce the risk of inaccuracies and hallucinations, the clinical content of the recommendations must come exclusively from an explicitly chosen corpus of authoritative guidelines, retrieved at inference time, rather than from the parametric knowledge of the language model.

2. **Traceability**: every recommendation must be linked to the guideline excerpts that support it, so that the reviewing physician can check it without having to search for the source.

3. **Completeness and readability**: the recommendations must cover not only pharmacological therapy but also the secondary guidance most often omitted under time pressure, and must be phrased in plain and understandable language for the patient.

4. **Deployability and Privacy**: the system must accept the documents already produced by the ED, protect patient data before any processing, and be designed for deployment within local servers, without relying on third-party services that would transfer sensitive data outside the hospital network.

The scope of the system is deliberately narrow. It does not diagnose, suggest treatments, or review clinical decisions made by the medical staff. The diagnosis and the prognosis are written by the physician and taken as input: the system neither questions them nor proposes alternatives, and its task begins where the clinical reasoning of the visit ends. Furthermore, it is important to emphasize that the system is a support tool, not an autonomous one. Its output is a draft that the physician can review and edit as needed before signing the final document, as he retains full clinical and legal responsibility for it. The references attached to each recommendation exist precisely to make this review fast.

The system is designed to handle the two possible outcomes of an ED visit: discharge home or hospital admission, though its main contribution is in the former, since the latter can already be handled by structured templates. When the patient is discharged home, the system produces recommendations addressed to the patient and their general practitioner, covering therapy, follow-up visits, lifestyle changes and criteria for returning to the ED. When the patient is admitted to hospital, ongoing therapy and follow-up become the responsibility of the admitting team, and the system only produces a short handover note.

The system is built for Italian EDs. It takes as input the PDF document produced by the information system of the ED, with plain text available as a fallback, and returns its output in Italian. Internally, retrieval and generation operate in English, the language of the guideline corpus, with translation performed at both ends of the pipeline.

By default, the system draws its knowledge from a corpus of European clinical guidelines published by medical societies, built through the automated procedure described in Chapter 5. However, the system is not necessarily bound to this corpus, as the ingestion pipeline accepts any collection of guideline documents, and the index can be rebuilt directly from the application. Users can therefore elect to extend the default corpus with their own documents, or replace it entirely, for instance with national, regional or hospital-specific protocols. The only constraint is language: since retrieval operates in English, documents must be in English, and guidelines published in other languages would need to be translated before ingestion. In every configuration, the recommendations are bounded by the content of the corpus, since the system can only recommend what its documents contain.

Finally, the system is a research prototype. It has not undergone the certification process required of medical devices, and its evaluation, presented in Chapter 7, is a feasibility study conducted with ED physicians rather than a clinical trial.

1.3 Research Questions and Contributions

1.3.1 Research Questions

The objectives stated above translate into three research questions, each addressing a different component of the system and answered in a dedicated part of this thesis.

RQ1: Corpus construction *To what extent can a guideline corpus covering the scope of emergency medicine be built automatically from open-access European guidelines, with minimal manual intervention?*

Since the system can only recommend what its corpus contains, the corpus sets an upper bound on the quality of every output. Clinical guidelines, however, are published by many independent societies, in different formats and on different platforms, and no single repository collects them all. Gathering them by hand would be a slow process, and still run into the problem of a lack of a comprehensive list of sources to consult. RQ1 asks whether this collection process can be automated, and how complete the resulting corpus is. It is answered in Chapter 5, by measuring the coverage of the corpus against the official guideline lists of European medical societies and against the scope of the specialty, defined by the chapters of a reference emergency medicine textbook[18] and by the conditions of the Model of the Clinical Practice of Emergency Medicine[19].

RQ2: Quality of the recommendations *How do the recommendations generated by the system compare with those written by physicians in terms of clinical accuracy, completeness, readability and traceability?*

This is the central question of the thesis, as it assesses the system against the requirements of Section 1.2 on the task it was built for. Physician-written recommendations are the natural reference, since they represent current practice. They are not, however, a gold standard, as one of the goals of the system is precisely to include the guidance that they often omit. For this reason, the question asks how the two compare, rather than whether the generated recommendations reproduce the original ones. Traceability is enforced by design, as every recommendation must cite the guideline excerpts it draws on or declare that it has none; whether a cited excerpt actually supports its item is left to the reviewing physician. RQ2 is answered in Chapter 7, through a blinded evaluation of realistic clinical cases.

RQ3: Deployability *How does the quality of the recommendations change when moving from a frontier model accessed through an external service to open-weight models that can be hosted locally?*

The best-performing language models are currently available only through external services, which conflicts with the data protection constraints discussed in Section 8.4. In fact, despite a de-identification process being implemented, this has not undergone the process of approval from the hospital's ethics committee, and the data is therefore not allowed to leave the hospital network. Open-weight models can be hosted within the hospital, but their quality depends on their size, and their size on the hardware available. RQ3 quantifies this trade-off by comparing open-weight models of increasing size, chosen to fit hardware configurations that a hospital could realistically host, against a frontier model used as a reference. It is answered in Chapter 7, using the same evaluation protocol as RQ2, complemented by measurements of generation time and cost per document.

1.3.2 Contributions

In addressing these questions, this thesis makes four contributions, concerning the system itself, the knowledge it draws on, the way it was evaluated and what the evaluation reveals about its deployment.

The central contribution is a Retrieval-Augmented System that drafts discharge recommendations from the clinical note of an Italian ED visit, producing an Italian draft ready for review (Chapters 4 and 6). The note is anonymised before any other processing, translated into English and recomposed into a set of focused queries. These can be built either automatically based on structural and semantic rules, from the clinical entities found in the note, or by the Large Language Model itself. The rule-based option runs in under a second and requires no additional call to the model, while the second lets a capable model decide what is worth looking up for each patient. The queries retrieve excerpts and tables from the guideline corpus, which are reranked and passed to the language model. The model then writes the recommendations using only this evidence, linking each of them to the excerpts that support it, so that the physician can verify any recommendation without searching for its source. The system is not tied to the corpus used in this thesis, and is specifically built to adapt to any guideline written in English. A hospital can therefore extend or replace it with its own protocols, and rebuild the index directly from the User Interface.

Because the system can only recommend what its corpus contains, the thesis also contributes an automated procedure for building a comprehensive corpus of European guidelines, and uses that corpus to test the model (Chapter 5). The procedure derives its topics from a reference pivotal Emergency Medicine textbook and from a model of the clinical practice of Emergency Medicine, searches PubMed for European guidelines on each of them, and relies on a language model to select among the results. Manual intervention is limited to retrieving the guidelines not freely available for download, so that a third party, such as a hospital, can run the procedure without assistance. Its final run produced a corpus of 457 European guidelines, which was evaluated on three aspects: whether the selected documents are the right ones, whether the guidelines of five major European societies were retrieved, and whether the diagnoses of the evaluation notes are covered.

To answer RQ2 and RQ3, the thesis then designed an evaluation that works within the constraints surrounding clinical data in Italian hospitals. Since real notes cannot leave the hospital without the approval of an ethics committee, physicians of the partner department wrote 25 synthetic notes that imitate real reports down to their abbreviations and typing errors, closing with the physician's own advice. The system was run on these notes in four configurations, each standing for a realistic deployment scenario, from frontier models accessed through an API to open-weight models sized for different sizes of hardware that a hospital could theoretically host. The resulting outputs were scored blind against the physicians' advice on coverage, errors, useful additions and overall quality.

Finally, the results of this evaluation offer practical indications for deploying such a system, although they rest on a small sample. With a frontier model, the generated recommendations compare favourably with the physicians' advice, covering most of its clinical content while adding the secondary guidance that is usually left out. When the frontier model is replaced by open-weight models, the loss in quality comes mainly from writing the retrieval queries rather than from writing the recommendations, so a local deployment is best served by rule-based

queries and a small reranker. These findings translate into a recommended configuration for each deployment scenario, determined above all by whether the clinical note may leave the hospital.

The source code of the system, together with the scripts used to build the corpus and run the evaluation, is publicly available in the project repository[1].

1.4 Thesis Outline

The remainder of this thesis is organised as follows. Chapter 2 reviews prior related work, from the use of language models in clinical documentation to the evaluation of generated clinical text, while Chapter 3 introduces the theoretical background on which the system relies. Chapter 4 describes the architecture of the system and the principles behind its design. Chapter 5 follows with the construction and evaluation of the guideline corpus upon which the system bases its knowledge, before Chapter 6 details how each component was built. Chapter 7 evaluates the system end to end and discusses the results. Chapter 8 examines the limitations of the work, together with its ethical and regulatory implications and the directions for future work, and Chapter 9 rounds up the thesis. Finally, the appendices collect the prompts and tables used to build the corpus (Appendix A), the prompts, table storage examples, a table of costs (Appendix B), the queries, a note template and example outputs of the system (Appendix C).

[1] `https://github.com/damianozaccaron/ERDischargeRecommendationsRAG/`

Chapter 2

Related Works

2.1 LLMs for Clinical Documentation

Clinical documentation is among the most time-consuming parts of medical work, with a 2016 study finding that, while in the examination room with patients, physicians in the U.S. spent 52.9% of the time on direct clinical face time and 37.0% on registering Electronic Health Records and desk work[20]. It is therefore not surprising that documentation has been one of the first targets of large language models (LLMs) in healthcare. The clinical competence of these models has been tested and demonstrated on knowledge-oriented benchmarks since they first emerged, with pioneering studies like Singhal et al.[21] showing that an LLM could reach passing-level accuracy on medical licensing exam questions, despite physician assessment of its long-form answers still revealing important gaps with respect to clinicians. While these results show that LLMs possess the capabilities to reason about medical knowledge, they say little about their ability to produce clinically useful text, and a growing body of work has since evaluated LLMs on generation tasks drawn from routine clinical practice.

Van Veen et al.[22] adapted eight LLMs to four clinical summarization tasks: radiology reports, patient questions, progress notes, and doctor-patient dialogue. The study reported that summaries from the best adapted model were judged equivalent to those written by medical experts in 45% of cases and superior in 36%. Two further findings of that study are relevant for the construction of this thesis. First, the accompanying safety analysis found errors with potential for harm, including fabricated information, in both model and expert summaries, so that expert-level quality does not remove the need for review. Second, a model further trained on medical text performed worse than its general-purpose counterpart, which the authors used as evidence to argue that that adaptation to the task matters more than adaptation to the medical domain.

The discharge documentations section has received specific attention, as it is both burdensome to produce and central to continuity of care. For inpatient stays, Williams et al.[23] compared physician-written and GPT-4-generated hospital discharge narratives in a blinded evaluation, finding that LLM-generated narratives were more concise and coherent than their physician-generated counterparts, but less comprehensive, while LLM-generated narratives were also more likely to contain errors, but their overall potential for harm was low. Ellershaw et al.[24] prompted GPT-4-turbo with the Royal College of Physicians guidelines on discharge summaries structure and form (not containing any medical information) and with physician

notes from MIMIC-III, reaching a micro accuracy of 0.81 on 53 summaries, despite finding 7 median errors per summary, 36% of which deemed severe.

In another study, Williams et al.[13] asked GPT-4 and GPT-3.5-turbo to turn the physician's note of 100 emergency department visits into an encounter summary for patients discharged from the department. They found that only a third of the GPT-4 summaries (33%) were free of errors: inaccuracies were rare (10%), but 42% contained hallucinations and 47% omitted clinically relevant information. The nature of these hallucinations is instructive. Apart from content that had been redacted during de-identification, the most frequent fabrications were follow-up arrangements, return precautions and discharge instructions that the original note did not contain. The authors observe that many of these instructions were clinically reasonable, resembling the standard advice associated with the final diagnosis, but had to be classified as hallucinations because nothing in the input supported them.

A parallel line of work addresses the patient receiving the document. Zaretsky et al.[9] used GPT-4 to rewrite 50 inpatient discharge summaries into patient-friendly language, significantly raising understandability, as measured by the Patient Education Materials Assessment Tool, from 13% to 81%. Accuracy, however, did not keep pace, with only 54 of the 100 physician reviews rated as entirely accurate, and 18 raising potential safety concerns.

Taken together, these studies frame clinical text generation as a transformation of the input: the output summarizes, simplifies or rewrites what the clinician has already documented. Under this framing, faithfulness to the source is the dominant criterion, and any content that cannot be traced back to the note is an error by construction, as highlighted by the analysis of Williams et al.[13]. Emergency room discharge recommendations fit this framing poorly. The recommendations most often missing from ER discharge documents, such as lifestyle modifications or instructions that were only given orally, are precisely those the physician had no time to write down (Section 1), and therefore cannot be recovered from the note. Producing them requires knowledge external to the patient record, and in a clinical setting that knowledge must come from a source the reviewing physician can verify, rather than from the parametric memory of the model. Reference-based evaluation compounds the problem: when the physician-written text is treated as the gold standard, a system that adds a correct but omitted recommendation is penalized rather than rewarded.
This thesis departs from the transformation framing by using the clinical note as the input to a retrieval step rather than as the only source of content: recommendations are generated from guideline excerpts selected for the specific patient, and each recommendation is linked to the excerpt that supports it. Section 2.2 reviews how retrieval-augmented generation has been applied in medicine and where existing systems fall short of this setting.

2.2 Retrieval-Augmented Generation in Medicine

Retrieval-Augmented Generation (RAG) was proposed by Lewis et al.[25] to enhance the generation performance on knowledge-intensive tasks by integrating retrieved relevant information. RAG not only mitigates the problem of hallucinations, as LLMs are grounded on given contexts, but can also provide up-to-date knowledge that might not be encoded by the LLMs. Retrieval-augmented generation, whose mechanics are described in Chapter 3, has been widely adopted in medicine[26] to mitigate two limitations of LLMs: hallucinations and outdated knowledge, such as recent updates of clinical guidelines that are absent from

their training data. MIRAGE[27] constructed a benchmark of 7,663 questions drawn from five medical question-answering datasets, on which the authors evaluated 41 combinations of corpora, retrievers and backbone LLMs. RAG improved the accuracy of six different LLMs by up to 18% over chain-of-thought prompting, bringing GPT-3.5 and Mixtral to the level of GPT-4, with the best results obtained by combining multiple corpora and retrievers. Interestingly, the study also observed a "lost-in-the-middle" effect, with evidence placed in the middle of the context being exploited less than evidence at its extremes.

Beyond benchmarks, several systems have been implemented to target clinical decision support directly. Almanac[28] retrieves from curated medical resources to answer clinical questions on guidelines and treatment; on 314 questions spanning nine specialties, a panel of clinicians stated that the model represented a significant improvement in performance compared with the standard LLMs (namely ChatGPT-4, Bard and Bing) across axes of factuality, completeness, user preference, and adversarial safety. Clinical guidelines, however, have proven to be a difficult source to retrieve from. Kresevic et al.[29] built a RAG framework with GPT-4 Turbo on guidelines for the management of chronic hepatitis C, and ran an ablation on how the guideline was presented to the model. Accuracy rose from 43% for the baseline model to 99% for the full framework, and the intermediate steps show that most of the improvement came from formatting: cleaning the text and exporting tables as CSV files raised accuracy to 78%, and rewriting tables as text-based lists raised it to 90%.

Using clinical notes as input, Ke et al.[30] applied RAG over 35 local and 23 international preoperative guidelines to assess fitness for surgery and produce preoperative instructions from 14 de-identified clinic notes, comparing ten LLMs with human clinicians. With international guidelines, GPT-4 reached 96.4% accuracy on fitness for surgery against 86.6% for the human answers, and produced preoperative instructions comparable to those written by clinicians, without hallucinations. The same study also shows the limits of smaller models: with retrieval, Llama2–7B hallucinated more than without it (48.6% and 32.9% with international and local guidelines respectively, against 12.8%). Two aspects, apart from the implementation itself, set the setting of the article apart from the one addressed here: the guideline corpus is topically homogeneous, since every document concerns preoperative assessment, and the target output (preoperative instructions and fitness, against discharge recommendations) and input (presurgical notes against ED). Nonetheless, the study shows that RAG can be used to ground LLMs on clinical guidelines with promising results.
These positive results need to be read alongside Kim et al.[26], in what its authors define the most extensive expert evaluation of RAG in medicine to date, which assessed retrieval, evidence selection and generation separately. Across 800 model outputs on 200 clinical queries, annotated by eighteen medical experts, only 22% of the retrieved passages were judged relevant, models were poor at selecting the useful evidence among them, and conventional RAG lowered factuality by up to 6% compared to the same models without retrieval. Simple interventions on the retrieval side, namely filtering the retrieved evidence and reformulating the query, improved performance by up to 12%. The key takeaway being that a blind application of RAG is not sufficient to guarantee improved performance, and that the retrieval step must be carefully designed to the task at hand.

Most of these works share the same starting point: a question, which also serves as the retrieval query. MIRAGE explicitly retrieves on the question alone, Almanac and the hepatitis C framework are queried with clinical questions, and only Ke et al.[30] start from a clinical note, on a narrow single-domain corpus. The setting of this thesis differs from these

works in two ways. An ER clinical note does not ask anything: it is a long, heterogeneous document in which the diagnosis coexists with comorbidities, routine examinations and negated findings, and the guideline corpus spans many specialties, so retrieval must identify which guidelines are relevant before selecting passages within them. Consistently with the findings of Kim et al.[26], the design of this system concentrates its effort on retrieval: on how the clinical note is turned into queries, and on how the retrieved candidates are filtered and reranked before generation. The following section reviews prior work on retrieval from patient descriptions.

2.3 Retrieval from Patient Descriptions

When retrieval starts from a patient description rather than from a question, the query is not given and must be derived from the note. This problem was studied systematically in the TREC Clinical Decision Support track, in which participants were asked to retrieve biomedical articles relevant to the diagnosis, test or treatment of patients described by real, de-identified admission notes[31]. Each topic was also released in two rewritten forms, a case-report-style description and a one or two sentence summary. The summary was the best-performing representation, and all the top automatic runs used it. The organizers concluded that the clinical note itself is a challenging input for retrieval systems, while also observing that clinicians cannot reasonably be expected to rewrite their notes to obtain better results.

TrialGPT[32] shows that the problem persists with modern retrievers, and that LLMs can be used to address it. To retrieve candidate clinical trials for a patient, it prompts an LLM to generate a ranked list of up to 32 keywords from the patient note, runs each keyword through a hybrid lexical and dense retriever, and fuses the results with a weight that decreases with the rank of the keyword. Using the raw note as the query instead gave the lowest recall on all three test collections: on average, recall at 500 retrieved trials was 50.0% for the raw note against 83.4% and 86.2% for keywords generated by GPT-3.5 and GPT-4, and on the one collection where they were available, keywords written by clinicians fell in between. These results were obtained on short patient summaries, averaging between roughly 90 and 160 words; early experiments on this thesis found the same effect on full ER notes, which are considerably longer.
Clinical entities have also proven to be an effective retrieval unit within clinical text. CLEAR[33] uses named entity recognition to identify the clinical entities in a note, selects those relevant to the variable to be extracted, and retrieves only the passages that contain them. On 20,000 clinical notes and 18 variables, it reached an average F1 of 0.90, against 0.86 for embedding-based RAG and 0.79 for full-note prompting, while reducing token usage and inference time by more than 70%. Although the direction of retrieval is reversed with respect to this thesis, since CLEAR searches the note rather than an external corpus, it supports the use of extracted entities, rather than the full text, as the basis of the retrieval signal.

Clinical notes also contain many negated findings, such as symptoms the patient denies or tests with negative results, which are informative for the physician but would steer retrieval toward conditions the patient does not have. Neural retrievers cannot be relied upon to handle them. On NevIR[34], a benchmark in which models must rank two documents that differ only by a negation, most retrieval models performed at or below random, with bi-encoders producing nearly identical representations for the two documents. Cross-encoders performed

better, but only the most powerful models surpassed the random baseline, and even then they performed significantly below humans and had far from ideal performance. Since the first retrieval stage of this system relies on a bi-encoder, negated entities are removed before the queries are built rather than left to the retriever. The filter, though created ad hoc for this work, follows the principle introduced by NegEx[35], which detects negation cues and applies them to the terms within a limited window.

In the general RAG literature, query reformulation is also often delegated to an LLM. HyDE[36] embeds a hypothetical answer document generated from the query, and Query2doc[37] expands the query with LLM-generated pseudo-documents. Both methods enrich short queries with additional context, whereas a clinical note poses the opposite problem: the information is abundant, and has to be condensed and split into focused queries. These studies show that the LLMs' ability to generate relevant text and adapt to different input structures can be leveraged to improve retrieval, and the experiments in this thesis show that it is also effective when applied to clinical notes.

The query construction of this thesis supports both approaches. A lightweight generator combines entity extraction, negation filtering and corpus-based term weighting into several focused sub-queries without any LLM call before retrieval, which keeps the step deterministic, fast and easy to run on modest hardware. Alternatively, the sub-queries can be generated by an LLM, the same model that produces the final recommendations, at the cost of additional latency.

2.4 Machine Translation in Clinical Pipelines

The system described in this thesis translates at both ends of the pipeline: Italian clinical notes are translated into English before retrieval, and the generated recommendations are translated back into Italian before reaching the physician and the patient. The two directions raise different questions, and are discussed separately.

On the input side, translating a non-English text into English in order to apply English models, known as translate-test, is the classical alternative to cross-lingual transfer with multilingual models. Gaschi et al.[38] compared the two strategies for clinical named entity recognition in French and German, without any annotated data in those languages. Translation-based methods reached performance comparable to cross-lingual transfer and were preferable with small models, but proved sensitive to the choice of translation model, and in translate-test any translation error is reflected directly in the downstream output. The same study also found that domain-specific monolingual models did not consistently outperform large general-purpose multilingual ones. In this work, the sensitivity of translate-test to translation errors is addressed with a custom glossary that fixes the translation of section headers and of clinical abbreviations that a general-purpose translator handles poorly (Section 6.3.4). This allows the system to operate on less powerful models, which are more affordable and easier to run on local hardware, while still achieving good performance.

On the output side, machine translation of emergency department discharge instructions has been evaluated directly. Khoong et al.[39] translated 100 sets of ED discharge instructions with Google Translate and found 92% of sentences accurately translated into Spanish and 81% into Chinese, with potential for clinically significant harm in 2% and 8% of sentences respec-

tively. Similarly, a more recent comparison on 50 sets of patient-specific ED instructions[40] found sentence-level accuracies of 97% for GPT-4 and 96% for Google Translate in Spanish, and of 95% and 90% in Chinese; GPT-4 was significantly more accurate in Chinese and Russian, with a similar potential for harm.

None of these studies evaluated DeepL, the service used in this work, or Italian as a target language, so their figures are indicative rather than directly transferable. Two observations nonetheless apply. First, translation quality depends on the readability of the source text, which aligns translation safety with the rules already imposed on the generated recommendations: short sentences without medical jargon. Second, LLMs can match or exceed dedicated machine translation services, which is relevant to the choice between translating the output with an external service or with the generating model itself (Section 7.2). In either case, the translated recommendations are reviewed by the physician before reaching the patient, consistently with the caution expressed by the studies above.

2.5 Evaluating Clinical Text Generation

Current evaluation practices are heavily dependent on automated metrics such as accuracy, ROUGE, BLEU, or concept coverage score. However, their validity for clinical text is limited since they correlate poorly with clinical usefulness and safety[41]. In fact, in the reader study of Van Veen et al.[22], the correlation between NLP metrics and physician ratings was low, around 0.2 in magnitude: semantic and concept-based metrics tracked correctness best, while BLEU and ROUGE-L tracked completeness, which the authors attribute to longer outputs offering more surface for overlap. Automatic metrics can still work when the reference is a good target. On MedAlign, where clinicians wrote reference answers specifically for the evaluation, the best metric, COMET[42], agreed with clinician rankings almost as well as clinicians agreed with each other[43]. In this task, however, the only available reference is the discharge text written by the physician under time pressure, which the system is explicitly meant to extend (Section 2.1) and rewrite, so a correct recommendation missing from the reference would lower the score.

Expert evaluation is therefore the reference standard, and recent studies converge on a small set of axes. Singhal et al.[21] had physicians rate long-form answers for agreement with scientific consensus, possibility and extent of harm, and missing or inappropriate content, among other axes; Van Veen et al.[22] asked readers to judge completeness, correctness and conciseness, and connected errors to potential medical harm; Williams et al.[13] counted inaccuracies, hallucinations and omissions, and rated the potential harm of each error on an adapted version of the AHRQ Common Format Harm Scale. Expert evaluation is also costly, which keeps samples small. Moreover, experts often disagree with each other: when 149 physicians compared answers to real clinical questions in their own specialty, agreement between raters was low to moderate on most quality axes[44].

LLM judges have been proposed as a scalable substitute. In the general domain, Zheng et al.[45] found that strong LLM judges agreed with human preferences about as often as humans agreed with each other; still, they found important self-enhancement biases. Clinical evidence is mixed. On a German open-response benchmark, the best LLM evaluator reached a κ of 0.694 with physicians, close to the inter-physician ceiling of 0.709, yet it never abstained on difficult items, unlike physicians, and models tended to favour outputs from related model

families[46]. On real point-of-care queries, LLM judges agreed with physicians on which system was best but not on the remaining ranks, showed self-preference, and a jury of three frontier models reached an average Kendall's τ of -0.067 with the expert ranking[44].
LLM judges are therefore better suited to narrow, well-specified judgments than to the assessment of complex clinical correctness.

Consistently with these findings, the evaluation of the final recommendations, described in Chapter 7, relies on human judgment rather than on overlap with the original discharge text. NLLM-as-a-judge was used only in the retrieval phase, where judging the relevance of thousands of excerpts was not delegable to physicians. Given the known limitations of LLMs in judging clinical text, the judge was asked a deliberately simple question: whether an excerpt is relevant to a given output or not, rather than how relevant it is. Thus, the task becomes one of assessing textual pertinence rather than one of clinical reasoning.

Chapter 3

Theoretical Framework

3.1 Transformer-Based Language Models

Most of the system, from the model that embeds guideline excerpts to the one that writes the final recommendations, is built on the same neural architecture: the transformer. In this section, we introduce its architecture and the two families of models derived from it that are relevant to this work: Encoder models, discussed in Section 3.1.2, are used to produce contextual representations of text and to underpin the retrieval components; while Decoder models, discussed in Section 3.1.3, generate text and include the Large Language Models.

3.1.1 The Transformer Architecture

The transformer, introduced by Vaswani et al.[47] in their pivotal paper *Attention is All You Need* with the goal of creating a more efficient model for machine translation, replaced the recurrent networks that dominated sequence modelling at the time with a mechanism based entirely on attention. In the following years, with transformers consistently outperforming recurrent architectures in direct comparisons, in multilingual machine translation[48] as well as in speech recognition[49], they have since become the dominant architecture in natural language processing[50].

The input of a transformer is a sequence of tokens, typically subword units, obtained through algorithms such as byte-pair encoding. Splitting rare words into frequent fragments keeps the vocabulary to a manageable size while still allowing any word to be represented, including the specialised terminology of medical text[51]. Each token is mapped to a learned embedding vector, and information about its position in the sequence is added, since attention by itself is insensitive to word order.

The core operation of the architecture is what the authors of the attention paper call self-attention[47]. For each token, three vectors are computed through learned linear projections: a query, a key and a value. Stacking these vectors for all tokens of the sequence into the matrices Q, K and V, the output of the attention layer is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{3.1}$$

where d_k is the dimension of the keys. Intuitively, each token compares its query with the keys of all tokens in the sequence, and the resulting weights determine how much each

token's value contributes to its new representation. As a result, the representation of a word depends on its context, and the same word receives different vectors in different sentences[52]. The token *discharge*, for instance, is represented differently in *hospital discharge* and in *digital discharge*. In practice, several attention operations run in parallel, a mechanism called *multi-head attention*. Analyses of trained models show that different heads specialise in different relationships between tokens, such as specific syntactic dependencies[53].

A transformer layer combines multi-head attention with a position-wise feed-forward network, residual connections and layer normalisation, and complete models stack many such layers. Because every token attends to every other token, the cost of attention grows quadratically with the length of the sequence[47]. This cost limits the length of the sequences a model can process, so each model is trained with a maximum input length, its *context window*[54]. This limit has affected the building of the system in various stages of the pipeline, from the size of guideline chunks to the length of the generation prompt.

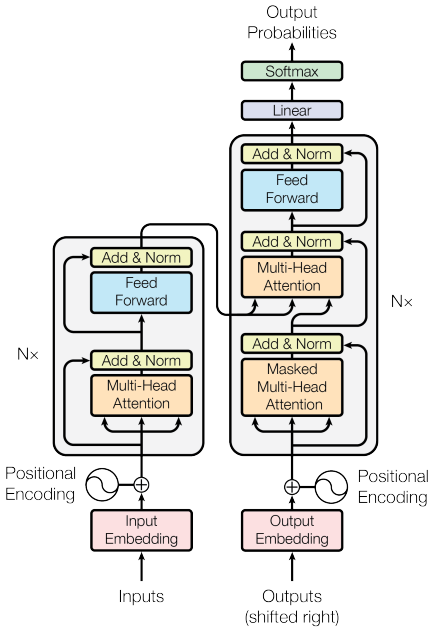


Figure 3.1. The transformer architecture, with the encoder stack on the left and the decoder stack on the right[47].

The original transformer pairs an *encoder*, which reads the input sequence, with a *decoder*, which generates the output one token at a time while attending to the encoder's representations[47] (Figure 3.1). This encoder-decoder configuration is still used for sequence-to-sequence tasks, including large-scale machine translation[55], but subsequent work showed that each half can also be used on its own. Encoder-only models let every token attend to the whole input and are suited to producing representations of text[56], while decoder-only models apply a causal mask, so that each token can only attend to the tokens preceding it, and are suited to generating text[57].
In our system the embedding model, the entity recogniser and the reranker are all encoders, while the generator is a decoder-only model.

3.1.2 Encoder Models

While powerful, the encoder-decoder architecture is computationally intensive and introduces latency, since the decoder must wait for the encoder to complete its processing[58]. Encoder-only models simplify this by removing the decoder. A pivotal example is BERT (Bidirectional Encoder Representations from Transformers)[56], which following its release quickly became a ubiquitous baseline in natural language processing[59]. Its key contribution is a pretraining objective that exploits bidirectional context. In *masked language modelling*, a fraction of the input tokens is hidden, and the model learns to predict them basing itself on the words on both sides of it. Since this objective requires no annotation, the model can be pretrained on large unlabelled corpora[56]. Probing studies show that, through this pretraining, the model acquires substantial syntactic and semantic knowledge, which is stored in its weights[59]. A pretrained encoder is then adapted to a downstream task by adding a small task-specific layer on top of its output and *fine-tuning* the whole network on a much smaller labelled dataset.

The same pretrained backbone can thus serve very different purposes, three of which appear in this work: classifying each token through an entity recogniser; classifying a pair of texts fed jointly to the model through a relevance scorer, the cross-encoder; pooling the token representations into a single vector through a sentence embedding model.

In the years following BERT's introduction, encoder architectures have been updated with improvements first developed for large generative models[60]. ModernBERT, for instance, extended the context window from the 512 tokens of the original BERT to 8,192 tokens, and is designed for efficient inference on common GPUs[60]. This allows for both longer passages to be encoded without truncation and more efficient training and inference.

Domain-specific pretraining. General-purpose encoders such as BERT are pretrained on generic sources such as Wikipedia and books[56], whose word distribution differs considerably from that of more specific domains like that of the biomedical corpora[61]. Two strategies have been proposed to close this gap. The first continues the pretraining of a general model on in-domain text: BioBERT, for example, starts from BERT and is further trained on PubMed abstracts and PubMed Central full-text articles[61]. The second pretrains a model from scratch on in-domain text only, with a vocabulary built from the same corpus. Gu et al.[62] showed that this second approach, used to build PubMedBERT, outperforms continued pretraining across a broad range of biomedical tasks. They also noted that a domain-specific vocabulary represents many biomedical terms as single tokens rather than fragmenting them into uninformative pieces.

Domain also matters at a finer granularity than "biomedical" versus "general". Clinical notes differ linguistically from the biomedical literature, and models further trained on clinical notes perform better on several clinical tasks than models trained on literature alone[63]. This distinction is relevant to the present system, in which queries originate from clinical notes while the documents to be retrieved are guidelines written in formal prose (Chapter 5). Domain-specific pretraining can also improve parameter efficiency: Lehman et al.[64] found that relatively small models pretrained on clinical text match or outperform much larger general-domain models on clinical tasks.

3.1.3 Large Language Models

Decoder-only models are trained with a *causal language modelling* objective: given an unsupervised corpus of tokens $\mathcal{U} = \{u_1, \ldots, u_n\}$, predict the next one[57]. For a sequence of

tokens $u_1, \ldots, u_T$, the model learns the factorisation

$$L(\mathcal{U}) = \sum_i \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta), \tag{3.2}$$

Where k is the size of the context window, and the conditional probability P is modeled using a neural network with parameters Θ; it generates text by repeatedly selecting a token from this distribution and appending it to the input[57]. Generation is therefore sequential: producing K tokens requires K successive passes through the model.

The term *Large Language Model* (LLM) refers to language models, typically decoder-only, scaled to a very large number of parameters and trained on vast text corpora[65]. The performance of such models improves predictably as parameters, data and compute increase[66]. At sufficient scale, LLMs also become capable of *in-context learning*: performing a task described in the input, possibly with a few examples, without any update to their weights[67]. In the following paragraphs, we cover how LLMs are adapted to follow instructions and to the medical domain, how their behaviour can be controlled through prompting, how their memory requirements can be reduced for local deployment, and their main limitation in clinical settings, hallucinations.

Instruction tuning. The objective of predicting the next token differs from that of following the user's instructions, so a model pretrained only on this objective tends to continue text rather than respond to requests. To obtain assistants that follow instructions, pretrained models are further fine-tuned on collections of tasks phrased as natural-language instructions, and then aligned with human preferences through reinforcement learning from human feedback[68]. The effect of this stage can outweigh that of scale. In the study by Ouyang et al.[68], human evaluators preferred the outputs of an instruction-tuned model to those of the original GPT-3, despite the former having over a hundred times fewer parameters.

Domain adaptation. As with encoders, medical variants of LLMs have been obtained by continuing the pretraining of general models on medical corpora[69]. Their benefits, however, appear less consistent than for encoders. Nonetheless, a systematic comparison by Jeong et al.[70] found that medically adapted LLMs do not consistently outperform their general-domain base models counterparts on medical question answering, once each model's prompt is optimised separately.

Prompting. Since the weights of an LLM are fixed at inference time, its behaviour is controlled through the *prompt*, the input text that precedes the generation. A prompt typically combines a role description, task instructions, constraints on the output format and any material the model should work with[71]. Models are sensitive to the way a prompt is written: even formatting changes that leave the meaning untouched can alter performance substantially. Since this sensitivity differs from model to model, a prompt optimised for one model is not guaranteed to work equally well with another[72]. The position of information within the prompt also matters. When relevant content is placed in the middle of a long input, models use it less reliably than when it appears at the beginning or the end[73].

Quantisation. The memory required to run an LLM is largely determined by the number of its parameters, all of which must be stored in memory during inference. Stored as 16-bit floating-point numbers, each parameter occupies two bytes, so a model with 7 billion

parameters would take up about 14 GB for its weights alone. Further memory is needed during generation for the *key-value cache*, which stores the keys and values of Equation (3.1) for all previous tokens and therefore grows with the length of the sequence[74].

Quantisation reduces these requirements by storing the weights at lower numerical precision. In post-training quantisation, the weights of an already trained model are mapped to 8-bit or 4-bit representations, using methods designed to limit the resulting loss of accuracy[75,76]. Halving the precision roughly halves the memory footprint, at the cost of a degradation that becomes more pronounced as the number of bits decreases[77]. Under a fixed memory budget, Dettmers and Zettlemoyer[77] found 4-bit precision to be almost always the best trade-off, meaning that a larger model quantised to 4 bits generally outperforms a smaller model kept at higher precision.

Hallucination. A major concern for the clinical use of LLMs, especially in the context of medical domain, is *hallucination*[11] (an issue introduced in Section 1.1), the generation of content that is fluent and plausible but not supported by facts or by the given input[78]. Two distinctions are useful to characterise it. The first, introduced for summarisation by Maynez et al.[79], separates *intrinsic* hallucinations, which contradict the source material, from *extrinsic* ones, which add information that cannot be verified from it. The second separates *factuality* hallucinations, which conflict with real-world knowledge, from *faithfulness* hallucinations, which diverge from the instructions or context provided by the user[78].

Hallucinations are not an occasional malfunction, and theoretical analyses argue that they cannot be entirely eliminated since they stem from the very way LLMs are trained and evaluated[80,81]. In fact, the causal objective of Equation (3.2) rewards statistically plausible continuations rather than true ones, and common training and evaluation procedures further reward confident guessing over acknowledging uncertainty[81]. Moreover, the knowledge of an LLM is stored implicitly in its parameters: it cannot easily be inspected or updated, and the model cannot point to the source of a statement[25]. In a clinical setting, these properties are particularly problematic. Since hallucinated content is produced with the same confidence as correct content[81], a fabricated dosage or treatment may be indistinguishable from a correct one to a reviewer with no reference to check it against.

Retrieval-Augmented Generation, presented in Section 3.2, addresses these limitations by supplying the model with external sources at inference time.

3.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines a language model with an external collection of documents, from which relevant passages are retrieved and provided to the model at generation time[25]. Lewis et al.[25] state that the knowledge stored in the weights of the model constitutes a *parametric* memory, while the document collection constitutes a *non-parametric* memory, which can be inspected and revised independently of the model. This section describes the general framework, the properties that make it suited to clinical text generation, its limitations and the extensions to the basic pipeline that are relevant to this work.

3.2.1 Formulation

A RAG system leverages two components: a *retriever* $p_\eta(d \mid x)$, which selects passages relevant to an input x from a document collection $\mathcal{D}$; and a *generator* $p_\theta(y_i \mid [x, d, y_{1:i-1}])$,

which produces the output conditioned on both the input and the retrieved passages[25]. The retriever assigns each document $d \in \mathcal{D}$ a relevance score with respect to x and returns the k highest-scoring documents, denoted $\mathcal{D}_k(x)$.

In the original formulation, the retriever and the generator are fine-tuned jointly, and the probability of an output y is obtained by marginalising over the retrieved documents[25]:

$$p(y \mid x) \approx \sum_{d \in \mathcal{D}_k(x)} p_\eta(d \mid x)\, p_\theta(y \mid x, d), \tag{3.3}$$

where p_η is the distribution over documents defined by the retriever and p_θ is the generator. Each retrieved document thus contributes to the output in proportion to its estimated relevance.

With the spread of LLMs, a simpler variant has become common: the retrieved documents are concatenated to the input, and an off-the-shelf model generates the output without any additional training[82,83]. The output is then generated as

$$y \sim p_\theta\left(y \mid [d_1; \ldots; d_k; x]\right), \qquad d_i \in \mathcal{D}_k(x), \tag{3.4}$$

where $[\,\cdot\,;\,\cdot\,]$ denotes concatenation. This *in-context* formulation decouples the two components, so that the retriever can be designed independently and any language model, including one accessible only through an API, can act as generator[82]. The system described in this thesis follows this formulation, which is what allows the generation backend to be changed without modifying the retrieval pipeline. In practice, the retrieved passages are inserted into a prompt together with the input and the task instructions (Section 3.1.3).

3.2.2 Advantages over Parametric Knowledge

An alternative way of providing a model with domain knowledge would be to fine-tune it on domain documents. However, a comparison of the two approaches found that retrieval consistently outperforms unsupervised fine-tuning for injecting factual knowledge[84]. Beyond accuracy, RAG offers three properties that are especially relevant in a clinical setting.

First, knowledge can be updated without retraining: replacing the document collection is sufficient to change what the system knows[25]. This matters when dealing with clinical guidelines, whose recommendations quickly become outdated as new evidence emerges and which therefore need periodic revision[85].

Second, the content of the output can be traced back to the passages it was derived from, so that it can be verified against its sources[86]. This property is what allows a physician to review each recommendation against the guideline excerpt that supports it.

Third, conditioning generation on retrieved evidence reduces hallucination compared to relying on parametric knowledge alone[87].

3.2.3 Advanced Retrieval Pipelines

In their 2023 paper, Gao[83] introduces the distinction between *naive* RAG, in which the input is used directly as the query and the top-ranked passages are passed to the generator, and *advanced* RAG, which adds optimisation steps before and after retrieval. This thesis's system implements several techniques to enhance retrieval, and therefore falls into the latter category.

Query formulation. A query that perfectly concides with the input can work well for short questions, but less so for long, information-rich inputs. Verbose queries are a known difficulty in information retrieval, since the terms that matter are mixed with many that do not[88]. A clinical note, the input data to this RAG system, is a clear example of such a query. Classical remedies include *query reduction*, which retains only the key concepts of the input, and *query expansion*, which adds related terms to improve recall[89]. Another strategy is to issue several queries, each representing a different aspect of the information need, and to combine their results. Early work in information retrieval already showed that combining multiple representations of the same query is an efficient method to improve retrieval effectiveness[90].

LLMs have added new options to these techniques. The input can be rewritten into a query better suited to the retriever[91], complemented or replaced by model-generated text resembling a relevant document[37], or decomposed into multiple queries whose results are then fused[92].

The system described in this thesis builds multiple focused queries from the clinical note, either through entity extraction or through an LLM (Chapter 4).

Multi-step retrieval. Some information needs cannot be satisfied by a single round of retrieval. In multi-hop retrieval, the passages retrieved in a first step are used to formulate the following queries[93]. Retrieval can also span heterogeneous sources, such as text passages and tables, which may require different handling[94].

In this thesis, we make use of a second retrieval step for tables, which are retrieved separately using terms and information found both in the clinical note and in the text excerpts returned by the first step.

3.2.4 Limitations

Retrieval reduces hallucinations but does not eliminate them. The outputs of RAG systems can still contain content that is unsupported by, or in contradiction with, the retrieved documents[95]. Citations are not guaranteed to be accurate either: in a 2023 survey of generative search engines, the authors found thatnearly half of the produced statements were not fully supported by the sources they cited[96].

A first cause of these failures can be found in the interaction between retrieved and parametric knowledge. in fact, when the retrieved context contradicts what the model learned during training, models may disregard the context and answer from memory[97]. Studies on LLMs partially reproduced this behaviour: models readily adopt external evidence that is coherent and convincing, but tend to favour their parametric knowledge when the context partly agrees with it[98]. Prompting strategies that explicitly direct the model towards the provided context have been shown to improve faithfulness[99]. This motivates the instruction, included in the prompts of this system, to rely exclusively on the provided guideline excerpts.

A second cause is the quality of the retrieved context itself. LLMs can be easily distracted by irrelevant information included in their input[100]. Together with the less reliable use of information placed in the middle of long inputs[73], these findings make the precision of retrieval, and not only its recall, a determinant factor in output quality.

3.3 Document Processing for Retrieval

Before guidelines can be retrieved, they need to be converted into units of text of moderate size that can be indexed. This is performed in three steps: extracting the text from the

source files, segmenting it into sentences, and finally grouping sentences into chunks.

3.3.1 Text Extraction from PDF

The guidelines in the corpus are distributed as PDF files (Chapter 5). PDF is a layout-based format which specifies the fonts and positions of the individual characters rather than the semantic units of the text and their role in the document. This makes the task of extracting the text non-trivial, as the structure must be reconstructed from the position of the characters, a process prone to errors such as: broken words and paragraphs; wrong reading order across columns; and headers, footers or captions mixed into the body text[101]. The task of identifying the regions of a page and their roles, such as titles, paragraphs, figures and tables, is known as *document layout analysis*[102].

Tables can be a particularly challenging case. Their meaning lies in the arrangement of cells into rows and columns, and recovering it requires a dedicated step, *table structure recognition*, beyond locating the table on the page[103]. This step matters for clinical guidelines, in which key information such as drug dosages is often reported only in tables (Chapter 5).

3.3.2 Sentence Segmentation

The extracted text is then split into sentences, which serve as the basic units for building chunks. Sentence boundary detection in machines is not as trivial as it may be to human readers, since punctuation, the most common set of identifiers for sentence ending, is also used in abbreviations, initials and numbers. The problem is more severe in medical text, which is rich in abbreviations: general-purpose tools perform noticeably worse on clinical text than on the general-domain text they were originally designed for[104].

The Punkt algorithm[105] addresses the problem without the need for manually annotated data. It learns from the target corpus itself which tokens are abbreviations, based on the observation that an abbreviation behaves as a collocation of a truncated word and the following period, and that abbreviations tend to be short and often contain internal periods. A period following a token identified as an abbreviation is not treated as a sentence boundary, unless further evidence, such as the capitalisation of the following word, indicates otherwise[105]. Since the performance of sentence segmenters degrades on domains different from those they were developed on[106], the unsupervised nature of Punkt is an advantage: it can be trained on the guideline corpus itself, learning its abbreviations directly.

3.3.3 Chunking

Whole documents are poor units of retrieval. Embedding models have a limited context window (Section 3.1.1), and long documents span several topics, of which only a small part may be relevant to a query. Documents are therefore divided into shorter passages, or *chunks*, which become the units of retrieval.

Determining the size of the chunks means necessarily running into a trade-off: small chunks are focused but may lack the context needed to interpret them, while large chunks preserve context better but can mix relevant and irrelevant content[83]. Chen et al.[107] showed that the choice of retrieval unit has an effect on both retrieval and downstream question answering, with finer-grained providing generally better performances over passage-level units.

The simplest chunking strategy splits text into windows of a fixed number of characters or tokens, with the important caveat that it may lead to sentences or words abruptly cut in half[83]. Structure-aware strategies instead aim to respect natural boundaries, such as sentences or sections, while *semantic chunking* places boundaries where the topic of consecutive sentences changes, as measured by their embeddings. However, a systematic evaluation found that the computational costs associated with semantic chunking are not justified by consistent performance gains when compared with simpler approaches[108]. Finally, consecutive chunks can be made to overlap, so that information spanning a boundary is not lost[83].

The system described in this thesis combines these concepts, as chunks are built by grouping complete sentences up to a target length, with a small overlap between consecutive chunks, while tables are kept as separate units. Each chunk retains the identity of its source document and its page range, which is what makes the traceability discussed in Section 3.2.2 possible.

3.4 Clinical Information Extraction

Clinical notes are written as free-text narratives, in which the information relevant to a given task is embedded among many other details. Extracting structured information from such text is the goal of clinical information extraction[109]. For the construction of the retrieval queries, two tasks are necessary: recognising the clinical concepts mentioned in the note, and determining whether each of them actually applies to the case in question. Most resources for these tasks, including annotated datasets and pretrained models, are available exclusively in English, while other languages receive significantly less coverage[110]. In this system, extraction is performed on the English translation of the note, which, among other advantages that led to this design choice, gives access to these resources.

3.4.1 Named Entity Recognition

Named Entity Recognition (NER) is the problem of locating and categorizing important nouns and proper nouns in a text[111]. Originally defined for general categories such as persons, organisations and locations, it has been adapted to the clinical domain, where the categories of interest include medical problems, tests, and treatments[112].

NER is usually framed as a sequence labelling problem. Given a sequence of tokens $x_1, \ldots, x_n$, the model assigns to each token a label y_i from a set that combines the entity categories with positional prefixes.

Earlier NER systems relied on building probabilistic models to segment and label sequence data[113]. This approach was later replaced by neural architectures that are able to learn features directly from text[114]. Today, the dominant approach is to fine-tune a pretrained transformer encoder, as described in Section 3.1.2, with a token classification layer[115]. Since encoders operate on subword tokens, the label of a word is typically predicted from the representation of its first subword[56]. In the clinical domain, encoders pretrained on clinical text have been shown to improve concept extraction over general-domain ones[63].

3.4.2 Negation Detection

NER extracts every mention of a concept, regardless of whether the concept applies to the patient. Clinical narratives, however, frequently mention findings and diseases in negated

form[35], as when a patient *denies chest pain* or a routine test is performed and reported as negative. For retrieval, such mentions constitute a source of noise. Using a negated finding as a query term retrieves guideline passages about conditions the patient does not have, which is precisely the kind of related but non-relevant context shown in Section 3.2.4 to harm generation.

A classical approach to this problem is NegEx[35]. The algorithm relies on a list of negation *triggers*, such as *no*, *denies* and *without*, and marks a concept as negated if it falls within the *scope* of a trigger, originally defined as a window of a few words. It also handles triggers that follow the concept they negate, and *pseudo-triggers*, such as *no increase*, which contain a negation word without expressing a negation[35]. ConText later extended the approach, ending the scope at the end of the sentence or at *termination* terms such as *but*, and adding the detection of hypothetical and historical findings and of findings referring to someone other than the patient[116].

3.5 Text Representations

Retrieval requires representing queries and documents in a form that allows them to be compared. Two families of representations are commonly distinguished: *sparse* representations, whose dimensions correspond to the terms of a vocabulary, and *dense* representations, learned vectors of much lower dimension[117]. Both appear in this system, with different roles: dense representations are used for retrieval, while sparse statistics are used to filter the terms from which queries are built.

3.5.1 Sparse Representations

In the vector space model, a text is represented as a vector with one dimension per term of the vocabulary, whose value represents the importance of that term in the text[118]. Since any given text contains only a small fraction of the vocabulary, most components are scored as zero. This sparsity allows such representations to be searched efficiently through inverted indexes, which list for each term the documents containing it[119].

The question of how to score the importance of a term has been studied extensively, as the solution is not as trivial as it might appear at first glance. In fact, raw frequencies favour words that are common across the whole collection and are therefore poor at distinguishing documents. Sparck Jones[120] proposed to weight terms by their *specificity*, that is, inversely to the number of documents in which they appear. This idea is formalised by the *Inverse Document Frequency* (IDF), defined as:

$$\mathrm{idf}(t) = \log \frac{N}{\mathrm{df}(t)}, \tag{3.5}$$

where N is the number of documents in the collection and $\mathrm{df}(t)$ the number of documents containing the term t. Combined with the frequency $\mathrm{tf}(t, d)$ of the term within a document d, it forms the TF-IDF measure[121]

$$w(t, d) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t). \tag{3.6}$$

According to TF-IDF, a term receives a high weight when it is frequent in a document, but rare in the collection.

Another solution, the BM25 ranking function, refines this scheme by saturating the contribution of term frequency and normalising for document length[122]. BM25 remains to this day

a strong baseline, particularly outside of the domains on which neural retrievers are trained[123].

The main limitation of sparse representations is that they only match terms that appear literally in both the query and the document. Since people describe the same concept with different words, relevant documents can still be missed, a problem known as *vocabulary mismatch*. The problem is particularly prevalent in medical contexts, where a single concept can be expressed through synonyms, abbreviations and lay terms[124], as with *myocardial infarction*, *MI* and *heart attack*.

3.5.2 Dense Representations

Dense representations have their roots in the distributional hypothesis, according to which words occurring in similar contexts tend to have similar meanings[125]. Word embeddings learned from co-occurrence statistics map each word to a vector of a few hundred dimensions, in which semantically related words lie close to each other[126]. Contextual encoders extend this idea to words in context (Section 3.1.1), and sentence embedding models extend it to whole passages.

Dense retrieval applies these representations to search: queries and passages are mapped into a shared vector space, and passages are ranked by their similarity to the query[127]. The bi-encoder is considered to be the standard architecture for performing first-stage dense retrieval[127], and the one implemented in this system. Because passages are encoded independently of the query, their vectors can be computed once, offline, so that only the query needs to be encoded at search time[128]. Relevance is then estimated from the similarity between the query vector and the passage vectors, as discussed in Section 3.6. Dense retrieval helps mitigate vocabulary mismatch, since different expressions of the same concept can be mapped to nearby vectors, and has been shown to outperform BM25 in open-domain question answering[128].

However, a pretrained encoder is not generally able to produce good sentence embeddings out of the box. In fact, Reimers and Gurevych[129] found that averaging the outputs of BERT yields sentence embeddings that perform worse on semantic similarity when compared to averaged static word embeddings. Sentence embedding models are therefore further trained with a *contrastive* objective, which pulls the representations of related texts together and pushes unrelated ones apart. A widely used contrastive objective is the InfoNCE loss, introduced by Oord, Li, and Vinyals[130] to learn representations by predicting future observations in a sequence from their context:

$$\mathcal{L}_{\mathrm{N}} = -\underset{X}{\mathbb{E}}\left[\log \frac{f_k(x_{t+k}, c_t)}{\sum_{x_j \in X} f_k(x_j, c_t)}\right]. \tag{3.7}$$

Here, c_t is the representation of the context at time step t, and x_{t+k} is the observation k steps ahead, which acts as the positive sample. The set $X = \{x_1, \ldots, x_N\}$ contains this positive sample together with $N - 1$ negative samples drawn at random, and f_k is a positive scoring function that should be high for the positive pair and low for the negative ones. The loss is the categorical cross-entropy of identifying the positive sample within X, so minimising it raises the score of the positive pair relative to all the negatives[130].

In a retrieval setting, the context is constituted by the query q, the positive sample is a related passage d^+, and the negatives are unrelated passages $d_1^-, \ldots, d_{N-1}^-$.

In practice, the negatives are often the related passages of the other queries in the same training batch, which provides many negative examples at no additional cost[128]. Training pairs can come from labelled datasets, or be collected at scale from naturally occurring pairs of related texts, such as the titles and abstracts of scientific articles. In the biomedical domain, an example is given by Jin et al.[131], who trained the MedCPT models on pairs of queries and articles derived from PubMed search logs.

Nonetheless, dense retrievers also have weaknesses. They tend to generalise poorly to domains different from those they were trained on, where they are often outperformed by BM25[123]. They also struggle with queries centred on rare entities, which sparse methods handle through exact matching[132]. Moreover, no single embedding model performs best across all tasks and domains[133], so the choice of a model benefits from evaluation on data from the target application.

This system relies on dense retrieval with an embedding model, selected after comparing several candidates (Section 6.2.5).

3.6 Vector Similarity Search

Once queries and passages are represented as dense vectors (Section 3.5.2), retrieval amounts to finding the passage vectors closest to the query vector. This section introduces the main similarity functions that can be used to compare vectors, and the ways in which the closest vectors can be found.

3.6.1 Similarity Functions

Let $\mathbf{q}, \mathbf{d} \in \mathbb{R}^m$ be the vectors of a query and of a passage. The three most common ways of comparing them are the inner product, the cosine similarity and the Euclidean (L_2) distance:

$$s_{\text{IP}}(\mathbf{q}, \mathbf{d}) = \mathbf{q}^\top \mathbf{d} = \sum_{i=1}^{m} q_i d_i \tag{3.8}$$

$$s_{\cos}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q}^\top \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} \tag{3.9}$$

$$d_{L_2}(\mathbf{q}, \mathbf{d}) = \|\mathbf{q} - \mathbf{d}\| = \sqrt{\sum_{i=1}^{m} (q_i - d_i)^2} \tag{3.10}$$

The inner product and the cosine are similarities: in those cases, higher values indicate closer vectors On the other hand, the Euclidean distance is a dissimilarity, so lower values indicate closer vectors. The cosine depends only on the angle between the two vectors, and ignores their lengths[134], whereas the inner product grows with the norm of the vectors, so that searching by inner product is a different problem from searching for the nearest neighbours in the Euclidean space[135].

The three functions coincide, however, when vectors are normalised to unit length. If $\|\mathbf{q}\| = \|\mathbf{d}\| = 1$, the denominator of Equation (3.9) equals one, so that $s_{\cos}(\mathbf{q}, \mathbf{d}) = s_{\text{IP}}(\mathbf{q}, \mathbf{d})$ (Intuitively, since cosine similarity is the same function that also takes length into consideration). Furthermore, expanding the square of Equation (3.10) gives

$$\|\mathbf{q} - \mathbf{d}\|^2 = \|\mathbf{q}\|^2 + \|\mathbf{d}\|^2 - 2\,\mathbf{q}^\top \mathbf{d} = 2 - 2\,s_{\cos}(\mathbf{q}, \mathbf{d}) \tag{3.11}$$

so the Euclidean distance is a decreasing function of the cosine similarity. For unit vectors, ranking passages by decreasing inner product, by decreasing cosine similarity or by increasing Euclidean distance therefore yields exactly the same order. In practice, this means that a model trained with cosine similarity can be served by an index built for the inner product, provided that all vectors are normalised beforehand.

The appropriate function is the one used to train the embedding model, namely the similarity s inside the scoring function of the contrastive loss (Section 3.5.2), since this is the geometry that the training objective has shaped. Dense Passage Retrieval, for instance, was trained with the inner product[128], and Sentence-BERT with the cosine similarity[129]. Even then, similarity in the embedding space is only a proxy for semantic relatedness. In fact, Steck, Ekanadham, and Kallus[136] show that, for embeddings learned by regularised linear models, cosine similarity can produce arbitrary results depending on the regularisation.

3.6.2 Exact and Approximate Search

Given a query vector $\mathbf{q}$ and the set $\mathcal{D}$ of passage vectors, k-nearest-neighbour search returns the k passages with the highest similarity to the query:

$$\mathcal{D}_k(\mathbf{q}) = \arg\operatorname*{top\text{-}k}_{\mathbf{d}\in\mathcal{D}} \; s(\mathbf{q}, \mathbf{d}), \tag{3.12}$$

where the operator selects the k elements with the largest scores.

Exact search computes the similarity between the query and every vector in the collection. Its cost grows linearly with the size of the collection, but it is guaranteed to return the true nearest neighbours[137]. While optimised GPU implementations have made exact search considerably faster, extending the range of collection sizes for which it is practical, For very large collections its cost remains prohibitive: even with the usage of GPUs, search at the scale of billions of vectors is time consuming and instead relies on compressed, approximate indexes[137]. Classical space-partitioning index structures do not solve the problem either, since in high-dimensional spaces their performance degrades to that of a linear scan[138].

Approximate nearest neighbour (ANN) methods give up the guarantee of returning the exact neighbours in exchange for large gains in speed and memory[139]. Several strategies are available for performing this search: Partition-based methods cluster the vectors and, at query time, only search the clusters closest to the query[140]; Quantisation methods compress vectors into short codes, so that similarities can be estimated cheaply and large collections fit in memory[140]; Graph-based methods connect each vector to its neighbours and answer queries by navigating the graph[139].

The quality of an approximate index is measured by its recall, the fraction of the true nearest neighbours it returns, which is traded off against query speed[139]. Libraries such as FAISS implement both exact and approximate indexes, for both inner-product and Euclidean metrics[141].

Since approximation only pays off when the collection is large and given the size of the corpus used in this work, exact search is fast enough, and it avoids any loss of recall in the first retrieval stage. The system therefore uses an exact inner-product index over vectors that are explicitly normalised to unit length (Chapter 6).

3.7 Reranking

The first stage of retrieval is designed for recall. It must be fast enough to search the whole collection, and therefore relies on representations of queries and passages computed independently of each other (Section 3.5.2). Its ranking is consequently approximate, and its top candidates can include passages that are apparently related to the query, without actually containing the information it seeks[142]. Multi-stage retrieval addresses this by re-scoring a small set of candidates with a more accurate but more expensive model, a step known as *reranking*[143]. Given the sensitivity of generators to irrelevant context (Section 3.2.4), this step is particularly relevant in RAG systems. This section first discusses how scores coming from multiple sources can be combined into a single ranking, and then the cross-encoder, the architecture most commonly used for producing these scores in neural reranking[127].

3.7.1 Score Aggregation

When candidates are retrieved by several queries or by several retrieval systems, a passage may receive more than one relevance score. In such cases, the problem on how to combine them into one single measure arises. This issue is also known as *data fusion*[144]. Let $s_i(d)$ denote the score assigned to a passage d by the i-th of M sources, and $n(d)$ the number of sources that retrieved it. In that regard, three simple functions were proposed by Shaw and Fox[144]: the maximum of the scores (CombMAX), their sum (CombSUM), and their sum multiplied by the number of sources that retrieved the passage (CombMNZ):

$$\text{CombMAX}(d) = \max_i \; s_i(d), \tag{3.13}$$

$$\text{CombSUM}(d) = \sum_{i=1}^{M} s_i(d), \tag{3.14}$$

$$\text{CombMNZ}(d) = n(d) \cdot \text{CombSUM}(d). \tag{3.15}$$

The three functions differ in what they reward. CombSUM and CombMNZ favour passages on which several sources agree, while CombMAX favours passages that receive a high score from any single source, regardless of the others. In the original experiments, CombSUM and CombMNZ performed best[144]. Later analyses, like that of Lee[145] attributed this to the tendency of different retrieval runs to retrieve the same relevant documents but different non-relevant ones, so that agreement among sources is itself evidence of relevance. Score-based fusion assumes that the scores of different sources are comparable. However, this does not generally hold when the scores are produced by different systems, and motivates score normalisation[146].

Rank-based methods avoid the problem altogether by combining positions instead of scores. Reciprocal rank fusion[147] scores each passage as

$$\text{RRF}(d) = \sum_{i=1}^{M} \frac{1}{\kappa + r_i(d)}, \tag{3.16}$$

where $r_i(d)$ is the rank of d according to the i-th source and κ is a constant, set to 60 in the original work, which limits the influence of a single very high ranking. Like CombSUM, RRF rewards passages ranked highly by several sources, and despite its simplicity it outperformed more complex fusion and learning-to-rank methods in its original evaluation[147].

Nonetheless, relevance is not the only possible criterion for the final ranking, since utilising relevance alone can place several redundant passages at the top of the rankings. Maximal Marginal Relevance (MMR)[148] addresses this by building the ranking one passage at a time, choosing at each step the passage that adds the most new relevant information to those already selected. A passage has high *marginal* relevance if it is both relevant to the query and, at the same time, dissimilar from the passages selected before it[148]. Starting from an empty selection, MMR repeatedly adds the passage

$$\text{MMR} = \arg\max_{d \in \mathcal{R} \setminus \mathcal{C}} \left[\lambda \, \text{sim}(q, d) - (1 - \lambda) \max_{d' \in \mathcal{C}} \text{sim}(d, d') \right], \tag{3.17}$$

where $\mathcal{R}$ is the set of candidates, $\mathcal{C}$ the set of passages selected so far, and $\lambda \in [0, 1]$ controls the balance between relevance to the query and novelty with respect to the passages already selected: with $\lambda = 1$ MMR reduces to a standard ranking by relevance, while with $\lambda = 0$ it selects passages that are as different as possible from each other, disregarding relevance[148].

3.7.2 Cross-Encoders

A cross-encoder estimates relevance by processing the query and the passage together[149]. The two texts are concatenated into a single input, `[CLS]` q `[SEP]` d `[SEP]`; where `[CLS]` is a special token whose final representation summarises the whole sequence and `[SEP]` separates the two segments[56].

The difference from the bi-encoder of Section 3.5.2 lies in where the comparison happens. A bi-encoder compresses each text into a single vector before any comparison takes place (Figure 3.2, left), so that everything the model can express about their relationship must be captured by the similarity of two independently computed vectors, and the output of the quality is dependent on the quality of the compression. In a cross-encoder, instead, every token of the query can attend to every token of the passage in every layer, through the attention of Equation (3.1) (Figure 3.2, right). The model can therefore capture interactions between the two texts that a comparison of independent vectors cannot represent[150].

Negation offers a telling example. On a benchmark designed to test whether retrieval models distinguish passages that differ only by a negation, cross-encoders were the best-performing family, followed by late-interaction models and, last, by bi-encoders.[34]. Sensitivity to negation is of direct interest for the clinical domain, in which findings can often be reported as absent (Section 3.4.2).

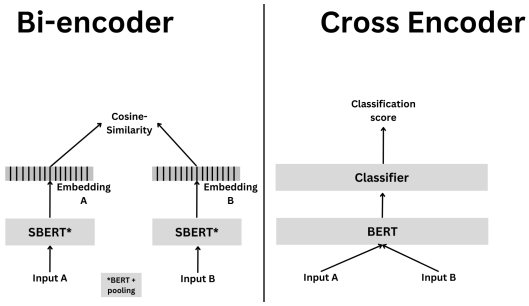


Figure 3.2. Comparison between a bi-encoder (left) and a cross-encoder (right)[151].

Cross-encoders are trained on query-passage pairs labelled as relevant or not. In the pioneering formulation of Nogueira and Cho[149], the model classifies each query-passage pair independently, and is fine-tuned with the cross-entropy loss

$$\mathcal{L}_{\mathrm{CE}} = -\sum_{d \in \mathcal{D}_q^+} \log \sigma\big(s(q,d)\big) - \sum_{d \in \mathcal{D}_q^-} \log\Big(1 - \sigma\big(s(q,d)\big)\Big), \tag{3.18}$$

where $\mathcal{D}_q^+$ and $\mathcal{D}_q^-$ are the sets of relevant and non-relevant passages for the query q, the latter drawn from the top 1,000 passages retrieved by BM25, and σ is the sigmoid function.

The effectiveness of the approach was evident from the start: applied to passage ranking, BERT-based cross-encoders produced large improvements over all previous methods[149]. Cross-encoders also generalise well beyond the domains they were trained on. In the BEIR benchmark, reranking with a cross-encoder achieved the best zero-shot performance overall, ahead of all first-stage retrievers, although at the highest computational cost[123].

However, this accuracy comes at a computational price. Since the representation of the passage depends on the query, nothing can be precomputed: every query-passage pair requires a full forward pass through the model at search time[150]. The cost of reranking therefore grows linearly with the number of candidates, which is why cross-encoders are applied only to the small pool returned by the first stage, never to the whole collection[143]. Moreover, attention over the concatenated input grows quadratically with its length (Section 3.1.1), and BERT-based models can accept at most 512 tokens, so query and passage together must fit within this window or be truncated[56,149]. The size of the candidate pool is therefore the fruit of a trade-off between the chance of recovering relevant passages ranked low by the first stage and the cost of scoring them.

The same trade-off between accuracy and cost has been shaping the more recent developments in the reranking setting. Sequence-to-sequence models can be fine-tuned to generate the relevance label as a word, such as *true* or *false*, and in this form have outperformed encoder-only rerankers, especially when little training data is available[152]. LLMs go one step further: given a query and a list of candidates, they can return the candidates reordered by relevance without any task-specific training, with results competitive with or better than those of supervised rerankers[153]. Each step towards larger models, however, increases the cost of what is already the most expensive stage of retrieval[123].

In the system described in this thesis, reranking follows the encoder-based approach: a cross-encoder scores the candidates retrieved by all queries, and the resulting scores are combined through aggregation functions (Chapter 6).

Chapter 4

System Architecture

The system takes as input the incomplete clinical note written during an Emergency Room visit and returns a draft of the discharge recommendations, along with the reasoning for providing those recommendations, grounded in a corpus of European clinical guidelines. It is organised as two pipelines that run at different times and never call each other directly (Figure 4.1).

The *guideline ingestion pipeline* runs offline, the first time the model is established and whenever the corpus changes. It is used to build what will constitute the ground truth source database for the final model. Particularly, it's responsible for parsing the guideline documents, removing content that carries no clinical guidance, splitting the remaining text into chunks and storing both their vector representations and their provenance. ables are extracted separately, and those holding structured data are kept in a store of their own. The operation of collecting the guidelines precedes this pipeline and is described in Chapter 5.

The *query processing and generation pipeline* runs once per clinical note, and is the online component of the architecture. The note, uploaded as a PDF following the department's template, is anonymised before any other processing and translated into English, to conform with the language of the corpus. It is then recomposed into several focused queries, which are embedded and used to retrieve and rerank relevant guideline excerpts and tables. A language model generates the recommendations from the note and the retrieved evidence alone, and the result is finally translated back into Italian and presented to the user for review.

The output is divided into two parts. The first is a numbered list of recommendations addressed to the patient and the general practitioner, ready to be reviewed and inserted into the discharge letter. The second justifies the rationale behind each recommendation and provides links to the evidence supporting it, to be consulted by the physician if the output is suspect. When the patient is admitted rather than discharged home, the first part is replaced by a short handover note for the admitting team.

The two pipelines communicate only indirectly, through stored files written by the guideline ingestion pipeline and read by the generation pipeline: the vector index of text chunks, the table store and the vocabulary statistics of the corpus. This separation allows the corpus to be updated or replaced without changes or disturbances to the inference pipeline, and keeps the cost of processing hundreds of documents out of the time-critical path. A web interface wraps both pipelines, allowing the physician to simply upload a note and review the

output, and allowing the corpus index to be rebuilt without the need to utilise or understand coding or the workings behind the system.

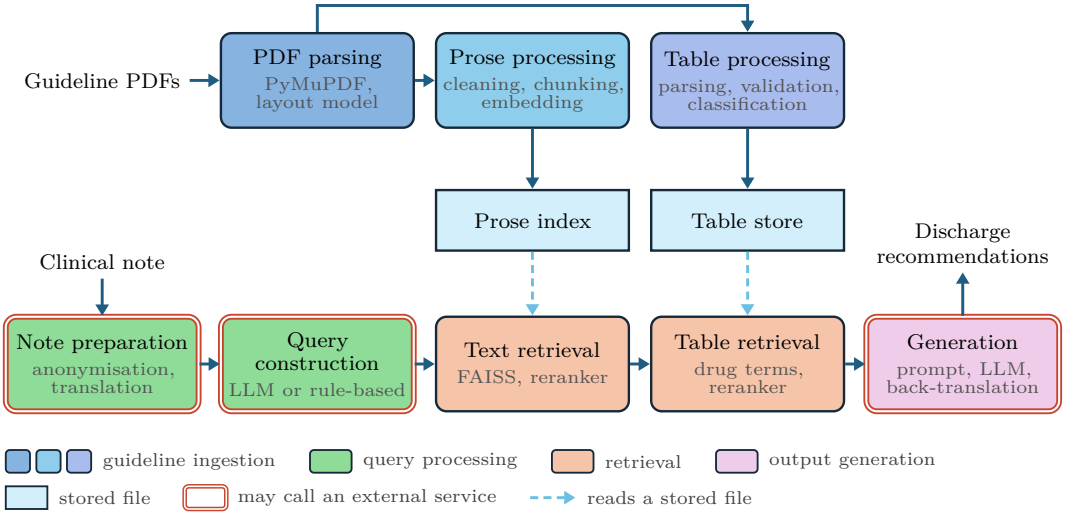


Figure 4.1. Overview of the system. The ingestion pipeline runs offline and writes the stored files; the query processing and generation pipeline runs once per clinical note and reads them.

4.1 Design Principles

4.1.1 Traceability

Ultimately, the physician retains full responsibility for the clinical notes they sign, independently from whether they have used an external model to produce or beautify them. Therefore, to be efficient, the recommendations produced by the system need to be verifiable and reviewable against their sources faster than the time it would have taken the physician to write them from scratch. In fact, while grounding generation in retrieved evidence greatly reduces the risk of unsupported content, it does not remove it[95]. Content of sensitive nature still needs reviewing, traceability is what keeps it fast.

Three design choices stem from this. First, the language model is instructed to rely exclusively on the clinical note and the retrieved evidence, and to defer to a specialist for anything that the evidence does not cover, rather than filling it from its own knowledge or guessing. Second, in the part of the output addressed to the physician, every recommendation is linked to the excerpts, tables or part of the clinical note it used to make the decision on, each carrying the source document and page range recorded at ingestion. Third, the knowledge base stores the guidelines' text as is. Summaries or paraphrases generated by a language model were initially considered, but ultimately excluded: the risk of incurring into hallucinations during the summarisation process was deemed too great, and ultimately going against the very philosophy of the model: providing clinicians with recommendation grounded in trusted and official sources.

4.1.2 Structural Rather Than Vocabulary-Based Rules

The corpus is collected automatically from the guidelines of several independent European societies (Chapter 5), which share no common format. Layout, terminology and caption positions vary from one publisher to the next, and the composition of the corpus might change with every update. Rules built around one publisher's vocabulary, such as a list of caption keywords or a pattern matching a specific evidence-grading field, works on the documents it was designed for, but may fail silently on the others.

Rules applied to the corpus therefore rely on structure or on statistics computed from the corpus itself. Structural rules use features such as the position of text on the page, the shape of a line, punctuation density, or the type of layout element a block belongs to. Statistical rules, such as treating a term as generic when it occurs in a large share of chunks, adapt automatically when the corpus changes. Conventions shared by scientific publishing as a whole, such as DOIs in reference lists, are not tied to any publisher and are treated as structural.

The philosophy applied derives from the coarse nature of structural rules when compared with vocabulary-based ones, and is deliberately conservative: it is preferrable to allow some noise to slip through, rather than discarding useful guidance, since noise can still be demoted or ignored in later stages of the pipeline.

This principle exclusively concerns the corpus. The clinical note follows a restricted set of known templates in a single language, so fixed vocabulary on the input side, such as section headings, negation cues or the translation glossary, does not carry the same risk.

4.1.3 Privacy

Clinical notes contain health data, a special category of personal data under the General Data Protection Regulation (GDPR)[17]. The guidelines are public documents containing no sensitive data, so this principle concerns the query pipeline only.

The note is anonymised in the interface before it enters the pipeline, so every later component, local or external, receives text without direct identifiers. Anonymisation is treated as a mitigation rather than a guarantee: under the GDPR, data stripped of direct identifiers still count as personal data if the patient remains identifiable by means reasonably likely to be used[17], an issue tackled more in detail in Chapter 8. External calls are therefore kept to a minimum and are optional. Note parsing, anonymisation, entity extraction, embedding, retrieval and reranking run locally in every configuration. Only translation, generation and the LLM-based construction of queries may call external services, and each has a local alternative. The degree of exposure is thus decided at deployment (Section 4.4) rather than fixed by the architecture.

4.2 Guideline Ingestion Pipeline

The scope of the ingestion pipeline is to turn the guideline documents that form the corpus, collected as described in Chapter 5, into stored representations that can be read efficiently by the query pipeline. It runs offline, and only needs to run when the corpus changes, or at the first time of implementation. Its stages are shown in Figure 4.2.

Each page is first split into two streams: the tables it contains and the running text around them. The choice of separating the two streams was made to avoid running potentially

harmful text cleaning operations on tabular content, and is further justified by the need for storing informative tables separately from text chunks (Section 6.2.4). They partially meet again only at the indexing phase, where tables classified as recommendations join the text chunks.

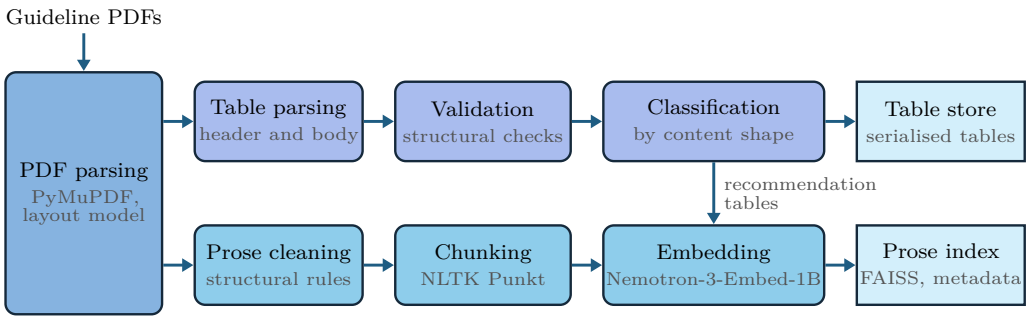


Figure 4.2. The guideline ingestion pipeline. Each page is split into a table stream and a running-text stream, which are processed separately.

4.2.1 Parsing and Region Separation

Guidelines are distributed as PDF files, which describe where text is drawn on the page rather than how it a semantic distiction describing how content is organised into paragraphs, tables and captions[101]. Each page is therefore normalised first, so that text, table cells and regions share a single coordinate system. Pages or page sections oriented horizontally, such as landscape tables printed on portrait pages, are re-rendered upright.

Tables are then located by a rule-based detector, assisted by a layout segmentation model that also finds tables drawn without borders. If the layout model fails on a document, the document is parsed again without it, so that a single problematic file loses only its borderless tables instead of stopping the build. The area of every accepted table, together with its caption, is recorded as an excluded region.

The running text is extracted from the remaining text blocks of the page, following further cleaning. Blocks that intersect an excluded region are skipped, so that the content of a table never appears twice. Images and blocks lying in the page margins, where files display running headers, footers and page numbers, are discarded. Citation markers printed as superscripts are removed at this point by their typography rather than by pattern matching, with the goal of keeping substantives that combine letters and digits, such as SGLT2, intact. This is done following the conservative philosophy introduced in Section 4.1.2.

4.2.2 Table Processing

Tables are handled separately because they crucial information, namely drug choices and dosage recommendations, were found almost exclusively in tables. A first approach of flattening a table into prose lead to separation of each value from the row and column that give it meaning and formatting issues rendering the text illegible for both humans and the final

generation model.

Each table is parsed from top to bottom and divided into three parts: caption, header, and body. The parser uses typography, rules, and position as cues to determine where each element stands; for example it might look for a row styled differently from every body row, or a line separating the header from the body. Captions are also recognized by the labels common across scientific publishing, though the usage of templates of a particular publisher (Section 4.1.2) is excluded. Parsing also refactors cells that spanned multiple rows.

Candidates are then validated, since the detector is fallible and can select text that is not a table. Candidates with too few rows or columns, or whose content looks like running text, are returned to the running text corpus. Tables of contents, abbreviation lists and listings of studies are dropped together with their text, since they carry no guidance. Reference lists are returned to the running text, where they undergo the cleaning process described in Section 4.2.3. Accepted tables are then assembled.

Finally, each table is classified by the shape of its content, with each class having a different destination:

- **Recommendation tables**, whose rows are mostly full sentences, are effectively prose laid out in a grid. They are split into text chunks of the same size as those built from running text, each carrying the caption and, where present, the strength of recommendation and level of evidence, and they are sent back to the prose index.
- **Data tables**, whose rows are short values such as drugs with their doses and contraindications, are serialised in a format that preserves rows and columns. These are the table containing the important information, and are therefore stored in the table store together with short textual surrogates of the table and of its rows and a set of normalised drug tokens, which are used to match it at query time (Section 4.3).
- **Numeric/Junk tables**, made of numbers without units, and tables of studies or epidemiological data, typically report statistical results with no actionable content for discharge, and are dropped entirely.

4.2.3 Prose Cleaning

Guidelines contain large amounts of text that serves no purpose in clinical guidance: reference lists, tables of contents, abbreviation lists, citation markers and captions. During development this text proved to be actively detrimental during the retrieval phase: being dense in clinical terms, it obtained high similarity scores for many queries and replaced the relevant excerpts in the final selection.

Cleaning removes it at three levels, each matching the unit at which a kind of noise can be recognised. In line with Section 4.1.2, every rule is conservative, and text whose nature is uncertain is kept.

At block and page level, during parsing, reference lists are recognised by the density of journal locators and author lists, and abbreviation lists by acronyms standing alone on their lines in alphabetical order. Both are removed block by block, so that guidance printed on the same page survives. Pages dominated by bibliography or by the structure of an index are removed whole.

At line level, citation numbers attached to words or punctuation are stripped, and lines garbled by broken font encodings, which often come from parsing failures in figures, are discarded. This is done with careful attention as to not remove medical glossary terms, such as CHA2DS2-VASc, which may be recognized as reference numbers in coarse cleaning.

At sentence level, after segmentation, copyright notices, packed summary lists, and figure and table captions are removed. Captions are handled here because a caption becomes a distinct unit only once the text has been split into sentences.

4.2.4 Segmentation and Chunking

The cleaned lines are merged and split into sentences by a tokenizer trained without supervision on the corpus itself. The tokenizer learns the abbreviations used in the guidelines instead of requiring a list of them. Sentences too long to fit in a chunk are split at punctuation.

Chunks are formed by joining consecutive sentences up to a fixed size, with an overlap of a few whole sentences between adjacent chunks so that context is not lost at the boundaries. Building chunks from sentences ensures that no boundary falls inside a sentence, and no chunk crosses from one guideline into the next. Each chunk records its source document and page range, which the output later uses to cite it (Section 4.1.1).

4.2.5 Indexing and Stored Data

The pipeline ends by writing five files:

1. The *prose index* holds an embedded vector for every text chunk, including those built from recommendation tables. Vectors are computed by a sentence encoder, and the same encoder embeds the queries at inference time. They are normalised to unit length and searched exactly by inner product (Section 3.6.1). Approximate search is not needed, since the corpus is small enough for exact search to remain fast.

2. The *chunk metadata* store the text, source document and page range of each vector.

3. The *table index* holds an embedded vector for the surrogate of every data table, computed and searched in the same way as the prose index.

4. The *table store* holds the data tables with their serialisation, surrogates, drug tokens, caption and provenance.

5. The *corpus statistics* record the vocabulary of the chunks and the share of chunks in which each term occurs. The query pipeline uses them to tell terms that are distinctive within the corpus from generic ones (Section 4.3).

4.3 Query Processing and Generation Pipeline

The query pipeline runs once for each clinical note, from the moment the physician uploads it to the interface to the moment the output is produced and displayed. It reads the stored files written by the ingestion pipeline, without ever modifying them. Its stages are shown in Figure 4.3.

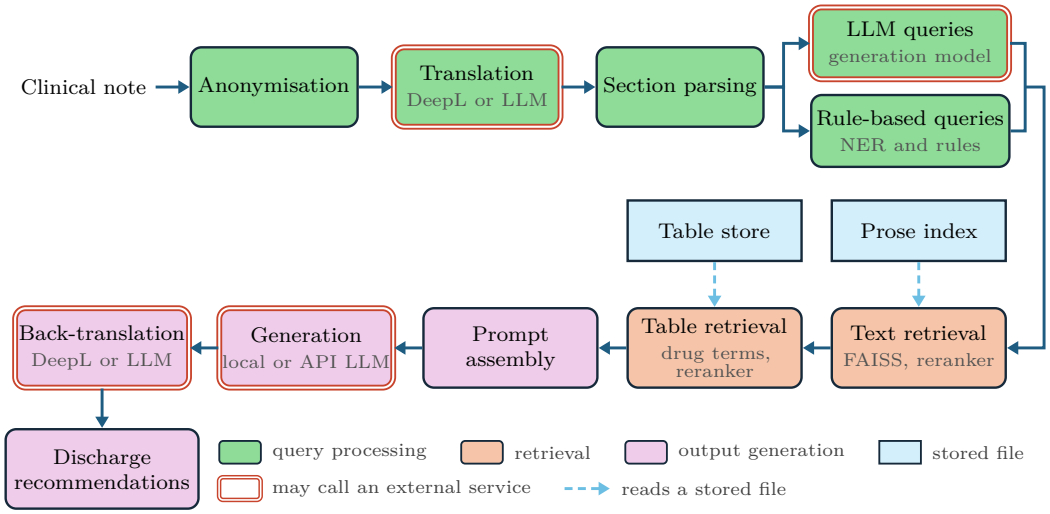


Figure 4.3. The query processing and generation pipeline. The note is anonymised and translated, decomposed into queries, and used to retrieve text excerpts and then tables, which ground the generated recommendation.

4.3.1 Input Acquisition and Anonymisation

The system takes as input the clinical note, exported as a PDF from the department's software. This design choice was made after consultations with physicians, which identified it as the most accessible method aside from a complete system integration with the existing software. These documents follow a fixed template, whose sections are produced during every Emergency Room visit: medical history, objective examination, clinical diary, diagnosis and prognosis. The interface checks that an uploaded file contains the markers of this template and rejects it otherwise, since every later stage relies on the sections being identifiable, and an unrecognized note might insert data that can't be anonymised by the system. Plain text following the same structure is accepted as a fallback, though not supporting sensitive data.

Before any other processing, the note is anonymised, so that no later component receive direct identifiers (Section 4.1.3).

4.3.2 Translation and Section Parsing

The notes are written in Italian, while the corpus is in English, and the models used for entity extraction, embedding and reranking are also monolingual in English. The note is therefore translated into English before retrieval to match the rest of the pipeline. After generation, the output is then translated back into Italian so that it can be inserted into the clinical note. References are also translated in Italian, since it is possible that the physician handling the system is not well versed in English.

Translation can be performed by a dedicated machine translation service or by the same language model that generates the recommendations. The second option removes one external dependency, when using a local model, and allows translation to run locally whenever generation does (Section 4.4).

Both options share a glossary, which serves three purposes. It fixes the translation of the section headings, so that the sections can be located in the translated text independently of the engine that produced it, avoiding potential detection issues. It expands abbreviations

common in Italian clinical notes that a general-purpose translator would not resolve (e.g. *f.c.* is mapped to *heart rate*). It maps brand names of drugs, frequent in Italian notes, to the names of their active substances, which are the names used by the guidelines in the corpus.

The translated note is then split into its sections using the fixed headings. The diagnosis anchors the queries built in the next stage, and the prognosis determines which output template is used (Section 4.3.6).

4.3.3 Query Construction

A clinical note covers several clinical aspects at once: the main diagnosis, comorbidities, the drugs the patient already takes and those administered during the visit. Early experiments showed that embedding the whole note as a single query merged these aspects into one vector, leading to sub-par retrieval. This is due to the nature of both the clinical note and the embedding: retrieval favored matching through generic identifiers and focused greatly on the anamnesis (the first part of the note), which contains few important information compared to the amount of noise. The note is therefore decomposed into several short queries, each targeting one specific aspect, and retrieval is performed independently for each.

Two interchangeable modes build these queries. Both produce the same kind of output, so the later stages do not depend on which one is used.

In the *LLM-based mode*, a language model reads the translated note and writes what it believes are the ideal queries. This mode adapts to notes of any wording without hand-built rules, at the cost of an additional model call.

In the *automatic mode*, queries are assembled without a language model. A named entity recognition model extracts diseases, medications and diagnostic procedures from the note, and entities that appear in negated form, such as a symptom the patient denies or a negative test, are discarded. The remaining entities are filtered with the corpus statistics written at ingestion: terms that occur in a large share of chunks are treated as generic and dropped, and terms absent from the corpus vocabulary are corrected or discarded. The surviving entities, the sentences of the note in which they occur and the diagnosis are then combined into queries, alongside a broad query on the diagnosis.

The automatic mode keeps the system usable when no language model can be called for this purpose, whether for privacy or for cost.

4.3.4 Text Retrieval and Reranking

Each query is embedded with the same encoder used at ingestion and matched against the prose index, which returns a fixed number of nearest chunks by cosine similarity (Section 3.6.1). The candidates of all queries are pooled, duplicates are removed, and each candidate records which queries retrieved it.

The pool is then reranked by a cross-encoder. A cross-encoder reads each query together with each candidate, which makes it more precise than comparing independently computed vectors but too expensive to apply to the whole corpus. The first stage is therefore tuned for recall, and the second for precision.

Each candidate is scored only against the queries that retrieved it, so a chunk can rank highly only for a query that both encoders consider it relevant to. A candidate that was retrieved by several queries takes the highest of its scores: the maximum rewards a chunk that is highly relevant to a single aspect of the note, whereas averaging would favour generic chunks moderately related to all of them. Near-identical chunks are then removed, keeping the higher-scored one.

The final selection reserves one place for each query, starting from the query with the strongest match. Without this, the aspect with the most guideline coverage, typically the treatment of the main diagnosis, could fill the whole selection and leave secondary aspects without coverage. The remaining places are filled by score.

4.3.5 Table Retrieval

Tables are retrieved in a second step, after the text. This is done because a table of doses is considered relevant when the drugs it lists are relevant to the case. The retrieved excerpts, together with the clinical note itself, are the best available indication of which drugs these are.

Candidates are gathered in two ways. A lexical filter selects the tables whose normalised drug tokens match drugs or drug classes named in the note or in the retrieved excerpts, with priority to the note. A single broad dense query, matched against the table surrogates, adds tables related to the case as a whole.

The candidates are scored by the cross-encoder and fill a number of places of their own, separate from those of the excerpts, since a table and a passage of prose are not directly comparable.

4.3.6 Prompt Routing and Assembly

What the document must contain depends on where the patient goes next. A patient discharged home needs instructions to follow: therapy, lifestyle changes, follow-up visits and the symptoms that should prompt a return. They constitute the main reason for which this system was built. On the other hand, a patient admitted to a ward is handed over to another team, which takes charge of therapy and follow-up, so the document only records the admission and what the patient has been told. This is a scenario already mainly able to being handled by existing templates, and was added only for completeness. The system detects which case applies from the prognosis section and selects one of two prompt templates accordingly.

Both templates share the same structure: a role definition, the clinical note, the retrieved excerpts, the retrieved tables and the task instructions. Each excerpt and table is labelled with an identifier, its source document and its page range, which the model must use to cite it (Section 4.1.1).

The template for home discharge asks for a numbered list of short, actionable recommendations addressed to the patient, without explanations, followed by the references for the physician. Prescriptions must report drug name, dose, route and frequency as given in the note or in the evidence, and points the evidence does not explicitly cover, such as the duration of a therapy, are deferred to the specialist. Referrals, the conditions for returning to the Emergency Room and the closing referral to the general practitioner follow fixed phrasings, matching the conventions of the department's discharge letters.

The template for admission asks for a short handover note and explicitly forbids prescriptions and follow-up, which are the responsibility of the admitting team.

4.3.7 Generation and Output

The assembled prompt is passed to the generation model. The generation back end is interchangeable: the same prompt can be sent to a model served through an external interface or to one run locally, with no change to the earlier stages (Section 4.4).

The output is translated back into Italian through the same path used for the input and displayed in its two parts. The first part is ready to be inserted into the discharge letter once the physician has reviewed it. The second lists, for each recommendation, the identifiers of the supporting excerpts and tables with a short justification, so that the physician can check each recommendation against its source before committing to it.

4.4 Deployment Configurations

While the architecture fixes what each stage does, it does not establish where it runs, or which model performs it. A hospital may have an interest in setting strict requirements on hardware, cost and the handling of patient data that the deployment must satisfy. This means that the configuration that produces the optimal results is not always correspondent to the configuration that is most suited for the job. To adapt to those circumstances, the system was built to accomodate different technologies at various stages of the pipeline.

Anonymisation, parsing, entity extraction, embedding, retrieval and reranking always run locally, using models small enough for modest hardware. Nonetheless, three components can instead be configured:

- query construction, which can use either the LLM-based or the automatic mode;
- translation, which can be performed by a dedicated service or by the generation model;
- generation, whose model can be served by an external provider or run locally.

Each choice moves the system along three dimensions: cost, including the hardware it requires; privacy, meaning the amount of patient data that leaves the site; and performance, meaning the quality of the output and the time needed to produce it. These dimensions pull in different directions. The models that produce the best recommendations are among the largest (Chapter 7), and they either require significant hardware capabilities or are available only as external services. Conversely, the configurations that keep all data on site are limited to the models that local hardware can run. Table 4.1 summarises how each option affects them.

Performance. Performance covers both the quality of the recommendations and the time needed to produce them, and both are dependent mostly on the generation model (Chapter 7). An external provider gives access to the strongest available models, including proprietary ones that cannot be run locally. A local model is bounded by the memory of the available GPUs, which must hold the generation model together with the retrieval models; Chapter 7 compares two such budgets, corresponding to a single high-end workstation GPU and to a multi-GPU server. There is no realistic option of generation without a GPU. In this setting,

Table 4.1. Options for the configurable components and their effect on cost, privacy and performance. All other components run locally in every configuration.

Component	Option	Cost	Privacy	Performance
Query construction	Automatic	None	No data leaves	Slightly worse
	LLM-based	As Generation	As Generation	Slightly better
Translation	Dedicated service	Service fees	Data in external servers	Excellent
	Generation model	As generation	As generation	Depends on the generation model
Generation	Local, CPU	None	No data leaves	Retrieval Only
	Local, GPU	Hardware	No data leaves	Bounded by GPU memory
	External provider	Few cents per document	Data in external servers	Strongest available models

the model can run a reduced retrieval phase, but users would still need to rely on external providers for the final output.

The other components weigh less on performance. The retrieval models were chosen to be small (Section 4.2.5), so running them without a GPU increases latency but does not prevent their use. The only exception is the reranking phase, which depending on CPU power needs to be either reduced (less excerpts up for reranking) or eliminated altogether. The automatic query mode needs no language model at all, while the LLM-based mode adds one model call before retrieval. Translation by a dedicated service is independent of the generation model, whereas translation by the generation model inherits its quality and speed.

Privacy. Each component that runs externally sends the anonymised note out of the site. When all three configurable components run locally, no patient data leaves the site, and anonymisation acts as an additional safeguard rather than as the main protection. However, while it reduces this exposure, it does not completely remove it (Chapter 8).

Cost. Cost takes two forms. Local components require hardware, whose cost is paid upfront and does not depend on the number of documents processed, while external services are paid per use, so their cost grows with the number of documents. Running locally without a GPU is the only option with no cost beyond the machines a department already has, but it is also the one that limits performance the most.

What stays fixed. Independently of the configuration used, the corpus, stored artifacts, retrieval stages and prompt templates remain identical. This has two consequences. First, a single build of the corpus serves every configuration, so moving from one to another requires no change to the ingestion pipeline. Second, since only the configurable components differ, differences in output between configurations can be attributed to them, which is the basis of

the comparison in Chapter 7.

Ultimately, the system must face a simple reality: cost, privacy and performance cannot all be optimised at once. A configuration can strongly secure at most two of them, while the third becomes a matter of damage limitation.
Strong models kept on site require expensive hardware; strong models at low cost are offered only by external providers, which then receive potentially sensitive data; and a free, fully local deployment is limited to the models that a machine without a GPU can run.

Chapter 5

The Guideline Corpus

This chapter describes the guideline corpus, a collection of clinical guidelines that constitute the ground truth on which the system bases its recommendations, and the process behind its building. Since the final generator is instructed to rely on the retrieved excerpts alone (Section 4.3.7), a recommendation can only be as complete as the corpus it is drawn from. In fact, content missing from the knowledge base is a documented failure point of Retrieval-Augmented Generation systems[154]. Therefore, the quality of the corpus sets an upper bound on the quality of every output (Section 1.3.1).

Guidelines are published by many independent medical societies, in different journals and on different platforms, often requiring institutional access. Since no single repository that collects them exists, the corpus has to be assembled from scratch.

The procedure implemented to construct it runs in five steps (Figure 5.1). First, a comprehensive collection of Emergency Medicine topics is extracted from the chapters of a pivotal textbook of emergency medicine, with each chapter turned into search terms drawn from its title and from the subjects its reference list cites most. The conditions of the Model of the Clinical Practice of Emergency Medicine (EM Model) that no chapter's terms reach are grouped into EM topics, which are searched alongside the chapters. PubMed is then queried for European guidelines covering those topics and an LLM selects, for each chapter and EM topic, the smallest set of guidelines that covers its topics. Chapters and EM topics left without a European guideline are searched again among guidelines of national European societies and of the World Health Organization (WHO). Finally, a consolidation pass removes duplicates and superseded versions across chapters, and the full files are retrieved and downloaded. The final corpus contains 457 guidelines covering 243 of the 277 selected chapters of the reference textbook and 133 of the 198 EM topics.

5.1 Requirements and Design Principles

Several requirements of the guideline corpus construction originate from a suggestion of the emergency department collaborating on the project: to search PubMed for guidelines, to keep the most recent European guideline for each topic, and to use the chapters of *Tintinalli's Emergency Medicine*[18] as a reference for a complete topic space. The others stem from the role of the corpus in the system and from the need to update it as guidelines change.

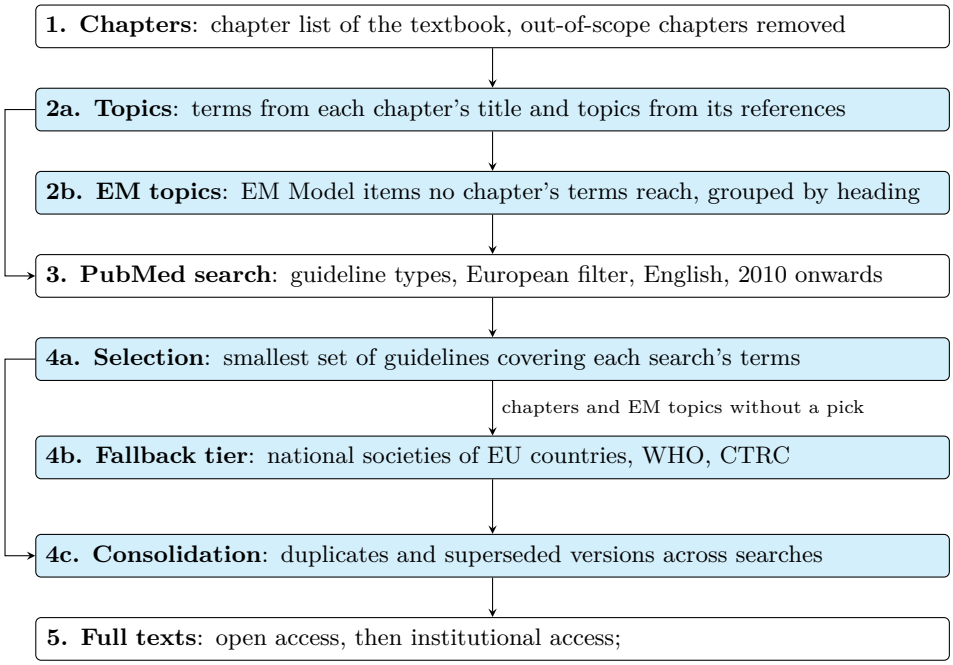


Figure 5.1. The procedure that builds the corpus. Shaded steps involve the judgement of an LLM.

5.1.1 Requirements

The first requirement is *scope*. The corpus must cover the conditions with which emergency physicians discharge or admit their patients, across the whole discipline. Anchoring the topic space to the chapters of a complete textbook of emergency medicine gives an explicit definition of what has to be covered (Section 5.2).

The second is *authority*. The documents must be clinical practice guidelines, which the Institute of Medicine defines as "statements that include recommendations intended to optimize patient care that are informed by a sustematic review of evidence and assessment of the benedits and harms of alternative care options"[155]. For topics that aren't covered by guidelines, consensus statements and position papers are admitted as well, since in fields such as toxicology and environmental medicine they are what the searches find in place of guidelines (Section 5.3). The issuing bodies must be European medical societies. This is because guidelines from different societies, especially on different continents, can disagree on certain conditions and treatments[156]. Restricting the corpus to European bodies is meant to keep it consistent with the practice of the Italian physicians who review and sign the recommendations. Guidelines of national bodies and of recognized international organisations, such as the World Health Organization are admitted as a fallback, for topics that no European guideline covers (Section 5.4). This choice is grounded in Italian Law (Gelli-Bianco Law), which provides that, in the absence of recommendations from officially published guidelines, healthcare professionals shall adhere to good clinical-care practices[157]. Where no national guideline is available, guidelines issued by other European countries are therefore reasonably regarded as good clinical-care practices[158].

The third is *currency*. Guidelines can become obsolete as new evidence accumulates. In a survival analysis of seventeen guidelines of the US Agency for Healthcare Research and Quality, the estimated time for half of them to become outdated was 5.8 years, and the authors recommended reassessing guidelines every three years[85]. The corpus therefore keeps only the most recent version of each guideline, and excludes documents published before 2010. This was set after a trial run without any date limit, in which a handful of older guidelines were selected, and every pick published before 2010 turned out to be outdated or off-target.

The fourth is *language*. Considering that the totality of International European guidelines is published in English, the decision to utilise monolingual models during the retrieval phase was taken (Section 6). While this means that documents in languages different from English cannot be ingested, losing potential coverage especially from Italian guidelines, the trade-off was accepted for a gain in performance, and considering the completeness of breadth of the English corpus of guidelines.

The fifth is *parsimony*. For each chapter, the corpus should hold the smallest set of guidelines that covers its topics, with each topic covered by at most one guideline. In fact, redundant documents are not harmless. An older version of a guideline competes with the current one for the limited number of excerpts passed to the generator (Section 4.3.4), and may carry recommendations that the current one has replaced. Tangential documents contribute passages that are related to the query without answering it, and passages of this kind have been shown to degrade the accuracy of retrieval-augmented generation[142].

The remaining requirements concern the procedure rather than the corpus. The procedure must run end to end with as few manual steps as possible, so that it is *replicable*, for future corpus updates, by someone other than the author. Thus, the only manual intervention required is the download of files that require institutional access (Section 5.5), since remote download is not a feasible option.

It must also be *reproducible*. Threfore, every guideline in the corpus must be traceable to the search that found it and to the decision that selected it.

Finally, its cost must remain bounded, so that a rebuild stays *affordable*.

5.1.2 Design Principles

Five principles guide how the procedure meets these requirements, and they complement the principles that govern the system as a whole (Section 4.1).

External sources over hand-written lists. Whenever the procedure needs a list, it takes it from an external source instead of being hand-picked heuristically. In practice, the chapters to cover come from the textbook's table of contents, and the topics a chapter's title leaves out come from the subjects its reference list cites most.
Similarly, the acronyms used to recognise European societies are extracted from PubMed's own records.

Code for rules, LLMs for judgement. Deterministic code handles everything that can be expressed as a rule, while an LLM is used only where a decision requires domain judgement. Building the queries, filtering the results, and validating and storing the files are all done by code. The LLM, on the other hand, is reserved for four tasks: turning chapter titles into search terms, choosing which topics from a chapter's references are missing from

its terms, selecting guidelines among the candidates, and deciding whether two records are versions of the same guideline. Even then, every answer is checked against what the model was given: a selected guideline must be one of the candidates shown to it, and any other identifier is discarded and flagged.
Limiting the LLM usage to short, well-defined tasks ensures more consistent results and reduces the risk of hallucinations. It is important to notice that, since the model only sees bibliographic metadata, no sensitive data is involved at any point.

Freezing what drifts. Results that change over time are stored, so that rerunning the procedure gives the same corpus. Two of the procedure's inputs behave this way. PubMed returns more records as new ones are published, and built-in result relevance varies vastly even minutes apart (Section 5.3). LLMs, in turn, can give different answers to the same request, even at temperature zero[159,160]. For this reason, each search result and each LLM answer is saved together with the exact inputs it depends on, and is requested again only when those inputs change.
This way, a rerun without changes makes no LLM call and reproduces the corpus exactly, while a rerun after a change only pays for the chapters it affects.

Logging. Nothing is discarded silently. Instead, each step writes a readable log of what it did, including every query and the way PubMed interpreted it, so that decisions can be traced afterwards.

Measuring before changing. Every change to the procedure was adopted only after measuring its effect. Before reaching its final form, the procedure went through several runs, and each change between them was tested on real searches and selections.

5.2 Defining the Scope

The scope comes from two sources. The chapters of a reference textbook provide its structure, with each chapter turned into the search terms that its guidelines should match. A second and independent list, the Model of the Clinical Practice of Emergency Medicine, is then used to catch what the chapters miss: the conditions it lists that no chapter's terms reach are grouped into EM topics, which are searched alongside the chapters. In the rest of the procedure, the search terms serve twice, first in the PubMed search (Section 5.3), then as the list that the selected guidelines have to cover (Section 5.4).

5.2.1 The Reference Textbook

The structure of the scope is taken from *Tintinalli's Emergency Medicine*[18], broadly considered one of the most comprehensive textbook of the discipline, with an independent study finding that it covers 94.5% of all Emergency Medicine subtopics[161]. The subtopics the textbook leaves out are the reason for the implementation of the second souce as a cross-check, as described in Section 5.2.4.

The chapter list of the 9th edition, published in 2019, is read from the publisher's public companion site,[1] which also links the reference list of every chapter. The site lists 303 chapters in 26 sections, with chapter 29 split into two entries (*Tracheal Intubation* and

[1] `https://tintinalliem.com/references_9e.php`

Mechanical Ventilation). Since the chapter list of a published edition does not change, the page is saved once and parsed locally, which also protects the procedure from events like a URL change or the page being taken down when a new edition appears.

Not every chapter can contribute to a discharge letter, so a heuristic decision to exclude the 28 entries listed in Table 5.1 was taken. Chapters on prehospital care and disaster management concern the organisation of emergency services rather than individual patients, while those on resuscitative procedures and on the neurological examination describe techniques performed within the department, but provide no relevant indications on discharge procedures. The chapters on death notification, advance directives, military medicine and legal issues are excluded for the same reason. The chapter on child abuse and neglect is excluded as well, since it is a topic that requires specific personalized treatments and would not provide discharge recommendations, but only psychological treatment and imaging which is not within the scope the model.

Chapters	Content
1–4	Prehospital care
5–10	Disaster management
22–33 (incl. 29A, 29B)	Resuscitative procedures
150	Child abuse and neglect
164	Neurologic examination
301–303	Death notification and advance directives, military medicine, legal issues

Table 5.1. Chapters of the reference textbook excluded from the scope.

Conversely, the 9th edition went to press before the COVID-19 pandemic, so a chapter on COVID-19 is manually added to the section on infectious diseases and processed like the others. The scope thus starts from 277 chapters, 276 from the textbook and one added.

5.2.2 Title Terms

The first source of a chapter's search terms is its title. In an early version of the procedure, an LLM was instructed to write a complete PubMed query for each chapter, but upon inspection, it was evident that the 304 queries it produced revealed many defects. Some queries appended generic words despite instructions not to, such as `AND guideline` or `AND management`. In several instances, the model ignored query construction syntax, producing plain text or using parenthesis on the wrong constructs, leading to unusable or misleading queries (`End-Stage Renal Disease | ("end-stage renal disease" OR "chronic kidney disease" OR "dialysis")`). Quoted phrases, finally, proved too strict, since PubMed matches them verbatim: `"caustic ingestion"` missed records titled *caustic ingestions*.

The LLM was therefore restricted to writing terms. For each chapter, it now produces one to six plain phrases that the title of a guideline on the chapter's core subjects would contain, and optionally the patient group the chapter concerns. The code then builds the query, using deterministic rules, from these terms, thus taking the syntax out of the model's hands and allowing the same terms to be reused when guidelines are selected.

This feature relies on MeSH, the controlled vocabulary with which the National Library of Medicine indexes PubMed records[162]. Each MeSH heading names a single concept and lists the synonyms under which it may appear, so that *heart attack*, for instance, leads to the heading *Myocardial Infarction*. The headings are arranged in hierarchical trees, with diseases under category C, and a search for a heading also retrieves the records indexed with the narrower headings below it[163]. Automatic Term Mapping matches each phrase of a query to the corresponding heading, and also searches the phrase's words in all fields. As a result, a term also reaches recent records that have not yet been indexed with MeSH. The term *sudden cardiac death*, for instance, is searched both as the heading *Death, Sudden, Cardiac* and as its three words anywhere in the record.

For each chapter, the model receives its number, section and title; the section helps it recognise the patient group, as for the chapters of the paediatrics section. It writes the terms for forty chapters per request, at temperature zero, following a prompt with three worked examples (Appendix A.1). The final run used Claude Opus 5.5, as a strong model is worth its cost at this step: every later step depends on these terms. The patient group, when present, is mapped by the code to a fixed clause, so that a paediatric chapter, for instance, restricts its search with `(child OR infant OR pediatric OR adolescent)`.

Still, even a powerful Large Language Model set at temperature 0, can occasionally deviate from a clear set of instructions. To limit this phenomenon, the terms are then cleaned. Quotes, brackets and asterisks are stripped, and a term made only of generic words, such as *management* or *emergency*, is dropped, since on its own it matches guidelines on unrelated conditions. Generic words inside a longer term are kept instead, because there they carry meaning, as in *hypertensive emergency*. Banning these words in the prompt was tried first and did not work, since the model often ignored the instruction.

5.2.3 Terms from the References

Titles alone miss part of what a chapter covers, especially in broad chapters. In an earlier run, for instance, chapter 77, *Esophageal Emergencies*, was searched for foreign bodies, perforation, food bolus and Boerhaave syndrome. Gastro-oesophageal reflux disease, a common diagnosis, was never searched, although PubMed does hold European guidance on it. The reference list of each chapter, however, records the subjects the chapter draws on, and can be used to fill these gaps.

The companion site publishes the references of every chapter as a PDF. The PubMed identifiers they contain point to more than 13,000 distinct records, which are retrieved through NCBI's E-utilities together with their MeSH indexing. For each record, the procedure keeps its major topics, the headings that indexers mark as the main subjects of the article[163]. Since the references of an edition do not change, this data is fetched only once.

Specifically, a major topic counts as a subject of the chapter if it names a disease or a mental disorder, which MeSH files under its trees C and F03. For every such subject, two counts are kept: how many of the chapter's references cite it, and how many chapters cite it at all. The second count was implemented to separate background conditions which are cited across many chapters, from the chapter's own subjects, which few other chapters cite. The idea resembles TF-IDF weighting[120], with the first count playing the role of term frequency and the second that of document frequency. The two counts are not combined into a score, however: the first sets the threshold for candidates, and both are passed to the LLM that

screens them. In fact, a subject becomes a candidate topic for its chapter if at least two of the chapter's references cite it, and only if the title terms do not already reach it.

Not every candidate deserves a search, however. A rule based purely on the two counts was tried first, but was found to be retrieving noise: it gave *Tachycardia* to the arrhythmia chapter, and *Abscess* to the appendicitis chapter, which then retrieved guidelines on periodontal and anal abscesses. The candidates are therefore screened by an LLM, which sees the chapter's title terms and its eight most-cited candidates together with their two counts. The model picks at most three candidates that it believes to be core subjects of the chapter missed by its terms, and is told to skip complications, comorbidities, differential diagnoses, side effects of treatment and subjects cited across many chapters (Appendix A.1).

In the final run, the screen added 187 reference terms to 111 chapters.

5.2.4 The EM Model

The chapters of the textbook and what their search terms reach depends on an LLM's reading of their titles and on the subjects their references happen to cite. It is possible that some finer topics were not included using the described method. A second list, independent of both the book and the model, is therefore used to check what the chapters miss. The Model of the Clinical Practice of Emergency Medicine (EM Model) describes the clinical content of the specialty, was originally derived from a practice analysis combining empirical data with the advice of expert panels, and serves as the basis for the content of the examinations of the American Board of Emergency Medicine[19]. The procedure uses its 2022 version.

The EM Model lists the conditions of the specialty in a table organised into twenty categories, and takes the clinical items of categories 2 to 18, 730 in all. Category 1, which lists signs and symptoms, is left out, since the presentation chapters of the textbook already search the symptoms that have guidelines of their own, and looking for generic symptomps like *Pain* or *Headache* would lead to noisy candidates. Categories 19 and 20, on procedures and core competencies, are left out for the same reasons as the corresponding chapters of the textbook. The table is extracted with the `pdftotext` utility of the Poppler library[2], because PyMuPDF silently dropped ten of its rows.

Many items only make sense under their headings: the item *Tension*, listed under *Pneumothorax*, means tension pneumothorax. An LLM therefore gives each item a single name, read in the context of its headings and without added synonyms, so that it can be compared with the textbook chapters' search terms.

An item counts as covered when PubMed's Automatic Term Mapping reads one of the book chapters' search terms as the same concepts as the EM item's name. Relying on PubMed's mapping rather than on the judgement of an LLM makes the decision deterministic and reproducible. Items on tumours, recognised by their MeSH category falling under *C04.557/C04.558*, are left out altogether, since the guidelines on their care concern oncological treatment, which lies outside the scope of an emergency discharge.

The items identified as uncovered by the chapters are pooled under their heading into EM topics and they go through the same search, selection, fallback tier and consolidation process as the chapters (Sections 5.3 and 5.4).

[2] `https://poppler.freedesktop.org`

Of the 730 items, 269 were covered by a chapter's search terms and 26 were left out as tumours. The remaining 435 were pooled into 198 EM topics. The share of covered items, just over a third, is not comparable with the textbook's 94.5% coverage of the 2019 EM Model: that figure measures whether the textbook discusses a subtopic, whereas an item is covered here only if a chapter's search terms reach it in PubMed.

5.3 Searching PubMed

The topics of each chapter are searched in PubMed, the database of biomedical literature maintained by the US National Library of Medicine, containing more than 40 million medical records of biomedical literature including worldwide guidelines[163]. PubMed is queried through NCBI's E-utilities, which return the IDs of the matching records, together with their bibliographic metadata. Every search is restricted by filters on document type, language, date and European origin. This section describes the query formation process and its filters, and explains how they are used to produce the candidates for selection. Topics left without a European guideline are later searched again among national bodies and international organisations; since this second search depends on the outcome of the selection, it is described in Section 5.4.

5.3.1 Query and Filters

Each chapter and each EM topic is searched with a query, in which its terms are joined as alternatives. The query of a chapter that concerns a specific patient group is further restricted by the corresponding clause (Section 5.2.2). For example, with the terms *otitis media*, *otitis externa* and *mastoiditis* and the paediatric clause, the query of chapter 121, *Ear and Mastoid Disorders in Infants and Children*, reads

```
((otitis media) OR (otitis externa) OR (mastoiditis))
  AND (child OR infant OR pediatric OR adolescent)
```

The query is then combined with the other filters, described in the rest of this section:

```
 (query) AND <document types> AND (<European filter>)
AND english[la] AND 2010:3000[dp] NOT <excluded types>
```

Document types. Two sets of document types are used, a strict one and an extended one. The strict set accepts the records that PubMed types as *Guideline*, *Practice Guideline* or *Consensus Statement*, as well as any record whose title contains the word *guideline*. While initially excluded from the strict set, the addition of *Consensun Statement* led to consistently better results, and was therefore included. The title clause is needed because publication types are not always assigned; for instance, the 2018 ESC guidelines on syncope are indexed as a plain journal article, and about a third of the guidelines in an intermediate version of the corpus carried neither guideline type. When the strict set returns three or fewer records, indicating that an appropriate document may not have been retrieved for that concept, the search is repeated with an extended set accepting titles containing *consensus*, *position paper* or *position statement*. In the final run, 33 chapters and 67 EM topics needed the extended set.

Language and date. Records must be in English and published from 2010 onwards, while comments, editorials, letters and errata are excluded. Since the language field describes the record rather than the downloaded file, the language is then checked again on the full text to avoid mismatches (Section 5.5). The date limit follows from the first trial run, which did not have one: eleven of its picks, nine distinct documents, predated 2010, and all of them were outdated or off-target. Among them were a 1997 report of a members' poll on bronchiolitis and a 2004 consensus on paediatric diabetes, picked for two adult chapters on ketoacidosis.

5.3.2 The European Filter

European origin, a system requirement, is inferred from the text of the record. In practice, a record counts as European if its title, its corporate author, one of its affiliations or the name of its journal contains a word beginning with *europe*, or if its title carries the acronym of a Europe-wide medical society.

The clause on journal names initially contained a subtle error. It was written as `eur*[ta]`, but PubMed only truncates terms of at least four characters[163], so the clause matched no journal at all. Replacing it with `europe*[ta]`, which matches the journal titles beginning with *European*, raised the number of European guideline records by 38%. A further clause catches the journals whose title begins with *The European*, such as *The European Respiratory Journal*.

Society acronyms. Even so, basing the search only on variations of the word *Europe* excluded guidelines that name their society only by its acronym. The 2023 ERS/ESICM/ESCMID/ALAT guidelines on severe community-acquired pneumonia, for instance, were published in two journals: the copy in the *European Respiratory Journal* is caught by the journal name, but the copy in *Intensive Care Medicine* is not. Had it been published only in the latter, a useful guideline would have been lost. Therefore, an acronym clause was added, with the list of acronyms extrapolated a priori from PubMed itself, so that its completeness could be estimated (Section 5.1).

The starting point is every guideline-type record in English from 2010 onwards, 97,935 in total, of which about a tenth were already caught by the field term search. From the titles of the remaining 88,334 records, every token of two to twelve characters with at least two capital letters is extracted, giving 8,830 distinct acronym candidates. The shape of a token cannot tell a society from a disease, a test or a trial, so the 3,571 tokens occurring in at least two missed records were fed to an LLM to be labelled. For each token, the model saw two example titles and assigned it to one of four classes: Europe-wide body, national body of a European country, other body, or not an organisation. In total, it labelled 174 tokens as Europe-wide bodies.

The 5,259 tokens that occur in a single missed record were left unlabelled, and their contribution was estimated from a random sample of 300. The sample contained five Europe-wide bodies, all from niche fields such as cutaneous lymphoma and restless legs syndrome. Scaled to the whole tail, this suggests that about 90 European records remain unrecognised (95% interval roughly 35 to 200), around 0.1% of the total records.

Finally, the 174 Europe-wide tokens were reviewed by hand. Fourteen were excluded, either due to ambiguity (as was the case for GA, which also stands for general anaesthesia) to scope (as for regulatory agencies and for veterinary bodies). Of the remaining 160, 156 are searched directly in titles.

Acronyms that are also common words. The last four acronyms need special treatment, because PubMed's title search ignores case, even for quoted terms. Searched directly, the acronym of the European Renal Association, ERA, would match every title containing the word *era*: 254 titles, against 17 relevant ones. For ERA, COST, SHARE and NATA, the procedure therefore runs a case-insensitive search, reads the titles, and adds to the filter only the records whose title contains the acronym in capitals as a separate word. The check is not perfect, as an Italian study group sharing one of the acronyms also passes it, but ultimately the risk is accepted. In fact, since a false positive only becomes a candidate which the selection can still reject, an error would not pollute the final corpus.

Together, the acronym clause and the case check further raised the European records of the strict set by 26%, from 6,282 to 7,913.

5.3.3 Keeping Every Hit

In earlier versions of the procedure, each search kept only the 30 records ranked highest by Best Match, PubMed's default relevance ranking, with the goal of limiting the length of the selection prompt. Best Match orders records with a machine learning model trained on past user searches, which weighs signals such as how often each article has been accessed[164]. Its order, however, proved to be significantly unstable. Sixteen chapters searched twice in a row with identical queries came back in a different order in fifteen cases, and in nine of them the top 30 contained up to three different results. Across days the drift was larger: with the same search query, the 2024 ESC guidelines on atrial fibrillation ranked second in the arrhythmia chapter one day, and between 86th and 154th three days later. Relying on PubMed's ranking model would have made the presence of a guideline among the candidates a matter of chance, and was therefore discarded as an option.

Several alternatives were considered before entirely removing the cut. A larger cut only moves the threshold, and the atrial fibrillation example showed it was not a viable option. A hybrid, taking the 30 best records of the broad query and the 10 best of a separate query for each term, with the goal of catching both broad and fine scoped guidelines, picked superseded guidelines and republished copies on seven test chapters. Moreover, it still inherited the same unstable ranking, and exhacerbated the issue due to a smaller result pool. Restricting the terms to titles or to major MeSH topics shortened the lists, but lost good guidelines on finer topics, such as the ESC guidelines on cardiac pacing.

The final procedure therefore keeps every result, up to the 10,000 identifiers that a single E-utilities request can return, and presents them to the selection newest first rather than by relevance. The search is performed exclusively with the query described in Section 5.3.1, so broad guidelines covering several of the terms of a search are still found. Without a cut, the candidates no longer depend on the day's ranking, but only on PubMed's content. The price of this choice is longer candidate lists, whose handling is described in Section 5.4.

5.3.4 Search Statistics

Table 5.2 summarises the searches of the final run, which covered 277 chapters and 198 EM topics. Of these, 18 chapters and 28 EM topics found no candidate at all in the European search, mostly toxicology-related (a known gap in the guidelines). Together with the topics whose European search found candidates but no pick, a total 146 topics were passed on to the fallback tier (Section 5.4), which returned 977 records and picked a national or WHO

guideline for 47 of them.

Quantity	Chapters	EM topics
Searched	277	198
Using the extended set of document types	33	67
With no result	18	28
Results per search: median; maximum	42; 763	8; 1,214
With more than 30 results	159	43
With more than 300 results	22	4
Candidates in total (distinct records)		8,878

Table 5.2. The PubMed searches of the final run, European sources only. The fallback tier is reported in Section 5.4.

5.4 Selecting Guidelines

For each chapter and EM topic, an LLM reads the candidates of its search and selects the smallest set of guidelines that covers its search terms. Chapters and EM topics left without a European guideline are then searched and selected again among national bodies and international organisations, in what is called the fallback tier. Finally, since each selection only sees its own candidates, a consolidation pass reconciles the picks of all searches, removing duplicates and superseded versions.

5.4.1 The Selection Call

For every chapter of the textbook and EM topic, a separate call is sent to the LLM. The model receives the title and section of the chapter, or the heading if it is a EM topic, together with its search terms and the list of its candidates. Each candidate is presented with its PubMed identifier, publication date, journal, publication types and title, and the list is sorted by newest. Records of books, such as the WHO guidelines held in the NCBI Bookshelf, carry a book title and a publisher instead of an article title and a journal. Abstracts are deliberately left out to avoid bias, as many candidates did not have one.

The model answers with a list of picks, each with its identifier, the search terms it covers and a short name for its file. The covered terms act as a categorical justification of each pick, in place of a relevance score, and allow the selection to be checked against the terms it was given.

The precise rules of the selection are set in the prompt (Appendix A.1). A pick must guide how an emergency physician assesses, treats or discharges a patient with the condition named by one of the search terms. It qualifies if it is a clinical practice guideline, a consensus statement or a position paper of a European medical society, while the guideline of a national society (Section 5.4.3) qualifies only for a term that no Europe-wide guideline in the list covers. Among the documents to reject are those of non-European bodies, documents on other conditions or populations, adherence and implementation studies, reviews and commentaries, republished or translated copies of a listed original, pocket or summary versions, and older versions of documents present in the candidate list. The model is asked for the smallest set that covers the terms, with each term covered by at most one guideline: the best one, or the most recent among equals. A guideline is preferred to a consensus statement and one broad guideline to several narrow ones, and a term may remain uncovered rather than be covered

by a guideline on a neighbouring or unrelated condition. Most of these rules were added in response to observed errors during earlier runs.

Every answer is validated before use. Identifiers that are not among the candidates are dropped and flagged as invented, covered terms that the chapter does not have are flagged, and selections of more than three picks are flagged, without being truncated since several broad chapters correctly contain more than three relevant guidelines. Failed calls, such as API errors or unreadable answers, are not stored, and are repeated on the next run.

5.4.2 Long Lists and Model Choice

Keeping every result (Section 5.3.3) means lists of hundreds of candidates are to be evaluated in the largest chapters, which raises the question of whether the model is still able to follow the rules on them. This was tested on the three largest chapters, with 485 to 752 candidates, by running each full list thrice. In all of the outputs, the model never returned an identifier absent from its list. The core picks were identical across the three runs, and the runs differed only at the margin, in a third or fourth pick for a weak term. A plausible reason for this robustness lies in the nature of the task: the model picks a few items from a list, with clear instructions and no formatting oir structural rules to follow, its answer is short, and the right guideline is usually evident from its title. In these tests, in fact, the errors came from terms without a clear guideline rather than from the length of the list.

Earlier runs selected with Claude Sonnet 5, while the final run uses Claude Opus 5.5. On seven test chapters, the two models made the same core picks, but at the margin Opus followed the rules more closely. The stronger model comes with an increase in pricing, as Opus is twice as expensive per token as Sonnet[165,166]. However, since the selection ultimately decides what enters the corpus, and the procedure only runs when the corpus is rebuilt, it was deemed worth paying.

Due to the nature of Large Language Models, even at temperature zero the selection is not fully reproducible. In a test, repeating the same seven requests gave identical picks in four chapters, and in the other three differences only at the margin, such as the copy of the same guideline in another journal. Therefore, the variance in the model's outputs is not invalidating.

5.4.3 The Fallback Tier

In an intermediate version of the corpus, 111 of the 277 chapters were left with no picks. Part of this reflects the literature, since for conditions such as vertigo, animal bites and tetanus PubMed holds no European society guideline. Part of it, however, was an effect of the European filter. The selection already accepted national societies for terms without a Europe-wide guideline. Alas, national guidelines rarely mention Europe, and so they seldom reached the candidates.

Chapters and EM topics left without a pick, either because they had no candidates or because the selection picked nothing, are therefore searched again with a similar query in which a fallback filter replaces the European one. The filter admits four kinds of issuers. National societies of European states are recognised by a country or nationality word in the title or the corporate author, such as *Italian* or *German*, and by corporate authors written in the society's own language. The World Health Organization is reached through its

publisher field only, since a title search would also add reprints of its documents in journals. Finally, the Clinical Toxicology Recommendations Collaborative (CTRC), led jointly by the European Association of Poisons Centres and Clinical Toxicologists and its American and Asia-Pacific counterparts, is searched by name without any filter on document type, since PubMed does not always type its recommendations as guidelines. All clauses keep the language, date and type exclusions of the European tier. For the same term, a Europe-wide guideline is preferred to a national one, NICE's included, and the guideline of an Italian society to that of another country. The last preference reflects the setting of the system, since Italian law refers clinicians to the recommendations of the guidelines in the national guideline system and, in their absence, to good clinical practice[157].

The fallback candidates go through the same selection call, with the clauses on which documents qualify or must be rejected adapted to the tier (Appendix A.1), and their picks take part in the consolidation step like any other. Table 5.3 reports the tier in the final run, which gave a pick to 32 chapters and 15 EM topics. Among the fallback picks are the Italian Paediatric Society's guideline on fever in children, the Italian diabetologists' consensus on diabetic ketoacidosis, NICE's guidelines on head injury and on fractures, and the WHO guidelines on malaria.

Quantity	Chapters	EM topics
Without a European pick	66	80
of which without candidates	18	28
With fallback candidates	50	48
With a fallback pick	32	15

Table 5.3. The fallback tier in the final run.

Toxicology remains the largest gap. Its guidance is issued mostly outside Europe or by international workgroups, such as EXTRIP, which despite including representatives of the European toxicology association, states that their participation does not amount to an endorsement. Other sources, such as the American Heart Association's 2023 focused update on the management of toxicity, are not European. Overall, 19 of the 32 toxicology chapters have a pick. Whether non-European toxicology bodies should enter the corpus is a question left to the clinical collaborators.

5.4.4 Consolidation

Since each selection sees only its own candidates, the same guideline or different versions of it may be picked in different searches. A consolidation pass therefore reconciles the picks of all chapters and EM topics, in both tiers.

The first attempt sent all the picks to a single call, asking for duplicates and superseded versions. It found one replacement, and an identical second call found none, although clear cases were present: asked only for replacements, the model silently skipped pairs. The final method therefore splits the task into a deterministic part and a judgement.

First, the code forms candidate pairs, deliberately broadly: two picks whose titles share at least 60% of the words of four or more letters of the title. The LLM then gives a verdict on every pair: `same` for the same document in two journals, a translation or a summary, `newer` for two versions of one guideline (applied only if it does not replace a guideline whose

title names a European body with one who does not), and `different` otherwise. Requiring a verdict for every numbered pair keeps the model from skipping pairs, and in tests repeated calls agreed. The rules are to keep the full original in the society's journal and the newest full version, and never to treat a guideline of another society, a national one included, or a practical or summary version as a newer version, while a focused update does not replace the guideline it updates. Finally, the code applies the verdicts, resolving version conflicts, and merging the covered terms of the picks it joins.

In the final run, 1,463 pairs were judged in 25 calls, each of which returned a verdict for every pair, and 70 picks were replaced. Since the consolidation step also runs through an LLM, it is not fully repeatable either. Still, the differences between runs are extremely marginal.

5.5 Retrieving Full Texts

Once the picks are final, the full text of each distinct guideline is retrieved as PDF files. Open copies are downloaded automatically, while the remaining guidelines are fetched through the browser, to allow for institutional login. Every file is validated before it is accepted, and is then marked with the PubMed identifier of its guideline, so that it can always be traced back to its record.

5.5.1 Automatic Retrieval

For each guideline, copies are tried in a fixed order. If the record has a PubMed Central identifier, the article is first sought in the PMC Open Access Subset, which the National Library of Medicine distributes as a public dataset on Amazon Web Services[167]. When PMC yields no valid file, the procedure looks up the guideline's DOI in OpenAlex, an open index of scholarly works[168], and tries every location it lists with a PDF link. These locations are ordered by their version label, with published versions first, accepted manuscripts second and the rest last. The labels only order the copies and never exclude one, since they proved unreliable: repository copies of the publisher's PDF, for instance, are often labelled as submitted versions. Finally, if none of a guideline's own copies passes validation (Section 5.5.2), PubMed is searched for records with exactly the same title, to identify the same guideline if it was co-published in another journal, and their copies are tried in turn.

In the final run, 175 of the 457 guidelines, were downloaded automatically, 106 of them from PMC.

5.5.2 Validation

A downloaded file is accepted only if it is a PDF that opens, and if it passes three checks:

1. *Ensuring the file is in English.* This check, added after OpenAlex was found to list the Italian translations of four ESC guidelines as copies of the English articles, complements the language filter used in the queries (Section 5.3.1). To be accepted, a file must have at least 8% of the words on three of its body pages among a short list of common English words. On the corpus, English guidelines scored at least 15.5%, with a median of 24%, against 1.6 to 3.0% for the Italian translations.

2. *Verifying it is the right document.* Either the DOI of the record or at least 80% of the words of four or more letters in its title must appear in the first three pages. The check is not performed solely on the front page to bypass possible cover sheets.

3. *Requiring more than two pages.* Performed to avoid previews that also carry the title without actual content.

A copy that fails the language check or does not open is never kept on the automatic retrieval pass. A copy that fails the identity or length check, instead, is kept only if no other copy passes, and is flagged for a check by hand. In the final run, none of the automatically downloaded files failed the language or identity check, and the five files longer than 150 pages turned out, on inspection, to be accepted manuscripts with their supplements rather than wrong documents.

Each accepted PDF then has the PubMed identifier of its guideline written into its metadata. Files are identified by this stamp rather than by their name, so that a guideline already on disk keeps its file even if the file name is changed later, and files that no longer correspond to a pick are reported.

5.5.3 Browser-Assisted Retrieval

The guidelines without an open copy available for automatic download are retrieved in a browser-assisted mode, to allow the user to insert its personal or institutional credentials. The mode processes one guideline at a time. It opens the DOI link of the guideline, or its PubMed page when there is no DOI, in the user's browser, and then watches the Downloads folder for the next complete PDF. Once the user has saved the file, it is automatically moved to the Guidelines folder, validated and stamped it accordingly to the process described in Section 5.5.2 and renamed it under the guideline's name. When the process is over, the system automatically opens the following DOI link. A file that fails the language check is not dropped by default, and the user is asked for confirmation over they would prefer to keep it or look for another copy. Progress is saved after each file, so that an interrupted session keeps what it obtained.

While the process may appear tedious, it was the only practical solution to obtain those files who are not available for open download. Since the system automatically opens the web pages, opening the PDF is normally a matter of one to three clicks, depending on the publisher. This does not include login time. In total, the full run of 282 files required approximately 93 minutes to finish.

In the final run, 261 of the 282 remaining guidelines were obtained this way, and 26 stayed out of reach behind publishers to which the University of Trento does not subscribe.

Quantity	Final run
Distinct guidelines selected	457
Downloaded automatically (open copies)	175 (38%)
of which from PubMed Central	106
Fetched through the browser	256
Not obtained (no institutional access)	26

Table 5.4. Retrieval of full texts in the final run.

5.5.4 Legal Basis

Full texts are obtained only through open access or the institution's subscriptions, and shadow libraries were excluded from the outset. The files are not redistributed and are kept out of public repositories.

5.6 The Resulting Corpus

The corpus described in this section is the output of the final run performed in October 2026. The run started from an empty folder, so that every search, every LLM answer and every download was produced anew. Its output consists of a folder of validated PDFs, one per guideline, and of a manifest that lists, for every chapter and EM topic, its picks with the search terms they cover, the source of each file, the replacements made by consolidation and the flags left for a check by hand.

5.6.1 Size and Composition

The corpus used for evaluation and testing holds 431 distinct guidelines (Table 5.5), though with complete access to every institution, like a hospital is assumed to have, it could theoretically be comprised of 457. The following statistics are therefore computed based on the theoretical tally. Of the 277 chapters, 243 have at least one pick, 32 of them through the fallback tier, and so do 133 of the 198 EM topics, 15 of them through the fallback tier. The chapters alone pick 341 guidelines, while the EM topics add 116 that no chapter picks, so that the cross-check of Section 5.2.4 accounts for about a quarter of the corpus. Of the 435 items pooled into EM topics, 321 belong to a topic with a pick, and 199 are named explicitly among the terms that a pick covers.

Quantity	Final run
Chapters with at least one pick	243 of 277 (88%)
through the fallback tier	32
EM topics with at least one pick	133 of 198 (67%)
through the fallback tier	15
Distinct guidelines	457
picked by chapters; by EM topics; by EM topics only	341; 171; 116
Picks per chapter: 1; 2; 3; more	131; 62; 29; 21
Files obtained	431

Table 5.5. The corpus produced by the final run.

Figure 5.2 breaks the chapters down by the section of the textbook they belong to. Most sections are covered almost entirely by European picks, while toxicology, environmental injuries, wound management and trauma account for 26 of the 34 chapters left without a pick (Section 5.4.3).

5.6.2 Cost and Running Time

Both cost and running time are heavily influenced by the choice of using the batch version of the LLM, or the standard one. In fact, for all Claude models providing batch access, the per token cost is halved. However, the running time is heavily dependent on the queue length,

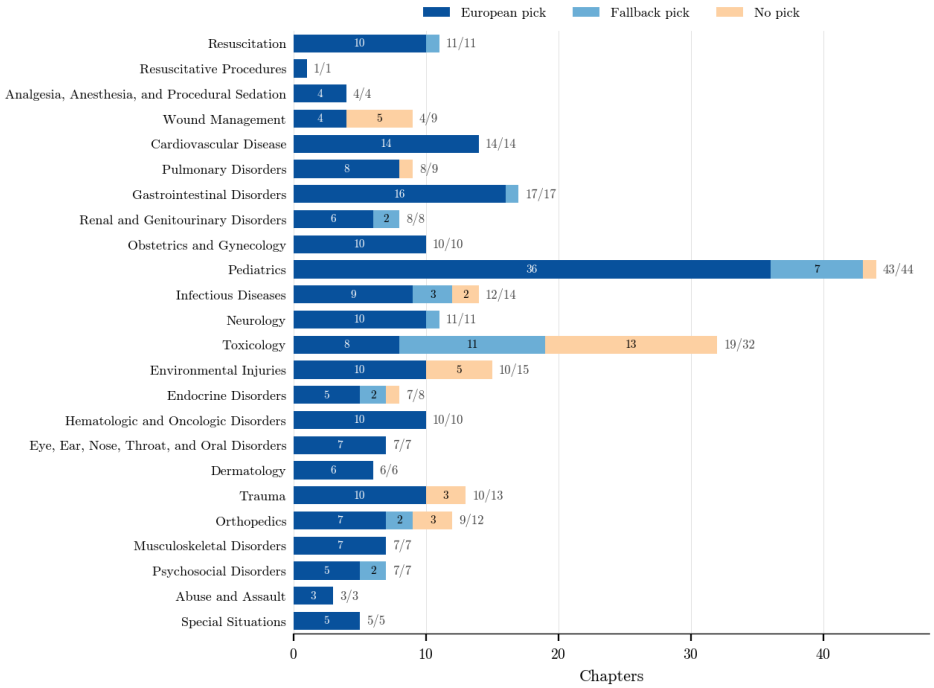


Figure 5.2. Chapters per section of the reference textbook by the tier of their picks. The fraction next to each bar is the share of chapters with at least one pick. Sections whose chapters were all excluded from the scope (Table 5.1) are not shown.

and vary from a few minutes per batch of calls to several hours. The final run, performed purely with the usage of standard model calls, took 45 minutes and cost about US$23.8 in LLM calls. The previous run, which used the batch interface, took 87 minutes and cost half. However, this is mostly due to it taking place in a moment where queues moved faster than usual. During testing, it was common to see waiting times of 60–90 minutes for a single batch of calls. Considering the corpus pipeline utilises six batch jobs, it is realistic to expect a total execution time of several hours.

Most of the cost lies in the selection, since its prompts carry every candidate of every search: in the previous run, the selection of the European tier alone accounted for approximately US$19.9 of the total.

The data sources are free, and the searches are bound by NCBI's limit of three requests per second rather than by computation. A rerun without changes makes no LLM call and therefore costs nothing. Browser-assisted retrieval, finally, depends on the person running it (Section 5.5.3).

5.7 Evaluation

The corpus was evaluated along three questions, designed to evaluate various aspect of its scope. The first is whether the guidelines it contains are the right ones: non-broken files, real guidelines, current versions, and relevant to the topics they were picked for. The second is whether guidelines from the most authoritative European societies were retrieved, which was

measured against the complete guideline collections of five societies. The third is whether it covers the conditions that patients are actually discharged with, which was checked on the diagnoses and context of 25 clinical notes. All checks refer to the corpus of the final run (Section 5.6), and the proportions they yield are reported with 95% Wilson score intervals[169], which behave well on small samples and on proportions close to 0 or 1[170].

5.7.1 Precision

Fifty guidelines were drawn at random from the corpus, and each was examined by hand on four questions. The file must open and be complete, the document must be a guideline in the English language rather than a summary, an addendum or another kind of publication, and it must not have been superseded by a newer version. Most importantly, it must match the topics it was picked to cover, as listed among its covered terms in the manifest. On the first three questions, the 50 files proved sound, as all 50 documents were guidelines written in English, and none had been superseded (50 of 50 each, 92.9–100%).

On the last question, 49 of the 50 guidelines matched the topics they were supposed to cover (98.0%, interval 89.5–99.6%). The exception was a guideline on antifungal strategies in neutropenic patients with haematological malignancies, picked to cover *chemotherapy-induced immunosuppression*: the document deals with a consequence of chemotherapy, but immunosuppression is not its main subject. Two of the 49 matched the topic only partially, lowering the share of complete matches to 47 of 50 (94.0%, 83.8–97.9%). In particular, *tetanus prophylaxis* was considered covered by recommendations on vaccination in patients with autoimmune rheumatic diseases, which address tetanus vaccination only in a specific population. *Malignant spinal cord compression*, in turn, was covered in part by a guideline on radiotherapy for complicated bone metastases.

Both partial matches reflect a lack of guidance in academic publications more than an error of the selection or retrieval failures. For malignant spinal cord compression, a search by hand found no dedicated guideline that meets the criteria of the corpus, and for tetanus the only dedicated guideline found is the WHO's, which only the fallback tier admits. The tetanus case also shows a side effect of the tier's design. Since a topic reaches the fallback tier only when it has no European pick, a weak European pick can keep out a better document of the fallback tier.

5.7.2 Recall

Recall was measured on five societies that publish comprehensive collections of guidelines: the European Society of Cardiology (ESC), the European Resuscitation Council (ERC), the European Stroke Organisation (ESO), the European Society of Gastrointestinal Endoscopy (ESGE) and the European Respiratory Society (ERS). Each of their guidelines was annotated by hand as relevant to the emergency department and not superseded, and as such expected in the corpus, or not. Of the 44 guidelines annotated as relevant, 41 were retrieved in the corpus (93.2%, 81.8–97.7%).

All three misses were among the candidates of the searches in several chapters, but were passed over by the selection, which judged their subjects as already covered by other guidelines. Two are ESO guidelines, on the management of Glycaemia in Acute Stroke and on the Acute Management of Basilar Artery Occlusion, while the third is the ERC guideline

on Post-Resuscitation Care. Every relevant guideline of the five societies thus reached the candidates (44 of 44, 92.0–100%), which means that none was lost to the search. The losses all come from the parsimony of the selection, which asks for one guideline per topic and prefers a broad guideline to several narrow ones (Section 5.4.1). In these three cases, the rule kept out focused guidelines that the annotation considered relevant alongside the broader ones.

Relaxing the rule would probably recover such guidelines, but would also create the risk of admitting overlapping documents, which could ultimately pollute retrieval by leading to the selection of repetitive or even contrasting chunks (Section 5.1). To measure how much noise the current rule lets through, the opposite check was also run. Fifty guidelines of the same societies annotated as not relevant were randomly selected, and the corpus contained only one of them, the 2022 ESO guidelines on unruptured intracranial aneurysms (2.0%, 0.4–10.5%). The current selection thus trades a few misses for very little noise.

5.7.3 Coverage of Clinical Notes

The last check moves from the corpus to its use. It relies on the 25 clinical notes used to evaluate the outputs of the system (Chapter 7), synthetic notes written by emergency physicians to be as realistic as possible, which played no part in building the corpus. For each note, the corpus was checked for a guideline covering its diagnosis contextualized by comorbidities and anagraphical data.

Of the 25 notes, 21 were considered to have a diagnosis covered or mostly covered by the corpus (84.0%, 65.3–93.6%). Among the four exceptions, an adult with gastroenteritis was only partially covered, as only paediatric guidelines were found in the corpus, and a painful callus-like lump on a toe finds none. The other two, a knee contusion and a chin laceration needing sutures in a child, are covered only in part by guidelines on infections of the skin and soft tissues, while no guideline addresses the management of fresh wounds and contusions. A search by hand was therefore conducted to study whether the gaps derived from a search and retrieval failure, and found no guideline for that could meet the criteria of the corpus. These gaps therefore lie in the literature rather than in the procedure, consistently with the absence of European guidelines on lacerations noted in Section 5.4.3.

5.7.4 Summary and Limitations

Table 5.6 summarises the results. The guidelines in the corpus are almost always the right ones, the search lost none of the relevant guidelines of the societies examined, and the selection traded three misses for very little noise. Moreover, where the corpus fails to cover a diagnosis the gap lies in the literature rather than in the procedure.

Several limits qualify these results, and are discussed further in Chapter 8. Namely, all judgements were made by the author as to what counts as a match, as relevant or as covered. For the third test, the samples are small, as the width of the intervals shows, since the clinical notes are few. Finally, the evaluation covers the corpus of a single run, and a rebuild from scratch would not yield an identical one, since the answers of the LLM vary between identical requests, even if only marginally.

Check	Result	Share (%)	95% interval
Sampled files sound; guidelines; current versions	50/50 each	100.0	92.9–100.0
Sampled guidelines matching their topics	49/50	98.0	89.5–99.6
matching completely	47/50	94.0	83.8–97.9
Relevant society guidelines in the corpus	41/44	93.2	81.8–97.7
among the search candidates	44/44	100.0	92.0–100.0
Non-relevant society guidelines in the corpus	1/50	2.0	0.4–10.5
Clinical notes with a covered diagnosis	21/25	84.0	65.3–93.6

Table 5.6. Results of the evaluation of the corpus, with Wilson score intervals.

Chapter 6

System Implementation

In Chapter 4, a description of the system in terms of its components, and the principles that shaped them, was discussed. This chapter describes how those components were actually built: the libraries and models chosen for each stage, the parameters that govern their behaviour, and the reasoning behind each decision. Models and tools are discussed at the point where they enter the pipeline, together with alternatives that were tested and set aside.

The chapter follows the path of the data through the system.

6.1 Codebase and Configuration

The system is implemented in Python 3.13, and its source code is publicly available in the project repository[1]. The codebase mirrors the division between the two pipelines described in Chapter 4. The ingestion side runs offline and produces a set of persistent weights, while the inference side only reads them without ever modifying the corpus.

6.1.1 Repository Layout

The main modules of the codebase and their responsibilities are summarised in Table 6.1. Four entry points sit at the top level. The guideline scraping process described in Chapter 5 is managed in the `scrape_guidelines.py` Python file. The `build_index.py` script is responsible for running the offline ingestion pipeline and writing the retrieval weights and indices to disk, while `main.py` orchestrates the inference pipeline for a single clinical note. The Streamlit application in `app.py` manages the same pipeline through a graphical interface. A `config.py` file contains the global variables used to determine configurations, model selection and their parameters. All modules read their settings from this file, described in Section 6.1.2.

The remaining packages are organised by pipeline stage, so that each of them corresponds to one of the sections of this chapter. The `scraping/` and `evaluation/` packages, finally, sit outside both pipelines: the former contains the scripts necessary to build the guideline corpus, while the latter supports the experiments reported in Chapter 7.

[1] `https://github.com/damianozaccaron/ERDischargeRecommendationsRAG/`

Module or entry point	Responsibility
`main.py`	Orchestrates end-to-end inference for clinical notes, including translation, query construction, retrieval, reranking, table retrieval, prompt assembly and generation.
`app.py`	Provides the Streamlit workflow for uploading and parsing a clinical report, selecting query and retrieval modes, displaying the assembled prompt, and rebuilding the index.
`build_index.py`	Coordinates offline processing of guideline PDFs, embedding of text chunks and table representations, and construction and persistence of the retrieval weights and indices.
`scrape_guidelines.py`	Manages the guideline corpus pipeline in its entirety, from topic selection to manual download of files.
`ingestion/`	Contains the PDF parsing, text chunking and table-processing modules used to prepare guideline content for indexing.
`storage/`	Implements document and query embedding functions, and the creation, loading and saving of FAISS indexes, chunk metadata, the TF-IDF vectorizer and the table store.
`pipeline/`	Handles clinical-note input and parsing, anonymization, extraction of patient fields, and Italian-English translation.
`retrieval/`	Implements query construction, dense candidate retrieval, cross-encoder reranking and the second-stage retrieval of guideline tables.
`generation/`	Defines the prompt templates and output-generation backends, including local inference and the OpenRouter API.
`scraping/`, `evaluation/`	Modules for guideline corpus construction and evaluation tasks, respectively.
`config.py`	Centralises configurable model names, paths, retrieval parameters and backend settings used by the application.

Table 6.1. Main code modules and their responsibilities.

6.1.2 Configuration

All tunable settings are collected in a single configuration module, `config.py`, which holds model identifiers, file paths and the parameters of each stage, such as the number of candidates retrieved per query or the number of excerpts passed to the generator. Since every backend is selected through a configuration value rather than in the code itself, different combinations can be tested on the same notes while the rest of the pipeline stays unchanged.

Three components have interchangeable implementations. Translation can be delegated to the DeepL API[171] or to a Large Language Model, normally the one that generates the recommendations, with both options drawing on a shared glossary for consistent clinical translations (Section 6.3). Query construction can rely on the same language model to formulate the retrieval queries, or fall back on an automatic procedure based on Named Entity Recognition that requires no external call (Section 6.4). Generation, in turn, can be routed either to a model served through OpenRouter[172] or to one running locally (Section 6.5).

6.1.3 Development and Runtime Environment

All development and testing were carried out on a laptop equipped with an NVIDIA RTX 4050 GPU with 6 GB of VRAM, Intel Core i7–12700H CPU, 16GB DDR4 RAM running on Ubuntu 24.04. This hardware shaped several of the choices described in the rest of the chapter. The embedding model and the cross-encoder run locally during both ingestion and inference, and must therefore fit in the available memory alongside each other. Both were consequently chosen among compact models, accepting that larger encoders might yield better retrieval.

The generation model, by contrast, is accessed through OpenRouter. As no machine capable of hosting large open-weight models was available, the local deployment tiers evaluated in Chapter 7 were simulated by querying the same open-weight models through the API. Output quality obtained this way approximates that of a local installation. Latency figures, however, reflect the provider's infrastructure and may vary vastly when deployed in local machines. In all configurations, models are loaded once at startup and kept in memory for the whole session, so loading time does not weigh on the processing of individual notes.

The main libraries on which the implementation relies are listed in Table 6.2.

Library	**Role in the system**
PyTorch[173]	Tensor backend for all locally executed models
Transformers[50]	Loading and inference of the named entity recognition model
Sentence-Transformers[174]	Embedding model and cross-encoder
FAISS[175]	Vector index and similarity search
PyMuPDF[176]	PDF parsing, layout analysis and table extraction
NLTK[177]	Sentence segmentation
scikit-learn[178]	Vocabulary and inverse document frequency computation
Streamlit[179]	User interface

Table 6.2. Main libraries and services used in the implementation.

6.2 Guideline Ingestion

The ingestion pipeline turns the guideline corpus, described in Chapter 5, into the weights and indices searched by the inference pipeline. It runs offline through the `build_index.py` script, and only needs to be repeated when the corpus changes. Each guideline goes through four stages: its pages are parsed into text blocks and tables, the text undergoes a cleaning process to remove as much of what is not guidance as possible, the remaining prose is segmented and grouped into chunks, and chunks and tables are finally embedded and stored. On the current corpus of guidelines, amounting to 12,528 pages, the process returns about 46,000 text chunks and about 2,000 structured tables.

Two principles guide the stage design and implementation. Cleaning is kept deliberately conservative: a noisy chunk is not likely to be retrieved, and the cross-can still demote it if it is. on the other hand, a deleted recommendation can never be retrieved, therefore filters remove only text that is unambigously noise. Structure is recovered from the geometry and typography of the PDF rather than from specific wording of publishers. In fact, since the corpus gathers documents from dozens of societies, and no layout convention among them exists, a rule fitted to one publisher's conventions would silently fail on the others.

6.2.1 PDF Parsing

Guidelines are parsed using PyMuPDF[176] and its layout model. PyMuPDF is fast and exposes everything needed to rebuild the structure of a page with rules: the position of every block, the font, size and flags of every text span, and the vector drawings that form table borders. The layout model is used only where these signals fall short, namely to locate tables drawn without borders and to label page headers and footers.

Before any text is read, each page is normalised. Rotated pages, as detected by PyMuPDF, are re orientated upright, and landscape tables printed sideways on portrait pages are redrawn as upright landscape pages, so that text blocks and table cells share the same coordinate system. The page is then divided into two streams. Tables are located and read first (Section 6.2.4), and are subtracted from the page, together with captions and header rows lying outside the detected box. The remaining content is read block by block as prose.

Citation markers are removed at this stage, relying on typography rather than on patterns. A text span is dropped when it is flagged as superscript, or when it is noticeably smaller than the preceding text and raised above its baseline. A pattern-based rule could not achieve the same result, as a reference number attached to a word is indistinguishable from the digits in names such as CHA_2DS_2-VA, SGLT2 or P2Y12, which must be preserved. A residual pattern only removes numbers attached directly after punctuation, where no drug or score name can occur.

Since each PDF is completely independent of one another, extraction runs in parallel, with one process per guideline. The number of workers depends on the available cores and memory. Running each document in its own process also isolates failures.

6.2.2 Cleaning

Raw extraction contains a substantial amount of text that is ultimately not useful for recommendation production. Running headers, footers and page numbers repeat on every page, while bibliographies, indexes and abbreviation lists pose a subtler problem: they are dense in clinical terms and therefore ahve high potential of scoring well against almost queries, despite carrying no actionable information themselves. Noise is removed at three different levels: pages, blocks and lines.

Page furniture is handled first. Blocks lying entirely within the outer 5% of the page, or shorter than ten characters, are discarded, since that space is reserved for running headers, footers and page numbers. This approach, while conservative, proved relatively successful and allowed to remove the noise in the majority of the pages, though it left significant room for improvement. An attempt at using labels of the layout model to identify them performed unsatisfactorily, as the model could not consistently identify them and even occasionally labelled entire pages of recommendations as headers. The adopted rule therefore combines the two signals: a block is removed when the layout model labels it as a page header or footer and its centre lies within 8% of the page edge, or if it is positioned within the 5% margin in any case.

Pages that are recognisably part of a bibliography or an index are then discarded as a whole, based on the ratio of lines carrying journal locators, DOIs or links, and on cues such as lines ending in page numbers or dot leaders.

On the remaining pages, reference lists and abbreviation lists are identified block by block. A block is attributed to a reference list according to the density of journal locators it contains, and to an abbreviation list only on pages holding many acronyms on lines of their own, mostly in alphabetical order. This block-level removal replaced an earlier page-level one, which lost prose text printed on the same page as the end of a reference list or next to an abbreviation table. While these blocks rarely contained useful guidance, the more conservative approach was favored, in line with the design principles introduced in Section 4.1.2.

The surviving prose is finally split into lines. Lines made up mostly of non-ASCII characters, typically garbled text deriving from failed figure parsing, are dropped, and reference numbers attached after punctuation are stripped. Lines are then merged back into readable sentences. A line ending in a hyphen is joined to the next one without it, reconstructing words split at line breaks, whereas a line ending in punctuation closes the unit. Dashes followed by a digit are the exception and are kept, since removing them would fuse a range such as 35–40 mg/kg into a single, incorrect value.

6.2.3 Sentence Segmentation and Chunking

Textual units are split into sentences with NLTK's implementation[177] of the Punkt algorithm[105]. Since Punkt learns abbreviations from the text it is trained on without supervision, the model was trained on the guideline corpus itself, where it was able to picks up domain abbreviations, such as *i.v.* or m.g., that the pretrained model missed on. Its errors were concentrated on a handful of general abbreviations, such as *vs.*, *Dr.* and page references, which were added to the model as a short curated list. Importing the full abbreviation list of the pretrained model was also tried, but it introduced new errors: Punkt matches abbreviations regardless of case, so entries such as *ms* or *ct* suppressed sentence breaks after the medical acronyms MS or CT.

After segmentation, sentences are grouped into chunks of up to 1,200 characters, with adjacent chunks sharing two sentences so that context is preserved across boundaries. Grouping by sentences instead of using a blind character split means chunks never contain split words or sentences, which would only serve to provide noise. Sentences exceeding the limit on their own are split at semicolons, colons and similar punctuation. A chunk never spans two guidelines, and it records the pages it was drawn from, so that every excerpt can be traced back to its source. The size was chosen with the cross-encoder in mind: a median chunk of 1,131 characters corresponds to about 270 tokens, which fits within the reranker's 512-token window together with the query, while remaining long enough to contain a recommendation with its conditions and context. Fewer than 1% of chunks exceed the window, in which case the reranker scores a truncated text, while the generator still receives the full chunk.

6.2.4 Table Extraction

Tables required a dedicated treatment. They could not be extracted as falt text, as trying that led to broken formatting, detached values and generally unreadable outputs for both machines and humans. it also left the lines extracted exposed to the line-level cleaning rules described above, which were explicitly designed for text prose and would have damaged cell content. Moreover, because they contain important information, as drug choices and doses are often reported in tables, they could not simply be ignored. Therefore, they are read as structured objects and never pass through prose cleaning.

Tables are located with the table finder function of PyMuPDF, assisted by the layout model for those drawn without borders. Each detected region is read as a stack of title rows, caption, header and body. Captions are recognised by labels in several forms and languages (Table, Box, Recommendation, Tableau, and others), searched above, below and beside the table. The header is identified, in order of preference, by a style that no later row uses, by a rule drawn beneath it, or by the shape of the first row. None of these rules depends on a specific publisher's vocabulary.

However, the finder is not infallible and also returns regions that are not tables in any useful sense. Regions with a single row or column are treated as text boxes and returned to the prose stream, as are reference lists laid out as grids, which are then removed by the reference filter. Tables of contents, abbreviation lists, evidence tables and study listings are discarded, and regions whose cells hold paragraphs rather than labels are rejected as running text. For the remaining tables, cell text is read directly from the page within the boundaries of each cell, thus preserving the spacing inside values such as 2.5–5 mg, and footnote markers are dropped by font size, as in the prose. Tables continuing across a page break are merged, and cells spanning several rows are copied to every row they cover.

Each table is then classified by its shape. Recommendation tables, whose rows mostly hold a single long sentence, are converted into text and packed into chunks like prose, with the caption repeated in every chunk and the class and level of evidence kept next to each recommendation. Tables made mostly of bare numbers, typically statistical results which are not useful for clinical recommendations, are discarded. All other tables, which include dosing tables, drug comparisons and scoring systems, are kept as grid tables in a dedicated table store.

Grid tables are serialised as Markdown, preserving their header and row grouping, and tables longer than 4,000 characters are split at group boundaries, with each part keeping caption and header. What gets embedded, however, is not the Markdown but a short surrogate made of the caption, the header, the group labels and the cells of the key column. In fact, a dosing table shares few words with a narrative query, and its caption and drug names are the parts a query can actually match. Each row also receives its own textual surrogate, which the cross-encoder scores at query time, and a set of normalised tokens for lexical matching, so that variants such as β-blockers and betablocker are treated as the same word.

6.2.5 Embedding

The embedding model is subject to three requirements. It runs locally, both during ingestion over the whole corpus and at query time for every query, on a 6 GB GPU that it shares with the NER model and the cross-encoder. It must map English guideline prose and short clinical queries into the same space. Finally, it must distinguish neighbouring conditions whose guidelines share most of their vocabulary, such as atrial fibrillation and supraventricular tachycardia. The models considered are summarised in Table 6.3.

The early candidates were compared informally, on retrieval obtained for example notes. Bio_ClinicalBERT produced near-random results, and was therefore discarded. Likewise, the leaders of the healthcare category of the MTEB leaderboard were excluded on practical grounds: the top model is available only through an external API, which would send clinical text off-site, while the strongest open models, with around 8 billion parameters, far exceeded

Model	Characteristics	Outcome
MTEB healthcare leaders[133]	voyage-4, Octen-Embedding-8B	API-only or too large for the GPU
Bio_ClinicalBERT[63]	BERT further trained on clinical notes, not trained for sentence similarity	Near-random retrieval
ModernPubMedBERT[180]	ModernBERT trained contrastively on PubMed title-abstract pairs, 0.1B parameters, 0.57 GB	Dropped
Nemotron-3-Embed-1B[181]	General-purpose and multilingual, 1.1 billion parameters, 2.2 GB	Adopted

Table 6.3. Embedding models considered for the system.

the available memory.

Therefore, a decision was made to evaluate two candidates of different nature who fit inside the stringent system requirements: ModernPubMedBERT, a BERT variation trained specifically on PubMed data, representing a small and fast model specialized in clinical domains; and Nemotron-3-Embed-1B, a larger general-purpose multilingual model.

The two models were tested on an end-to-end comparison on the 25 synthetic clinical notes used in the evaluation (Chapter 7). The whole retrieval stage was run with each embedder, and a blind LLM judge marked each final excerpt as useful or not for the discharge of that patient. With automatically constructed queries, Nemotron produced 76% more useful excerpts. The gap was narrower with LLM-written queries (+15%). The difference between the two settings is consistent with the additional role the embedder plays in automatic query construction, where it also selects the guideline that the queries are restricted to (Section 6.4): a wrong choice at this step removes the relevant guideline from several queries at once. Since the improvement in retrieval quality was deemed to be far more beneficial than the cost of a larger model, Nemotron-3-Embed-1B was ultimately selected. Its multilinguality, however, is not exploited. Translation remains necessary because the NER model, the negation rules and the cross-encoder all operate on English text (Section 6.3). Even excluding those models, the retrieval quality on a language other than English remains untested.

Like most recent embedders, the model is asymmetric: it expects a different prompt for passages than for queries, so that a short question and the longer text answering it are mapped to nearby points. Guideline chunks and queries are therefore encoded with their respective prompts. All vectors are then L2-normalised, so that the inner product equals the cosine similarity, which is what the index in the following section computes.

6.2.6 Vector Indexes and Lexical Resources

Chunk vectors are stored in a FAISS[175] flat inner-product index, which performs exact search. A dedicated vector database was considered and set aside, as the corpus fits comfortably in memory and is always rebuilt as a whole, so a database would add weight and costs without solving an actual problem. At the current size, an exact search over about 46,000 vectors takes a few milliseconds on a CPU, so using approximate indexes for search would make no practical sense. Exact search also keeps retrieval deterministic. The metadata of each chunk, namely its text, source guideline, page range and kind, are kept in a list aligned by position

with the index and serialised alongside it.

Grid tables have a separate index over their surrogates, together with a store holding their Markdown, row surrogates and token sets. Keeping them apart from the prose is necessary because the similarity scores of a table surrogate and of a prose chunk are not comparable, and because tables are allotted their own slots in the final prompt.

A TF-IDF vectoriser from scikit-learn[178] is also fitted on the chunk texts, though it is not used for retrieval. Its vocabulary serves to repair words damaged by translation or typing errors, while the Inverse Document Frequency distinguishes terms that identify a specific topic from those spread across the corpus. Both functionalities are utilised in the rule-based query construction (Section 6.4).

File	Content	Size
`faiss.index`	Flat inner-product index over the text chunks	378 MB
`metadata.pkl`	Text, source guideline, page range and kind of each chunk, aligned with the index	53 MB
`tables.index`	Flat inner-product index over the table surrogates	17 MB
`tables.pkl`	Grid tables with caption, header, row surrogates and token sets	14 MB
`tfidf_vectorizer.joblib`	Vocabulary and inverse document frequencies of the chunk corpus	1.6 MB

Table 6.4. Stored files produced by the ingestion pipeline.

6.3 Input Processing

The inference pipeline starts from the provisional document printed by the department's software at the end of an Emergency Room visit, the *Verbale di Pronto Soccorso*. Before any retrieval can take place, this document has to be turned into an anonymised English note whose sections can be located reliably. Therefore, once a report is received, its text is extracted and anonymised in a single pass over the PDF, so that no text leaves the parser and the local environment before being anonymised. The note is then translated into English, and finally split into the fields used by the later stages. Notes already typed as plain text following the same template are accepted as well, in which case reading and anonymisation are skipped and the text is assumed to contain no identifiers. This is because the option is meant as a fallback and used mainly for testing purposes. For this reason, the process described in Section 6.3.1 considers PDF documents as the source of input.

The parser was developed based on the layout and content of three documents: a provisional and a final version of the department's print, available as two sample reports, and the version used for the 25 notes written by the medical team for the evaluation (Chapter 7). Unlike the guideline corpus, where structure-based rules were utilised, these documents follow a single template, so vocabulary-based rules can be reliably used to locate sections.

6.3.1 Reading the Report

The report is read with PyMuPDF[176] as a sequence of spans, the smallest runs of text that share the same font, size and style. Each span carries its text, font and position on the page. Spans are then grouped into lines based on their position. Reading follows a conservative approach: only text positively recognised as clinical content is kept, and everything else is discarded by default. This choice is driven by privacy rather than by noise. In fact, a report contains personal data in many places: the patient's identity in the header, the names of the staff in the records, and administrative data in most sections. Listing what to remove (a deny-list) is therefore unsafe, as it would let through any new field added by a future version of the software. Listing what to keep (an allow-list) is more robust to those changes, and while such changes may cause failures, they would not silently leak data.

Page headers and footers are removed first, after the patient's sex and age have been read from the header, since retrieval makes use of them. Section titles are then recognised as the leftmost bold lines of the page body. Five sections are kept: the medical history, the objective examination, the clinical diary, the procedures and the nursing activities. The last two record what was done and ordered during the stay, such as a splint, sutures or the drugs administered, from which much of the discharge advice follows. Routine entries present in every visit, such as blood sampling, ECG or body temperature, are removed from them. In fact, if any of those values are relevant for the patient case, the physicians will report it in either the clinical diary or the objective examination. All other sections, from triage to vital signs and fall-risk assessment, are dropped, as will be any section introduced by a future layout.

Within the kept sections, only running text survives. Lines split into several cells by wide horizontal gaps are table rows, while lines starting outside the text column are typically signatures or form fields, so both are removed. Stand-alone dates and times, the names of the staff who wrote each record and log lines generated by the software are removed as well.

Once the clinical lines have been isolated, two further steps turn them into a readable note.

The first is to rebuild the sentences. Each line is joined to the previous one whenever the break between them looks accidental. This is assumed when the previous line ends on a word that cannot close a sentence (such as a preposition or an article), when the new line opens with a conjunction, when its first word would have fitted at the end of the previous line, or when a page break cut the sentence in two. While this might seems obvious to a reader thinking about normal text files, it is important to notice that a clinical note is comprised of many standalone lines rather than text walls. Moreover, due to the many abbreviations and lack of punctuation by the medical staff when writing them, normal grammar rules proved ineffective in identifying sentences over multiple lines, and ad hoc rules had to be created.

The second step concerns the examination checklist, which prints *Si* (yes) next to body regions with nothing to report. This is a problem for translation: a line such as *Segni di TVP: Si* would be rendered as *Signs of DVT: Yes*, which reads as the opposite of what is meant. These lines are therefore rewritten as followed by *nessun reperto patologico* (no pathological findings) before the note is translated.

Finally, the diagnosis and prognosis are taken from the discharge section and appended at the end of the note. The prognosis is completed with the outcome of the visit, such as an admission to a specific ward, because this determines whether the patient is discharged

home or kept in hospital (Section 6.3.5).

The parser is designed to fail closed: a file that does not look like a report, identified by the presence of certain identifiers such as section names, is refused with a message for the physician, rather than processed partially.

6.3.2 Anonymisation

The allow-list reading already removes most identifiers, because headers, forms and staff stamps never enter the note. Names can nevertheless reappear inside the running text, for instance when a diary entry mentions the consultant who was called. A second pass is therefore dedicated to names, and works in three steps.

1. **Known names.** The software prints some names in recognisable places: the patient's name in the header, and each staff member's name in record stamps and signature fields. All of these are collected, and each of their words is removed wherever it appears in the text, regardless of case, accents and word order. Name particles such as *de* or *di* are the exception, since they are also common Italian words.
2. **Titled names.** Names introduced by a title (*dott.ssa*, *dr.*, *sig.ra*) are removed even when the software never printed them elsewhere. The abbreviation *prof.* is not treated as a title, because in clinical text it also stands for *profonda* (deep). Nonetheless, it is not a commonly used abbreviation in clinical notes.
3. **Formatted identifiers.** CAP codes, phone numbers and e-mail addresses are removed by pattern.

Every removed span is replaced with the placeholder *[omissis]*, so that the sentence remains readable and it is clear that something was taken out.

While the method proved effective in the notes utilised for this thesis, it presents potential weaknesses in identifiers that the software never printed as such, such as a relative mentioned by name without a title, a place of residence outside of the anagraphical date, or the date of an earlier admission. Anonymisation should therefore be seen as a mitigation rather than a guarantee, in line with the privacy considerations of Section 4.1.3; its residual risks are discussed in Chapter 8.

6.3.3 Separation of the Physician's Advice

The clinical notes used for evaluation contain the discharge advice written by the physicians. This advice has a special role: it is the reference against which the output of the system is evaluated (Chapter 7). For the same reason, it must never be part of the system's input. This is done to reflect the practical case, where the system is meant to run before the physician writes the advice. A model that saw the advice could simply copy it, so that the evaluation would measure nothing. The parser therefore separates the advice from the note and stores it apart.

The advice appears in three kinds of places, each handled differently.
First, the prescription field of the discharge section, where most of the advice sits, is removed, together with the rest of that section.
Second, the field describing the patient's condition at discharge, can contain: a description of

the patient's state, which is useful and stays in the note; and instructions, which are moved to the advice.
Finally, the kept sections, most often the diary. Here the advice is recognised by its wording and moved out of the note. This covers sentences opening with an advice heading or an impersonal advice verb (*si consiglia*, *si prescrive*), referrals back to the general practitioner, requests for further tests and conditions for returning to the department. Diary lines that merely restate the final advice are deleted.

6.3.4 Translation

Notes are written in Italian, while the guideline corpus and the components of the following stages (the NER model, the negation rules and the cross-encoder) all work in English. Translating the note once, before any other component reads it, keeps every later stage monolingual. Translation also solves a second problem: Italian drug brand names, which guidelines never use and are therefore difficult to retrieve, are replaced with the names of their active substances.

Choice of engine. Running a translation model locally would have been preferable, as the note would then stay on site, so local models were tested first. However, smaller models like Opus-MT[182] produced extremely unsatisfactory and inconsistent translations of clinical text, and bigger ones like MADLAD-400[183] required over 500 seconds per note on the available machine, and is therefore too slow for any practical use. Therefore, two external engines are available instead: DeepL[171], a dedicated machine translation service; and a Large Language Model, normally the same one the pipeline already calls to construct queries or to generate recommendations. Using a local deployment of the LLM would effectively eliminate all privacy problems, as the whole pipeline would never leave the machine. The experiments discussed in Section 7.2 show that this would represent a concrete and feasible solution. Nonetheless, it would require significant investments to acquire the necessary hardware.

Shared principle. Both engines follow the same principle: the engine only translates the free text, while everything the later stages depend on is decided in code. The section labels are never left to the engine: each Italian label is mapped in code to a fixed English one, because the fields of the note are later located by their English labels (Section 6.3.5), and a label translated differently by the engine silently empties its field. The risk is real: on the synthetic notes, DeepL rendered the label of the presenting complaint (*Patologica prossima*) in three different ways.

Translation with DeepL. The note is sent section by section, each section prefixed with its Italian label to give the engine context. This is needed especially when the content of the section is short: in a case where the remote history only contained the term *Muta*, to indicate it was not available, *Patologica remota: Muta* correctly translated as an unremarkable history, whereas the word *Muta* on its own was translated as a wetsuit. The label returned by the engine is then ignored and replaced in code with its fixed English version. The line holding sex and age is written directly in code as well.

Translation with a language model. The prompt asks for a literal and complete translation, in which nothing is added, removed or corrected, numbers, doses and units are left unchanged and glossary terms are used whenever they occur. Two details were found by trial and error. Section labels are replaced with their English versions before the call, because

a model interpreted the instruction to keep each header as an instruction not to translate it. Moreover, only the glossary entries whose Italian term appears in the note are included in the prompt (usually a few dozen), because with the full glossary the model ignored the glossaty and left brand names untranslated.
The call runs at temperature zero, so that a note receives nearly the same translation on every run. The prompt fed to the language model can be found in Appendix B.1.

The glossary. The glossary is kept in a single file of 545 entries, of two kinds. About 95 are section labels, abbreviations and expressions specific to Italian medicine, such as *Giordano* for CVA tenderness. The other 450 or so map Italian brand names to active substances, such as *Clexane* to enoxaparin. The brand mappings serve retrieval rather than readability: a brand name left in the English note is unknown to the embedder, matches no guideline passage and escapes the lexical matching of the table stage (Section 6.4.5).

6.3.5 Field Parsing and Routing

The translated note is a single block of text, so it is split back into the fields used by the later stages. These are identified by their fixed English labels: the history (with presenting complaint, remote history, drug history and allergies), followed by the examination, the diary, the procedures, the treatments ordered in the department, the diagnosis and the prognosis. Each field extends from its label to the next label present in the note. This is necessary because reports may lack some sections: of the 25 synthetic notes, 4 have no objective examination and 8 no drug history. Diagnosis and prognosis close the note, so their last occurrence is used, which prevents a mention of a previous diagnosis in the history from being mistaken for the current one. Sex and age are read from the first line, where it is hardcoded in translation, with an age expressed solely in months identifying an infant.

Routing. The prognosis also decides which of two paths the rest of the pipeline follows, depending on whether the patient is discharged home or is admitted. A patient is considered admitted when the prognosis mentions an admission, a hospitalisation or a transfer, unless it also records the patient's refusal, in which case the patient is discharged home. The route then affects three things: the templates used for rule-based query construction or the prompt given to the LLM query writer, whether tables are retrieved, and the prompt template used for generation (Sections 6.4 and 6.5). Of the 25 synthetic notes, 4 are admissions and 21 are discharges home.

6.4 Query Construction and Retrieval

Once the note has been parsed, the system has to find the guideline passages that relate the most to it. Earlier experiments showed that using the whole note as a single query is not a viable approach. This is because a long text touching on many topics is embedded somewhere near the average of all of them, and the passages closest to it are generic ones that match none of those topics closely. Moreover, on long texts, the embedder may give more weight to the initial part of the notes, ignoring important information stored in later sections. The note is therefore broken down into several short queries, each targeting a single aspect of the case, to allow the system to retrieve relevant excerpts for specific conditions, comorbidities, symptoms and drugs rather than general ones relating to the letter broadly.

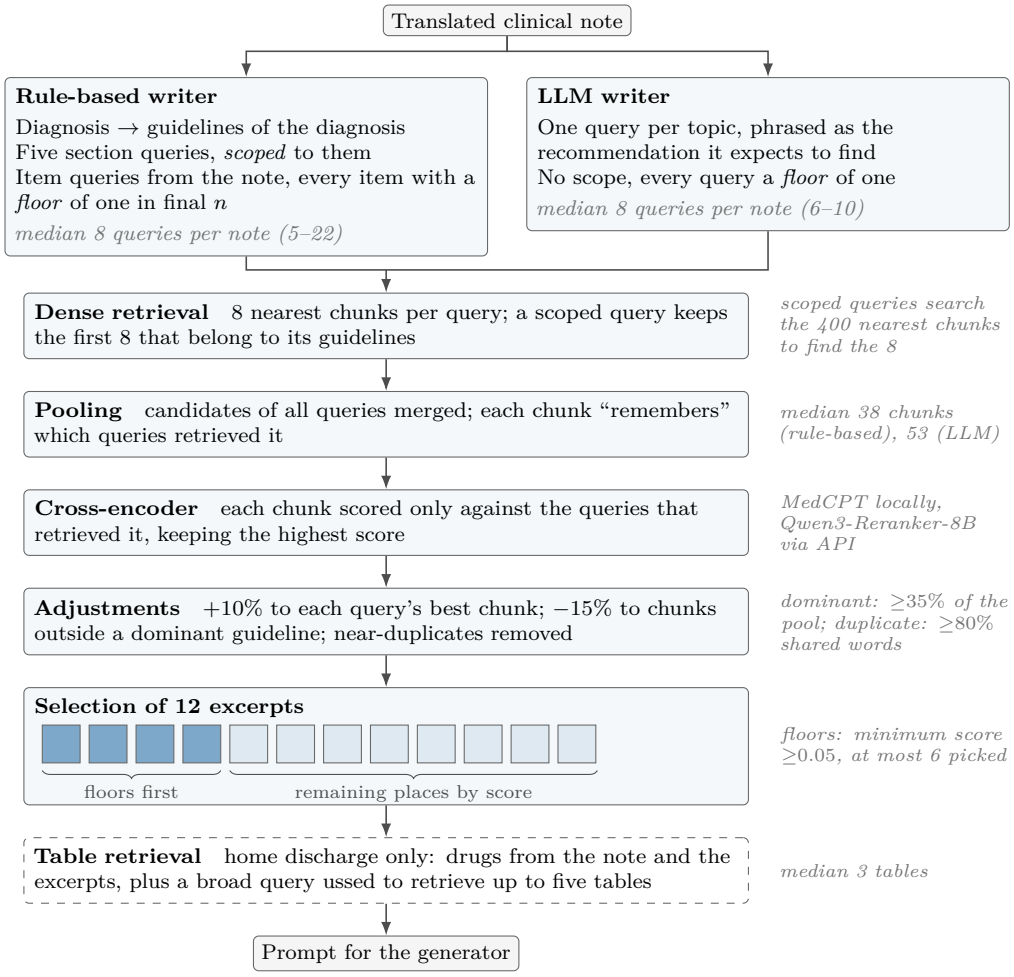


Figure 6.1. The procedure of the retrieval of guideline excerpts for a clinical note.

The system is designed to provide two separate options of writing the queries. A rule-based procedure builds them from the entities found in the note and needs no language model, making it suitable for a configuration with less computational power and free from API usage. Alternatively, an LLM reads the note and is tasked with writing the queries itself. Examples of resulting queries from a clinical note are included in Appendix C.2. To keep the rest of the pipeline independent of this choice, both writers return their queries together with two optional attributes. The *scope* of a query is the set of guidelines it is allowed to search, and is used to confine generic queries to the guidelines that are known to be about the diagnosis. The *floor* of a query is the number of places it is guaranteed among the final excerpts, and protects aspects of the case that would otherwise be crowded out by passages about the diagnosis. If the language model fails or returns nothing usable, the pipeline sends a warning and falls back on the rule-based writer, rather than proceeding without queries.

Each query is then used to retrieve the candidates from the vector index, and all resulting candidates are then pooled, reranked by a cross-encoder and reduced to the final set of

excerpts (Figure 6.1). For patients discharged home, a second step also retrieves the tables of the drugs involved, a step that is skipped in case of admission into other departments in the hospital. The figures reported in this section refer to the 25 synthetic notes of the evaluation (Chapter 7), and the main parameters are summarised in Table 6.5.

6.4.1 Rule-Based Query Construction

Every rule-based query is built around the diagnosis, which was found during development to be the most informative part of the note, and phrased to imitate the caption of a guideline recommendation table: *Recommendations for [item] in patients with [diagnosis].* Guideline showed better responses to this form than to a question or a list of keywords, since it resembles the way guidelines introduce their own recommendations. The words surrounding the item are kept to a minimum, because the cross-encoder matches them just as readily as the clinical content. Therefore, no universal anchor term is used to complement the query.

Before any query is built, the note is normalised. Lines written entirely in capital letters, which the NER model tends to fragment and the embedding model tends to overrate, are converted to sentence case, and words misspelled by the physician or damaged by translation are substituted by the closest word contained in the vocabulary of the corpus. It is important to note that these corrections apply to the queries only. The note passed to the generator is left untouched, so that a wrong correction can never alter the name of a drug in a prescription. This way, even if the retrieval mistakenly provides a wrong excerpt, the generator can ignore it in the final output construction.

Queries of this form are of two kinds, which differ in their subject and in the way they are retrieved. *Section queries* concern the diagnosis itself and ask for the topics any discharge document needs, such as treatment or follow-up. Since such generic questions could be answered by almost any guideline, they are confined to the guidelines that are about the diagnosis. *Item queries*, instead, take their subject from the rest of the note, such as a comorbidity or a drug given during the stay, and most of them are guaranteed a place among the final excerpts.

Diagnosis and Section Queries

Since the diagnosis appears in every query, its wording is cleaned as well. Hedges such as *suspected* are removed, as is the side of the body or a part of it, since no guideline distinguishes the left leg from the right, and a diagnosis listing several conditions is split into its components. Long names are also reduced to their core, so that *deep vein thrombosis of the gastrocnemius veins* becomes *deep vein thrombosis*. The reason lies in the behaviour of the cross-encoder, which lowers the score of a passage for every element of the query it does not mention: an overly specific name penalises precisely the general passages where the relevant recommendations are found.

The next step identifies the guidelines that are closest to the diagnosis. The reason for the implementation of this procedure is to limit the amount of noise in the final retrieval. In fact, especially in a large corpus comprising more than 400 documents, different guidelines may contain similar excerpts pertaining to contraindications or comorbidities of actual conditions, while not being relevant for the specific case of the clinical note in question. Therefore, the name of each condition, as identified by the NER model, is searched on its own, without any

surrounding words, and every guideline supplying at least a fifth of its 20 nearest chunks is taken as one of the guidelines of the diagnosis. Searching the bare name avoids the risk that the wording of a longer query pulls the search towards whichever guideline best matches that wording.

To avoid confining the retrieval to a restricted set of guidelines when the conditions are not completely covered by any document in the corpus, a guideline is only accepted if its closest chunk reaches a cosine similarity of at least 0.33 with the name of the condition. The threshold was set heuristically after experimentation. Overall, the step found one guideline of the diagnosis in 9 notes, two or three in 10 and none in 6, four of which concern diagnoses that the corpus does not cover. When no guideline is found, the queries simply search the whole corpus.

With the guidelines of the diagnosis known, five fixed *section queries* cover what any discharge document needs, whatever the diagnosis. For a patient going home, they ask for the recommended treatment, drug therapy and dosage, follow-up, lifestyle and risk factor management, and complications and warning signs. For an admitted patient, follow-up and lifestyle are replaced by monitoring and by diagnosis and assessment, which are more relevant to the team taking over the case. Section queries are scoped to the guidelines of the diagnosis. Without this restriction, a generic query about follow-up would match the follow-up section of every guideline in the corpus, and the cross-encoder would score these passages as highly as the relevant ones.

Items from the Clinical Note

The remaining rule-based queries cover what the note says about the patient beyond the diagnosis, such as comorbidities, home therapies and what was done during the stay. Each of these *items* becomes the subject of a query of the form described above. Items are found through the usage of the NER model *Medical-NER*[184], which identifies the conditions, drugs and procedures mentioned in the text.

Nonetheless, different parts of the note call for different treatment. In the remote history, a sentence naming a single comorbidity, home drug or habit becomes an item as a whole, since its context may span several clauses providing more useful context than the item alone, as in *former smoker, approximately one pack per day.* Sentences listing several conditions and the drug history are instead split into *listed pieces*, one per condition or drug, which are subject to a stricter selection described below, since a home therapy can easily include many drugs. The clinical diary, the procedures and the treatments ordered in the department are cut into clauses. A clause naming a drug, a procedure the patient goes home with, such as sutures or a cast, or a clinical decision, such as an indication for anticoagulant therapy, becomes an item. Clauses about vital signs, routine care, test orders or drugs that were not administered are discarded. Each item is finally stripped of measured values, irrelevant in retrieval, and of the narrative around it, so that *5 mg metoprolol tartrate is administered intravenously* becomes *metoprolol tartrate intravenously.* Items longer than twelve words are discarded, as they usually report the physician's reasoning rather than a single item.

Not every entity mentioned in a note is relevant to the patient, however. A note may record that the patient *denies chest pain*, or that a condition was treated years ago, and a query built on such an entity would retrieve guidance for a problem the patient does not currently have. Entities are therefore discarded when they are negated or refer to the past (Section 3.4.2). Negation cues such as *no*, *denies* or *without* are searched for in a short

window before the entity, which ends at the previous clause boundary. Commas require particular care, since a negation often extends over a list. In *denies chest pain, syncope or presyncope* all three symptoms are negated, while in *denies chest pain, reports migraines* the negation ends at the comma, because a new verb reasserts what follows. Historical cues such as *former* or *underwent*, on the other hand, apply only to their own element of a list, and dates mark an event as past unless they indicate an onset, as in *since 2010*. These rules were compared with an established algorithm for negation clause recognition, NegEx[35], and were found to be more precise and effective.

Still, not all the surviving items deserve a query. Findings (abnormal results of the examinations or laboratory tests) are the least specific of them, and most findings, such as a slightly raised inflammatory marker, are simply not discussed in the guideline of the diagnosis. Each finding is therefore searched in the whole index and kept only if at least 30% of its nearest chunks come from the guidelines of the diagnosis.

Listed pieces (clauses created by splitting sentences containing multiple conditions) are kept only when the guideline of the diagnosis actually mentions them, which is measured by counting the chunks of that guideline that contain all the meaningful words of the piece. Words appearing in more than 5% of the chunks of the corpus are ignored, as they are too common to identify a topic, and this is where the inverse document frequencies computed at ingestion come into play (Equation (3.5)). At most three findings and three listed pieces are kept, and both are restricted to the guidelines of the diagnosis, like the section queries.

The remaining items, namely the comorbidities, home drugs and habits stated in single-item sentences of the remote history, and the drugs, procedures and clinical decisions drawn from the diary, all receive a floor of one place. Without floors, the section queries would generally outscore a comorbidity, leading to an incomplete retrieval.

On the 25 notes, the rule-based writer produced between 5 and 22 queries per note (median 8), a number that grows with the length of the note, in a median time of 0.16 seconds.

6.4.2 LLM-Based Queries

In the second mode, a language model reads the translated note and is tasked with writing the queries itself. The model brings what the rules cannot: it knows how guidelines name a condition, it is not bound by semantic rules, and is more robust to writing errors from the compyling medical staff.

The prompt first explains the purpose of the search, which depends on the route of the patient. For a patient going home, the recommendations must tell the patient and the general practitioner what to take, how to manage other conditions and medications, what to change in their lifestyle, when to be seen again and which symptoms should prompt a return to the department. For an admitted patient, they only need to hand the case over to the admitting team.

The prompt then explains how the search works, namely that each query is compared with guideline passages and that the passages ranked highest are those that read most like the query. The instructions follow from this. Each query should be written as the recommendation the model expects to find, using the guidelines' own wording and the form of a declarative sentence rather than a question or a list of keywords. It should address

a single topic, name the condition as guidelines do, and be specific wherever the patient is, mentioning drugs, doses, thresholds or comorbidities, but only as stated in the note. Generic ward care and diagnostic work-up already completed in the department should not be searched for. The model writes one query per topic, usually five to eight and never more than ten.

Writing each query as the passage it hopes to find applies the idea behind hypothetical document embeddings[36,37]. Questions and their answers tend to occupy different regions of the embedding space, so a query phrased as an answer lands closer to the passages that actually contain one. The rule-based queries pursue the same goal with a fixed, caption-like sentence, whereas the language model can phrase each query as the specific recommendation it expects.

The call runs at temperature zero and with a low reasoning effort, since this call already accounts for most of the retrieval time. The output is read one query per line, discarding lines too short to be a query, such as an introduction to the list. This approach was deemed more secure than trusting the LLM to follow a specific formatting structure, as their adherence to such instructions can be inconsistent[185]. Every query receives a floor of one place but no scoe, since the guidelines of the diagnosis are not computed in this mode. Each query therefore searches the whole corpus, which also allows a query about a comorbidity to reach the guideline dedicated to that comorbidity.

On the 25 notes, the language model wrote between 6 and 10 queries per note (median 8), with a median length of 22 words against 12 for the rule-based ones. Its queries also overlap far less with each other: 86% of the chunks they retrieved were distinct, against 51% for the rule-based queries, whose section queries all search the same guidelines and return many of the same chunks.

Examples of queries produced through both the rule-based mode and the LLM mode are included in Appendix C.2, as well as the complete prompt passed to the language model to build them (Appendix B.1).

6.4.3 Dense Retrieval

Each query is embedded with the embedding model (Section 6.2.5) and compared with the whole index, which returns its eight most similar chunks. A query with a scope is searched more deeply: among the 400 nearest chunks, only the first eight that belong to its guidelines are kept. Since the search is exact and the index small, searching deeper costs very little.

The depth of eight chunks per query was chosen through a sweep reported in Chapter 7.

The candidates of all queries are finally pooled into a single set, in which each chunk keeps track of every query that retrieved it. On the 25 notes, the pool held a median of 38 chunks per note with the rule-based queries and 53 with the LLM-written ones.

6.4.4 Reranking and Selection

The embedder compares a query and a passage only through their vectors, which are computed separately and therefore cannot capture how the two texts relate to each other in detail. A cross-encoder reads the query and the passage together and judges their relevance based on a more complete text representation producing more accurate results[149], but it is too slow

to be applied to the whole corpus. Therefore, it is used only on the retrieved pool of a few dozen candidates. Its output is passed through a sigmoid function, so that every score lies between 0 and 1 and can be compared with fixed thresholds.

Four cross-encoders small enough to run on the development machine were compared: bge-reranker-large[186], MedCPT[131], Qwen3-Reranker-0.6B[187] and llama-nemotron-rerank-1b[188]. MedCPT, which was trained on biomedical search logs, performed best, slightly ahead of Qwen3-Reranker-0.6B and bge-reranker-large and well ahead of the Nemotron model, and is therefore used in the local configuration (Chapter 7). The configurations not bound to the available machinery use Qwen3-Reranker-8B through OpenRouter, an open-weight model that keeps the simulated local deployment realistic.

The final excerpts are then selected in a few steps. Each candidate is scored only against the queries that retrieved it, rather than against every query. A chunk can thus rank high only for a query that both the embedder and the cross-encoder consider it relevant to. On the example notes, this produced better excerpts than scoring every query against every candidate, and reduced the computational time fourfold.

A chunk retrieved by several queries keeps its highest score, following the CombMAX rule of Equation (3.13). The maximum function was selected as it rewards a chunk that answers one aspect of the case very well, which is what each query was built for. Summing the scores would instead favour chunks retrieved by many queries, which are typically generic passages about the diagnosis.

Two adjustments refine the ranking. The best chunk of each query receives a 10% boost, so that every query's best answer moves up. When one or more guidelines each supply at least 35% of the pool, chunks from all other guidelines are penalised by 15% of their score, on the assumption that a guideline dominating the pool is the one the case is about. The penalty is kept mild because the guideline of a comorbidity is among those penalised, and its best chunk remains protected by the floors described below.

Near-duplicates are then removed. Guidelines often repeat a recommendation, for instance in their summary and in the main text, and the corpus occasionally holds two versions of the same document. Going down the ranking, a chunk is dropped when at least 80% of its words already appear in a higher-ranked one.

Finally, the twelve excerpts passed to the generator are chosen. Queries with a floor are served first, each taking its best remaining chunk, provided it scores at least 0.05, below which a chunk is considered noise. The remaining places are filled in order of score. Floors exist because queries about the diagnosis would otherwise outscore those about a comorbidity or a drug of the stay everywhere, and the best-covered aspect of the case would take every place. For the same yet opposite reason, floor reserved spots are capped at half of the selection. A long note can produce two dozen items, and without a cap floors might fill every spot of the final selection, leaving no excerpt for the main diagnosis. Therefore, when there are more floors than available places, they are served in order of their best score, so that the queries the corpus answers worst are the ones that give way.

With rule-based queries, the selected excerpts came from a median of 3 guidelines per note, against 5 with the LLM-written ones, and reranking took about a quarter of a second per note.

6.4.5 Table Retrieval

Doses and drug choices are mostly found in tables, which are stored separately from the prose. They are retrieved in a second step, once the excerpts have been selected, and only for patients discharged home, since the department does not prescribe therapy to admitted patients.

The search terms are the drugs the NER model finds in the note, excluding negated ones, together with the eight drugs most often named across the selected excerpts. A multiplication factor of 20% is added for terms from the note, as they concern drugs the patient is actually taking or has received. Only tables from the guidelines of the selected excerpts are considered.

A term reaches every row of the tables that contains it, as well as all rows of a table whose caption or header names it. Each row is then scored by the cross-encoder against the term followed by the word *dose*. Since dosage tables rarely mention the diagnosis, including it would push their scores close to zero. A broader query, made of the diagnosis, a few anchor terms such as *dose*, *criteria* and *score*, and the age group and sex of the patient, is also matched by embedding, and the rows of its five closest tables are added to the candidates. When none of the guidelines selected in the text retrieval phase holds a qualifying table, the drug terms are dropped and only the broad query is used on the whole table store.

Each table takes the score of its best row, increased by 20% if the term came from the note, and up to five tables scoring at least 0.05 are passed to the generator. If a place remains free, it is given to the table closest to the broad query, regardless of its cross-encoder score. On the 21 notes of patients going home, the step returned a median of three tables per note with both writers, in a median time of 0.45 seconds.

Parameter	**Value**
Retrieval and selection	
Chunks retrieved per queryy	8
Final excerpts selected	12
Boost of each query's best chunk	10%
Share defining a dominant guideline; penalty for other guidelines	35%; 15%
Word overlap marking a near-duplicate	80%
Minimum score to fill a floor; maximum share of floors	0.05; 50%
Tables	
Tables selected; minimum score	5
Minimum cross-encoder score for selection	0.05
Maximum drug terms taken from the excerpts	8
Weight of terms from the note	1.2
Tables reached by the broad query	5

Table 6.5. Main parameters of query construction and retrieval.

6.5 Generation

The final stage turns the note and the selected evidence into the discharge recommendations. The excerpts and tables chosen during retrieval are assembled together with the note into a prompt, which a large language model then answers. The prompt defines what the document contains and in which order, how each item is phrased, which information it may rest on,

and how each item is traced back to its source. The complete prompts are reported in Appendix B.1.

6.5.1 Prompt Assembly

The route determined from the prognosis (Section 6.3.5) selects one of two templates, one for patients discharged home and one for admitted patients. Both templates open in the same way. A role first tells the model that it is an emergency physician, writing either for the patient and the general practitioner or for the team taking over the case. The clinical note follows in its translated form. Then come the selected excerpts, in order of final score, each with a label (`[E1]`, `[E2]`, ...) and the guideline and page range it was taken from. For patients going home, the selected tables follow in the same way, labelled `[T1]`, `[T2]`, ... and rendered in Markdown with their caption and header.

The task and the rules come last, after the evidence. In a prompt several thousand tokens long, this places the instructions immediately before the point where the model starts writing, rather than in the middle of the context, where language models have been shown to make poorer use of the information they are given[73]. On the 21 notes of patients going home, the assembled prompts ranged from about 4,500 to 10,000 tokens.

6.5.2 Home Discharge Template

For a patient going home, the model is asked for exactly two sections in plain text: the recommendations for the patient, and a list of references for the reviewing physician. Before describing them, the template defines what the recommendations must be based on. They may draw on the excerpts and tables concerning the patient's diagnosis or a condition treated during the visit, and on what the note itself records, such as a therapy started in the department or the advice of a consultant. The prompt also states openly that the excerpts were retrieved automatically and that some may concern other conditions, in which case they must be ignored.

The first section is a numbered list of instructions for the patient, organised in groups that always appear in the same order. Medications to start come first, including any therapy begun in the department that continues at home, followed by medications to stop or change, and by other treatments and self-care, such as looking after a cast or a dressing. Lifestyle changes come next, then follow-up visits, tests and referrals, then the symptoms that should bring the patient back to the department, and finally the referral back to the general practitioner. Groups that do not apply to the patient are skipped, except for the last two, which are always present. Each group comes with a fixed phrasing, such as *Take [drug] [dose] [route] [frequency]* for medications or *A prescription is issued for [specialist type] evaluation for [clinical purpose]* for referrals. This way, the output can be inserted into the letter without editing.

The rules on medications are the most detailed part of the template, since this is where an error would cause the most harm. Home medications that the patient keeps taking unchanged are never mentioned, as the patient continues them without being told. A home medication appears only when the note or an applicable excerpt calls for stopping or changing it. Every medication mentioned must carry its dose, route and frequency. These information must be taken from the excerpts or tables when they state them, otherwise copied from the note, and replaced by *as directed by the specialist* when neither source provides them, so that the

model never has to supply a dose from its own knowledge. Doses taken from a table must be copied exactly, never computed or interpolated, and the columns listing contraindications or criteria must be checked against the note before a drug is recommended. This means that if a drug is reported as a range in the guidelines, it will be replicated as such in the clinical note, and final judgement for the correct value is deferred to the reviewing physician. This design choice, which further limits the reasoning abilities of the model, was suggested by the partner ED. Since intravenous therapy does not continue at home, the oral form of a drug is given whenever the evidence states it.

Each item is limited to one or two sentences, without explanations, citations or formatting, and without speculation about how the therapy will evolve, for instance its duration or a later change of drug. When the case requires a decision that the excerpts do not cover, such as further investigation of the problem that brought the patient to the department, the model must not make that decision itself. It must instead issue a referral to a specialist, and only for the problem of this visit. Together with the general instruction to rely only on the note and on the applicable excerpts, these rules aim to keep the model from filling gaps in the evidence with its own knowledge, a tendency that explicit instructions to stay faithful to the given context have been shown to reduce[99].

The second section is addressed to the reviewing physician and contains one line for each recommendation, stating its number, a one-sentence clinical justification for that choice, and its sources. The goal of the clinical justification is to provide the physician with a quick reference in case the decision appears dubious. Sources can take one of three forms. An item supported by the evidence cites the guideline and page range of its excerpts or tables rather than their labels, so that the physician can look them up. An item resting on the note alone, such as the continuation of a therapy started in the department, cites the clinical note. Finally, an item supported by no source must say so explicitly. This last form makes unsupported items visible to the reviewer, instead of leaving the model free to attach them to a nearby but unrelated citation.

The task closes with a worked example for a made up patient, with renal colic, which is presented as an example of the format only and also illustrates the rule on unchanged home medications. A final block restates the core rules and asks the model to check, before answering, that no unchanged home medication appears and that every reference line takes one of the three forms. The prompt design purposefully instructs the model to check multiple times that the statements are backed in the provided knowledge rather than on its own parametric memory, to ground the decisions as much as possible.

6.5.3 Admission Template

When the patient is admitted, ongoing therapy and follow-up become the responsibility of the admitting team, and the department's existing templates already cover most of the document. The admission template is therefore much simpler, as it almost resembles a fixed template, and receives no tables. It asks for a short handover note stating the ward the patient is admitted to and the main planned interventions, and confirming that the patient and/or the family have been informed of the diagnosis, of the expected course according to the excerpts and of the planned treatment. For a surgical patient, the note also states that the patient has been prepared for the operation, if the note states so, and in every case it closes by handing the patient over to the admitting team. Prescriptions, follow-up visits and lifestyle advice are explicitly forbidden. The second section lists the sources of the items that

draw on guideline content, in the same format used for patients going home.

6.5.4 Generation Back Ends

The same prompt can be sent either to a model served through an OpenRouter[172] API or to a model running local without any change to the earlier stages. Various models were tested across different configurations of the system, and their results are discussed in Chapter 7.

Throught the API, the prompt is sent as a single message, with the model's reasoning left at its default effort. Current models can reason internally before answering, and these hidden reasoning tokens count towards the output limit: one model used about 7,700 of them at its default effort. The limit is therefore set to 16,000 tokens, far beyond the length of the letter itself, and a reply cut off by the limit raises an error rather than being returned incomplete.

Locally, models run either through llama.cpp[189], as quantised checkpoints loaded entirely on the GPU, or through the Transformers library[50] without quantisation. Local models bring a constraint that API models do not have, namely the size of the context window, which must hold both the prompt and the letter.

Some models may impose an input character or token limit, which could silently cut off important instructions at the end of the prompt, namely the task and rules or even the end of the letter. Therefore, before generation, the prompt can be measured with the model's own tokenizer, and the lowest-ranked excerpts and tables removed until prompt and letter fit inside the context window, always keeping at least one excerpt. The option is off by default, since no tested model presented such limitations.

6.5.5 Output Translation

The recommendations are written in English, the language of the evidence they rest on, while the patient and the physician needs them in Italian. They can be translated by the same model that wrote them, either in the same call, by appending translation instructions to the prompt, or in a second call on the finished output, at temperature zero. The translation must be faithful and complete, keeping the items in their order and with their numbering, and must give drug names in their Italian form. Numbers must keep their exact value but follow the Italian notation, so that *2.5 mg* becomes *2,5 mg*, and the citations are left exactly as written.

6.6 User Interface

Since physicians cannot be expected to edit a configuration file or run scripts from terminal, the system also offers a web application, implemented in `app.py`. The application provides a simple way to run all the stages of the system through a simple interface, written in Italian, while the pipeline behind it works exactly as described in the previous sections.

The application can be used in two modes. By default, it runs the whole pipeline and displays the recommendations in Italian, ready to be reviewed and inserted into the discharge letter. Alternatively, it can stop once the prompt has been assembled, leaving the physician to copy it into an external chat application of their choice. This second mode suits a physician who prefers to generate the letter with a chat model they already have access to, and eliminates the cost of the API call for generation, at the price of an additional manual operation of copy/paste.

6.6.1 Model Loading and Input

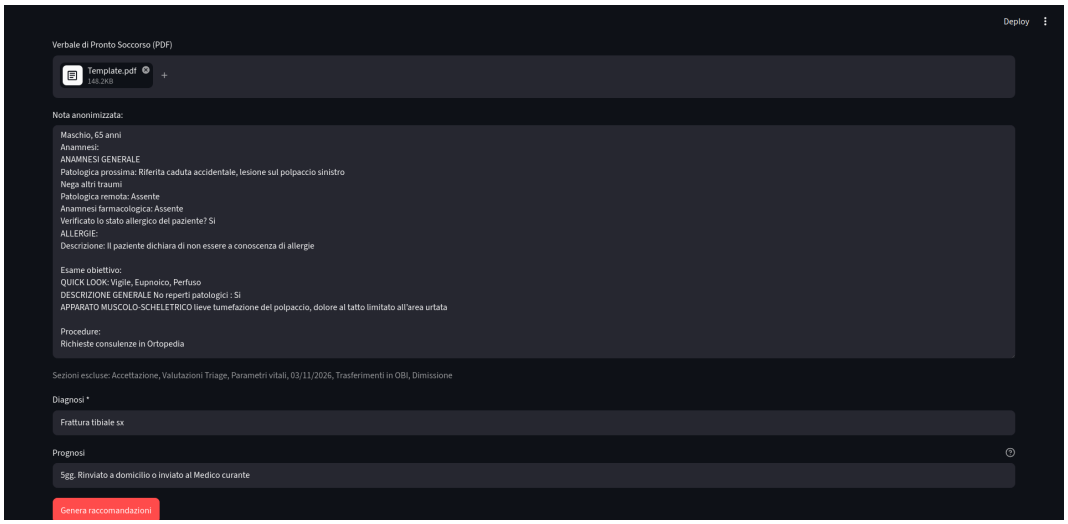


Figure 6.2. The interface after uploading a report: the anonymised note in an editable box, the titles of the excluded sections below it, and the diagnosis and prognosis fields. The report shown is the template of Appendix C.1, filled in for display only.

The application can be opened from a desktop shortcut or with a simple single terminal command, which starts the Streamlit server in a terminal window and opens the page in the default browser.

Models and indexes are loaded at page startup time and kept in Streamlit's resource cache, which holds a single copy of each in memory for the lifetime of the server. The physician can upload a report and review it while the models are still loading, and a generation requested in the meantime simply waits for loading to complete. On the development machine the first opening of the page takes about 15 seconds, after which every interaction finds the models already in memory.

The main input is the PDF of the report, uploaded through a file selector. As soon as the file is received, it is read and anonymised (Sections 6.3.1 and 6.3.2), and, if present, the physician's own advice is separated from the note (Section 6.3.3). A file that is not a report is refused with a message explaining why, since in that case anonymisation could not be carried out reliably.

Optionally, the note can also be pasted as plain text following the same template. However, pasted text is not anonymised, and the interface reminds the physician to remove any identifier before pasting it.

The result of this step is shown to the physician before anything leaves the machine, as illustrated in Figure 6.2. The anonymised note appears in an editable text box, together with the titles of the sections that were left out of it. The physician can thus check the note and remove anything the anonymisation missed, which adds the option of a human check to the automatic procedure. The diagnosis and the prognosis appear in two separate fields, which can be edited as well. The diagnosis is mandatory, since every rule-based query is built around it (Section 6.4.1), whereas the prognosis may be left empty, in which case the

patient is assumed to be discharged home. All three fields are tied to the uploaded file, so that a new report never inherits the edits made to the previous one.

6.6.2 Options and Processing

A handful of control options above the input select how the pipeline runs. They expose the interchangeable components described in Section 6.1.2, so that a configuration can be chosen without manually editing the `config.py` script, and are summarised in Table 6.6, while Figure 6.3 shows them as they appear at the top of the page.



Figure 6.3. The upper section of the interface in the full mode, with the option controls above the file selector and the index maintenance panel at the bottom. The excerpt selection option is hidden, since it only applies to the prompt-only mode. The interface is in Italian, the language of the department.

Two of these options require some explanation. The fast selection was introduced for machines without a GPU, where the cross-encoder is the slowest component of retrieval and might take several minutes to complete. In this setting, each query retrieves 15 chunks and the final excerpts are picked in rounds over the rankings of the queries, so that every query contributes its best chunk before any contributes its second. This offers the same guarantee as the floors of Section 6.4.4, namely that no aspect of the case is crowded out, without requiring any reranking. The full output always uses the accurate selection, since that is the configuration evaluated in Chapter 7. As for translation, the generation model is the default engine because it translated the notes more faithfully than DeepL in the comparison of Section 7.2.

Processing starts when the physician presses the button below the input. A status box lists each stage as it runs, from the translation of the note to query construction, retrieval, table retrieval and finally generation, and collapses once the run is complete. Any error is reported as a message, and no partial result is ever displayed.

In the prompt-only mode, the prompt is meant to be pasted into a chat application, whose input may be limited in length. It is therefore fitted to 32,000 characters, using the same procedure described in Section 6.5.4, and a message reports how many excerpts

and tables were left out. In the full mode, the prompt is sent through OpenRouter to the generation model set in the configuration, and the English recommendations are then translated into Italian. If the standard option is selected for Prompt, the system makes two separate calls, one for generation and one for translation. Otherwise, the translation instructions are appended to the end of the generation prompt so that everything is handled in a single call. This lowers the costs and the running time, but the additional instruction might confound smaller models.

Option	**Values**	**Mode**	**Effect**
Prompt only	Off (default), on	Both	Stops at the prompt instead of generating the recommendations
Query construction	Rule-based (default), LLM-written	Both	Selects the query writer, falling back on the rule-based one if the LLM call fails
Excerpt selection	Accurate (default), fast	Prompt only	Selects whether candidates are reranked by the cross-encoder
Translation	LLM (default), DeepL	Full output	Selects the engine translating the note into English and the recommendations into Italian
Prompt	Standard (default), with Italian translation	Both	Appends the output translation instructions to the prompt

Table 6.6. Options of the user interface and the mode in which each of them is available.

6.6.3 Output

In the prompt-only mode, the output is the prompt itself, shown in a box with a button on the top right that copies it to the clipboard. In the full mode, only the Italian text is displayed, with both sections, since the document is inserted into an Italian letter, and the references are translated as well, for a reviewer who may not be fluent in English. The text is presented as a draft that must be reviewed before use, as shown in Figure 6.4.

The most important feature of the output concerns the references. The traceability principle of Section 4.1.1 requires that every recommendation can be checked against its source faster than it could be written from scratch. Therefore, each citation in the reference section is turned into a link. When the cited guideline is present in the corpus folder, the citation is replaced by a superscript made of the initials of its name, so that *AtrialFibrillation* becomes ^{AF}. Clicking the superscript opens the PDF in a new browser tab, directly at the cited page. The recommendations themselves contain no citations, so they can still be copied into the letter as they are. Should the system not find the cited file inside the local Guidelines/ folder, it simply cites the references in full as (FileName, p. $n - m$), so that a physiscian still has the option to verify the source.

A switch above the text turns the output view into an editable text box, in which the citations appear in full, to allow the reviewing physician to make adjustments without leaving the page. The edited text replaces the generated one, and the links are rebuilt from it when the switch is turned off.

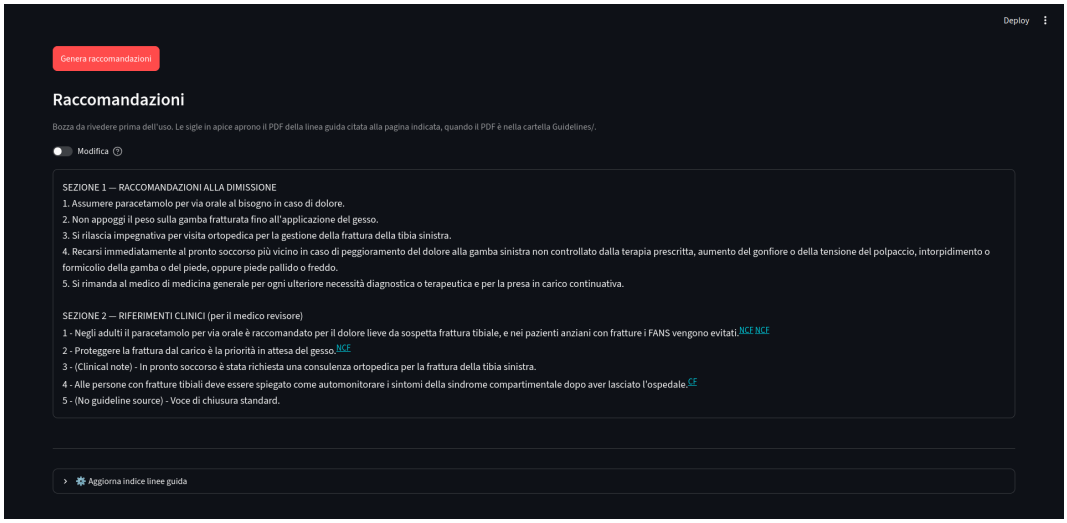


Figure 6.4. Recommendations generated for the note of Figure 6.2. The first section can be copied into the letter as it is, while each line of the second ends with links to the cited guidelines, labelled with their initials (NCF for *Non-Complex Fractures*, CF for *Complex Fractures*). Two links on the same line point to two excerpts, which come from different pages when the initials are the same.

6.6.4 Index Maintenance

A collapsible panel at the bottom of the page allows the corpus to be updated without the command line (Figure 6.4). It shows the current number of guidelines in the corpus folder and accepts new PDFs, which are saved there, while a file with the same name as an existing one is reported rather than overwritten. After confirmation, the index is rebuilt from scratch by the ingestion pipeline of Section 6.2. On the full corpus the rebuild takes several minutes, during which the application cannot be used, after which only the retrieval indexes and stores are reloaded while the models stay in memory.

Chapter 7

Evaluation

The evaluation of the system is performed at two levels. At the level of its components, it reports the comparisons through which the stages of retrieval were chosen. At the level of the whole pipeline, it assesses the recommendations themselves, which were scored with the aid of an emergency physician. This second level answers the two central research questions of the thesis (Section 1.3.1): how the recommendations compare with those written by physicians (RQ2), and how their quality changes when the frontier model is replaced by models that a hospital could host on its own hardware (RQ3). The evaluation of the guideline corpus was already reported in Section 5.7 and is not repeated here.

7.1 Evaluation Design

All experiments in this chapter rest on the same clinical notes as data and follow the same principles of comparison. This section describes both, together with the configurations and the protocol of the end-to-end evaluation.

7.1.1 The Clinical Notes

The evaluation had to work around the limitations that surround the handling of medical data in Italian hospitals. Given its extremely sensitive nature, real notes are not allowed to leave the hospital network without undergoing the approval process of an ethics committee, which often takes more than 12 months[190]. For the scope of this thesis, a team of emergency physicians of the partner department was therefore tasked with producing 25 synthetic clinical notes, with clear instructions to follow the usual procedures and style as closely as possible, rather than to make the cases clearer or more appealing. The physicians imitated real cases down to their telegraphic style, abbreviations and typing errors, and printed them with the layout of the department's information system. As a result, the system reads them exactly as it would read a real report and the testing setting can be considered realistic (Section 6.3.1). A template of a clinical note is included in Appendix C.1.

The notes reflect the usual case mix of an emergency department, from cardiovascular presentations such as atrial fibrillation and syncope to respiratory infections, a transient ischaemic attack, gastrointestinal complaints and minor trauma. Four of the patients are admitted to a ward, while the other 21 are discharged home. The corpus, for motivations

already described in Section 5.7.3, does not cover all of these cases equally well. In fact, it was judged as able to cover the diagnosis of 21 notes fully or mostly, but the remaining four only in part or not at all. Since no retrieval or generation strategy can recover guidance that the corpus does not contain, results are reported separately for these four notes wherever the difference matters. This also gives a perspective to how well the model can adapt to cases where official guidelines are missing.

Each note ends with the advice written by the physician who discharged the patient. Even in this section, physicians did not adulterate it to make it more appealing, maintaining instead their usual styling, which comes with the problems outlined in Chapter 1. Nonetheless, the system never sees this advice at any stage: the parser separates it from the note before any other stage runs, so that the output is not influenced by it. The advice serves as a reference rather than as a gold standard. Discharge advice written in an emergency department is often brief, and adding the guidance that physicians omit under time pressure is one of the purposes of the system, so an output that departs from the advice is not necessarily worse.

Development was carried out on a separate set of notes, produced by an LLM under the supervision of a physician, who did not participate in the writing process. While their styling did not present the limitations of real clinical notes, their realistic content allowed to tune the system without overfitting on the final test. Nonetheless, the model comparisons discussed in Section 7.3 and Section 7.4, which determined the final configurations and model choices, were performed on the 25 realistic notes.

7.1.2 Deployment Configurations

The end-to-end evaluation compares four configurations of the system, each designed with the objective of simulating a realistic and distinct deployment scenario (Table 7.1). All of them use the same embedding model, Nemotron-3-Embed-1B, which is small enough for any of the hardware considered. What changes is the cross-encoder and the generation model, which were chosen based on the memory (specifically, the VRAM) available in each theoretical scenario. Since the available hardware was limited, as discussed in Section 6.1.3, the use of bigger local models is simulated through API access.

Config.	Scenario	Cross-encoder	Generation model
1	Frontier models through API	Qwen3-Reranker-8B	Claude Opus 5.5 and Claude Sonnet 5.5
2	Multi-GPU server, 160 GB	Qwen3-Reranker-8B	Nemotron 3 Super (120B), 8-bit
3	Single workstation GPU, 24 GB	MedCPT	Gemma 4 31B, 4-bit
4	Development laptop, 6 GB, generation through API	MedCPT	Claude Opus 5.5

Table 7.1. Configurations of the end-to-end evaluation. Nemotron-3-Embed-1B used as the embedding model.

Configuration 1 is the reference, as it uses frontier models through a commercial API and therefore represents the upper bound of the system. It includes two models of the same family, Claude Opus 5.5 and the cheaper Claude Sonnet 5.5, so that the same evidence can

show whether the larger model is worth its price. Configurations 2 and 3, instead, stand for a hospital hosting open-weight models on its own hardware. To fit that hardware, both rely on quantisation, which stores each weight of a model with fewer bits so that the model needs less memory.
Configuration 2 assumes a server with several GPUs and about 160 GB of memory. Quantised to 8 bits, Nemotron 3 Super, a model with 120 billion parameters, takes 128.5 GB[191], and Qwen3-Reranker-8B another 16.4 GB[192]. Together with the embedding and NER models, the weights of the system take about 148 GB.
Configuration 3 assumes a single large workstation GPU with 24 GB of memory, an expensive component but one that a single PC can realistically host. There, Gemma 4 31B takes 17.3 GB in a 4-bit quantisation[193]. The embedder, the NER model and MedCPT add about 3 GB, so the whole system comfortably fits within the 24 GB available.
Configuration 4, finally, is the setup on which the system was developed: every local component runs on a 6 GB laptop GPU, where the three small models take about 3 GB, and only generation is delegated to an API. The more powerful generation model was selected so as not to repeat the comparison between the two Claude models.

Every generation model, as well as the cross-encoder Qwen3-Reranker-8B, was accessed through OpenRouter[172] (Section 6.1.3). Models intended to simulate a local environment are all open-weights and freely downloadable and deployable. For the open-weight models this amounts to a simulation of local deployment, which required one precaution. The models were restricted to providers serving the precision of the hardware they stand for: 4-bit for Gemma 4 31B, 8-bit for Nemotron 3 Super. This allows to accurately simulate the output quality, though the time required to produce the recommendation is not a realistic representation of how the models would run in a locally deployed environment.

7.1.3 Protocol and Scoring

Each generation model was run in two modes: with rule-based queries, or with the generation model itself (Section 6.4). With the rule-based queries, all models sharing a cross-encoder receive exactly the same evidence, so any difference between them is due to generation alone. With their own queries, each model is instead evaluated on both its ability to generate good queries and to generate good recommendations. Using the rule-based system as a control group, this allowed to study the decay in query building qualities among models. Configuration 4 reused the queries written by Opus in configuration 1, so that the two configurations differ only in the cross-encoder. The evaluation thus comprises ten runs over the 25 notes, for a total of 250 outputs.

The protocol also kept every difference between runs attributable to the components being compared. Retrieval was run once for every combination of cross-encoder and query writer, and its results were stored, so that all models sharing a combination read byte-identical prompts. For the same reason, all configurations received the same DeepL translation of each note, and the quality of the translation was evaluated in a separate step. Each configuration generated a single output per note.

The outputs were scored by hand rather than with automatic metrics. This is because traditional metrics used in the Natural Language Processing field, such as BLEU[194] and ROUGE[195], measure how many words or word sequences an output shares with a reference text. In a setting where the system is designed to explicitly distance itself from the form of

the original text, they would penalise precisely what it is designed to do. In fact, a correct recommendation absent from the physician's advice would lower the overlap, as would a correct recommendation worded differently. Therefore, outputs were scored in person by the author with the aid of an emergency physician, and compared to the original notes' indications. For each note, a text file presented the original advice on top, followed by the outputs randomly shuffled to avoid bias. Each produced recommendation was labelled with a letter from A to J. The key linking letters to configurations was kept in a separate file, and opened only after scoring.

Each output was scored on the four criteria presented in Table 7.2. It is important to note that, in every configuration, the model improved significantly on instructions on when to return to the emergency department and the referral to the general practitioner, which were almost always missing from the reference notes. Consequently, a decision was made to exclude them from the *Useful Additions* criterion, to avoid polluting the results. Likewise, general readability improved greatly in all configurations, and is not scored. In both cases, it is evident that the implementation of the system greatly enhanced the clarity and readability of every note. The following evaluation is therefore focused exclusively on content and actionability of the recommendations.

Criterion	**What is assessed**
Coverage	How many clinical items of the physician's advice does the output contain, with partial matches counting half.
Errors	Number of clinically wrong statements, and how many of them are harmful or potentially harmful.
Useful additions	Whether the output contains correct and useful recommendations absent from the physician's advice, normally lifestyle recommendations.
Overall score	Quality of the output as a discharge document for this patient, from 1 to 5.

Table 7.2. Criteria used to score each output.

7.1.4 Comparing Settings

Two kinds of judgement are used in this chapter, depending on the stage being evaluated. The final recommendations, which are what the system exists for, were evaluated as described above. Retrieval was instead judged by a language model. The decision was taken for practical reasons, since comparing retrieval settings means deciding, for every excerpt that each setting selects for each note, whether it is useful for that patient. This amounts to thousands of decisions, far more than a physician could provide. Using strong language models as judges is an established practice where human annotation is impractical[45], and the judge used here is described in Section 7.3.1.

Whatever the judge, every comparison followed two principles. Components were compared end to end: each candidate was run through the whole stage it belongs to. An embedding model, for example, does not only rank chunks, since in the rule-based mode it also participates in the choice of the guidelines that the queries are confined to (Section 6.4.1). A test on a fixed set of queries would hide this second role. Comparisons were also paired by note. The notes differ widely in difficulty, so two settings were always compared on the same notes, counting the notes on which each did better and the ties.

These counts were tested with the two-sided sign test[196], which ignores the magnitude of the ties and asks how likely the observed split would be if neither setting were truly better. The test makes no assumption about how the differences are distributed, which suits a sample as small as this one. For the scores of the end-to-end evaluation, a further test was added.

7.2 Translation

Translation is the first stage of the pipeline, and every later stage depends on it. The NER model, the negation rules and the cross-encoder only read English, and the fields of the note are located through their English labels. Section 6.3.4 left open which engine should perform it. Two options were considered: a dedicated translation service such as DeepL[171] or a Large Language Model. This section compares the two options on the notes of the evaluation.

Five notes were drawn at random and translated by DeepL and by the four language models of the end-to-end evaluation (Section 7.1.2). For all the engines, the section labels and the demographic line were handled in code. The language models received the glossary in their prompt, while it was uploaded to DeepL through its pre-built *Glossary* functionality. The two open-weight models were served at the precision of the configuration they belong to.

A first check verified that the fields parsed from each translation, including the age of the patient, matched those parsed from the DeepL translation, while the second counted the glossary terms left in Italian. Then, each translation was compared with its Italian source to find the errors that change the clinical meaning.

Every engine passed the structural check: the same fields and the same age were parsed from all translations, so the safeguards described in Section 6.3.4 held regardless of the engine. However, the clinical meaning check was not as uniform.

DeepL made several errors of the kind that reach the recommendations. It referred to a male patient as *she* and to a female patient as *he*, although the sex of the patient enters the retrieval queries and could potentially condition the choice of drugs. It translated *PER OM VERBALE*, which marks a verbal medical order, as *for oral administration* on an intravenous line, and turned vials (*f*) into tablets for intravenous boluses, so that the form of a drug no longer matched its route. It also changed a dosing schedule at 8 and 20 into an interval between 8 am and 8 pm.

Claude Opus 5.5 made no error affecting meaning, its only flaw being that it carried the physician's typing errors into English, probably stemming from strict instructions to translate literally. Claude Sonnet 5.5 applied the glossary correctly in four notes, but ignored it on the longest drug list and left nine brand names in Italian. Gemma 4 produced faithful translations with a few overly literal renderings. Nemotron 3 Super was the weakest engine: it left many Italian abbreviations untranslated, wrote ungrammatical English, and once translated the abbreviation for thorax as *tongue*.

Each option thus trades faithfulness against speed and privacy. DeepL is the fastest engine by an order of magnitude, but on these notes it made more errors affecting meaning than any other engine, except Nemotron 3 Super. It is also an external service, so in a configuration meant to keep data within the hospital it would still send every note outside

the network. A frontier model translates far more faithfully, at about ten seconds and a few cents per note. In configurations 1 and 4 it also adds no new exposure, since the note is sent to the same API for generation anyway. Among the open-weight models, only Gemma 4 translated faithfully. While it took a significant amount of time to craft the notes (upwards of 100 sec/note) in both models, this reflects the providers' hardware rather than a local GPU, so conclusions about their deployability cannot be made with certainty, though they would likely be much faster.

These results suggest translating with the generation model is always preferrable compared to DeepL.

The end-to-end evaluation, however, used the DeepL translations for every configuration (Section 7.1.3), so that differences in the outputs could be attributed to retrieval and generation alone.

To check whether the usage of LLM translation in the generation evaluation would have changed the results, a test was performed on the configuration that scored best in Section 7.4, Claude Opus 5.5 with its own queries and Qwen3-Reranker-8B. This configuration was run again on notes translated by Opus itself, and its outputs were blind scored following the same protocol. The scores were essentially unchanged, with 20 notes sharing the same score, and only three scoring better with the Opus translation: despite the visible errors in the DeepL translation, they did not affect the recommendations.

7.3 Retrieval

This section evaluates the components of retrieval in the order in which a note passes through them: the embedding model, the depth of the search, the cross-encoder and the retrieval of tables. All comparisons rely on the judge described first.

7.3.1 The Retrieval Judge

No reference list of relevant passages exists for these notes, and building one would have meant reading and annotating the whole corpus for each of them. Every excerpt that a setting selected for a note was therefore judged by a language model, Claude Sonnet 5.5, as anticipated in Section 7.1.4. The judge received the translated note, the outcome of the visit and the physician's advice, together with a batch of excerpts identified only by an anonymous code, so it never knew which configuration had retrieved them. Each pair of note and excerpt was judged only once.

An excerpt counts as useful when a physician could ground one of this patient's recommendations in it, such as a treatment or dose for the diagnosis, the effect of a comorbidity on its management, follow-up or warning signs. Passages about another condition or population, tables grading the evidence and passages too generic to support any recommendation are not useful, and neither is a passage that merely shares a word with the note.

The binary *Useful/Not Useful* choice was preferred over a scoring system as it was deemed more objective and less dependent on model tendencies. Previous experiments performed with different agents and a 1–10 scoring system had proved inconsistent among models and even within the same model on different runs.

To check how stable the verdicts were, random samples of pairs were judged a second time, in batches of different composition. On a sample of 110 pairs, agreement was 86.4%, with a

κ of 0.68. All the verdicts that changed concerned excerpts the judge itself had described as only partly applicable, and they were spread evenly across the settings being compared. The judge's uncertainty therefore adds some noise to the counts, but favours no setting.

7.3.2 Embedding Model

The two candidates that fit the constraints of the system, ModernPubMedBERT[180] and Nemotron-3-Embed-1B[181], were compared end to end. Each model was used to build its own index of the corpus and to run the whole retrieval stage on the 25 notes, with both kinds of queries. Building the index took approximately 26 minutes on Nemotron, against 13 for ModernPubMedBERT. This was expected due to the difference in model sizes, but did not factor into the final comparison due to the irrelevance of time in an offline operation.

Table 7.3 reports how many of the selected excerpts were judged useful, both for the 12 excerpts passed to the generator and, as a check of robustness, for a selection of 15.

Queries, excerpts	**Nemotron**	**ModernPubMedBERT**	**Notes better/worse**	p
Rule-based, 12	112	63	2 / 17	0.001
Rule-based, 15	130	78	3 / 15	0.008
Model-written, 12	156	135	7 / 14	0.19
Model-written, 15	194	156	4 / 18	0.004

Table 7.3. Useful excerpts over the 25 notes with each embedding model, out of 300 with 12 excerpts per note and 375 with 15; p from a two-sided sign test; bge-reranker-large[186] as the cross-encoder.

Nemotron selected more useful excerpts in every setting, significantly so in three of the four. The gap is widest with the rule-based queries, where it selected 77% more useful excerpts, and the reason lies in a second role of the embedder in that mode. Before searching, the rule-based mode uses the embedder to find the guidelines closest to the name of the diagnosis, and confines the queries to them (Section 6.4.1). With the model-written queries, which search the whole corpus, the gap narrows.

Nemotron was therefore adopted, at the price of four times the memory (2.2 against 0.57 GB), which the development laptop could still accommodate alongside the other local models.

7.3.3 Retrieval Depth

The number of chunks retrieved for each query trades breadth against noise, while factoring in computational cost. A deeper search gives the cross-encoder more candidates to choose from, but also more chances to rank a misleading chunk highly, and the time spent reranking grows with the number of candidates. Depths of 8, 12, 16 and 24 chunks per query were compared with Nemotron.

Going deeper than eight never helped. With the rule-based queries, a depth of 12 gained four useful excerpts out of 375 in the best case, a difference within the noise of the judge, while 16 and 24 lost useful excerpts and took two to three times as long to rerank. Moreover, with the model-written queries every increase in depth lost useful excerpts. The depth was therefore set to eight.

7.3.4 Cross-Encoder

Four cross-encoders small enough for the development laptop were compared on the same candidates: bge-reranker-large[186], MedCPT[131], llama-nemotron-rerank-1b[188] and Qwen3-Reranker-0.6B[187]. The candidates were fixed, so that a cross-encoder could only change which of them were selected, among both the excerpts and, for the 21 patients going home, the tables selected after them, whose rows the cross-encoder also scores.

	bge	**MedCPT**	**Nemotron**	**Qwen3–0.6B**
Useful excerpts				
Rule-based, 12	112	118	122	118
Rule-based, 15	130	140	139	137
Model-written, 12	156	173	165	165
Model-written, 15	193	204	198	203
Useful tables (3 / 5 per note)				
Rule-based	8 / 10	11 / 13	4 / 4	13 / 18
Model-written	12 / 16	14 / 17	3 / 4	17 / 22
Cost on the development laptop				
Seconds per note	0.79	0.24	2.31	2.43
Weights (GB)	1.04	0.22	2.30	1.11

Table 7.4. Cross-encoders compared on the same candidates. Tables were counted over the 21 patients going home.

On the excerpts, the four models performed similarly (Table 7.4). MedCPT selected the most useful excerpts in three of the four settings, but no model differed from bge-reranker-large with a p-value below 0.36, so the differences are not statistically significant and lie within the noise of the judge. On the tables, Qwen3-Reranker-0.6B stood apart, selecting more useful tables than the others while retaining a similar precision, while the Nemotron reranker found almost none.

The two leading models differ sharply in cost, as Qwen3-Reranker-0.6B was ten times slower than MedCPT. Therefore, MedCPT was ultimately selected as the local cross-encoder model for the generation evaluation.

The configurations not bound to local hardware use the larger Qwen3-Reranker-8B through an API (Section 7.1.2), which could not run on the development laptop and was therefore left out of this comparison. On the notes of the end-to-end evaluation, it agreed with MedCPT on 7.2 to 7.7 of the 12 excerpts selected per note, depending on the queries, so that roughly a third of the evidence differed between the two. Whether this difference matters for the recommendations is examined in Section 7.4.

7.4 Generation Results

This section reports the scores of the ten runs of the end-to-end evaluation, carried out as described in Section 7.1.3. An interpretation of the results relative to the research questions is discussed in Section 7.6.

Cfg.	Model	Queries	Coverage	Wrong	Harmful	Additions	Score
1	Opus	rule-based	66%	1	0	11	3.96
1	Opus	own	84%	2	0	14	4.26
1	Sonnet	rule-based	73%	2	2	12	3.94
1	Sonnet	own	73%	3	2	11	3.90
2	Nemotron 3	rule-based	62%	3	2	14	3.86
2	Nemotron 3	own	59%	2	0	14	3.88
3	Gemma 4 31B	rule-based	58%	3	2	8	3.68
3	Gemma 4 31B	own	59%	1	1	8	3.76
4	Opus	rule-based	55%	1	0	11	3.92
4	Opus	own	78%	1	0	12	4.22

Table 7.5. Results of the ten runs over the 25 notes. Coverage is the share of the 37 items of the physicians' advice that the outputs contain; wrong and harmful statements are totals over the 25 outputs; additions is the number of outputs with useful additions compared to the original recommendations; score is the mean overall score (subjective), from 1 to 5.

7.4.1 Overall Results

Mean score, an evaluation of overall quality of the output on a scale from 1 to 5, where 5 represents "*The output is a significant improvement over the original and is ready to be presented as is*" and 1 "*The output needs heavy editing and may contain harmful information if used in this state*", range from 3.68 to 4.26 out of 5 (Table 7.5). Figure 7.1 breaks the overall score and the coverage of the advice down by note and run. The four admitted patients received the top score from every run, since the handover template leaves little room for error, and two of the three discharges whose advice contained no specific item scored 4.9 on average. These notes can hardly separate the configurations, so the differences are concentrated on the patients going home with specific advice. Readability, which was judged globally, was found to be better than that of the original advice in every output.

The model was not penalized for prescribing correct specialist visits that the physician would not normally issue. In fact, withholding non-essential, yet useful, visits is a response to limited health-system resources rather a clinical criterion, as it is a practice adopted to avoid further straining the SSN (National Health System). Since the guidelines do not separate necessary from elective visits, a judgment that lies upon the physician and varies between physicians, the model is not asked to make it. Instead, it prescribes inclusively, as the act of eliminating a prescription in excess is much less time consuming than that of writing a new one. Missed visit prescriptions, however, still count against the output's total coverage.

The overall score closely followed the coverage of the physician's advice. Over the 180 outputs for notes with at least one advice item, the two have a Spearman correlation of 0.70[197], a measure of how consistently two quantities rise together, which ranges from -1 to 1. The overall judgement thus rewarded, above all, the presence of what the physician considered necessary.

The coverage of the corpus weighs just as heavily. Averaged over all runs, the 21 notes whose diagnosis the corpus covers scored 4.06, while the four it does not cover scored 3.29, and no configuration exceeded 3.62 on them (Appendix C.4). Two of these four, an adult gastroenteritis and a callus on a toe, averaged 2.6 and 2.7 across the ten runs. A case of

acute bronchitis (Note 3) was difficult for a different reason, as every output scored 2.5 or less. The physician had prescribed an inhaled therapy and a mucolytic that the guideline in the corpus does not recommend for this condition, so the system followed the guideline, while the evaluators preferred the established local practice.

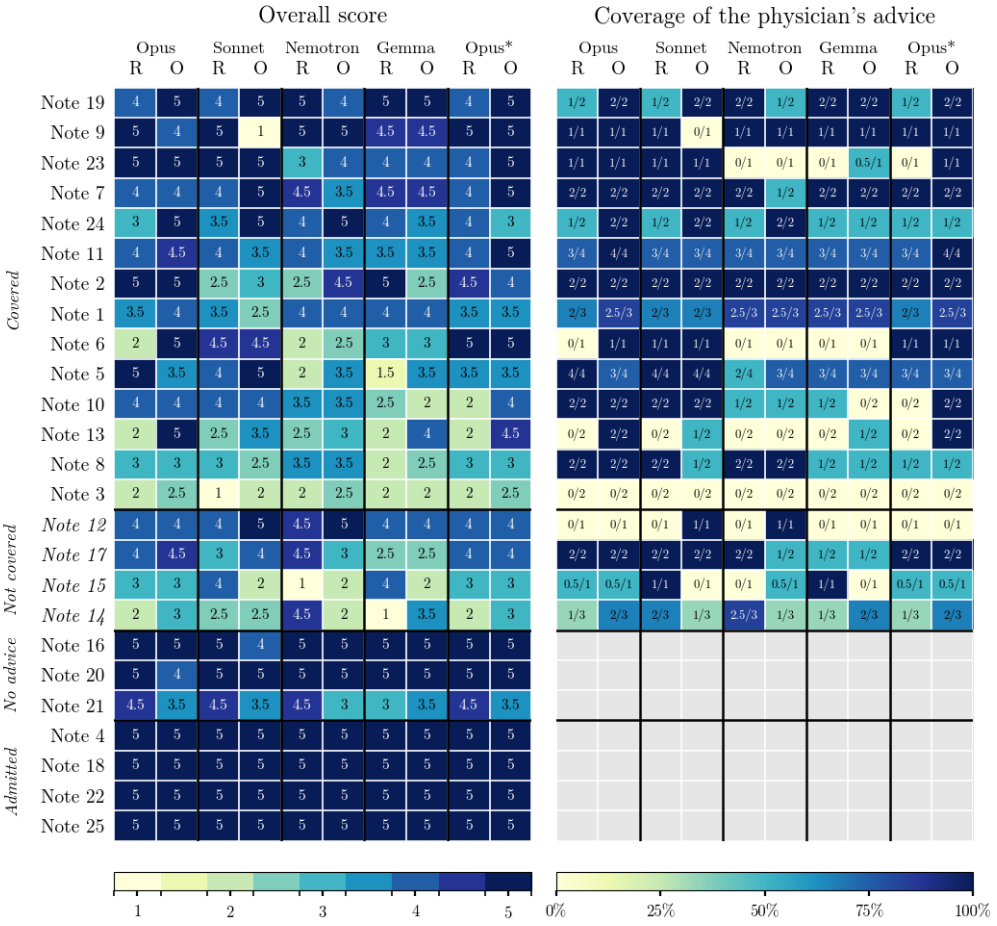


Figure 7.1. Overall score (left) and coverage of the physician's advice (right) of each note in the ten runs. R and O stand for rule-based and own queries, and Opus* for Opus in configuration 4. Notes the corpus does not cover are in italics. Coverage cells report the advice items found in the output over those in the advice.

7.4.2 Generation Models and Queries

Table 7.6 compares, note by note, the runs that differ in a single component. Due to the small amount of data available, almost no comparison reached statistical significance. While this means we cannot definitively establish whether a model is better than another, the trends still indicate which models performerd best. The lack of statistical significance should, therefore, be attributed to the small dataset over being taken as an indication of equality among models.

Comparison (A against B)	**A / tie / B**	**Mean diff.**	**Sign p**
Generation model, same evidence (rule-based queries)			
Opus vs Sonnet (Qwen3–8B)	4 / 16 / 5	+0.02	1.00
Opus vs Nemotron 3 Super (Qwen3–8B)	5 / 11 / 9	+0.10	0.42
Sonnet vs Nemotron 3 Super (Qwen3–8B)	5 / 11 / 9	+0.08	0.42
Opus vs Gemma 4 (MedCPT)	8 / 11 / 6	+0.24	0.79
Generation model, each with its own queries			
Opus vs Sonnet (Qwen3–8B)	12 / 9 / 4	+0.36	0.08
Opus vs Nemotron 3 Super (Qwen3–8B)	12 / 9 / 4	+0.38	0.08
Sonnet vs Nemotron 3 Super (Qwen3–8B)	10 / 9 / 6	+0.02	0.45
Opus vs Gemma 4 (MedCPT)	12 / 10 / 3	+0.46	0.035
Own queries against rule-based queries			
Opus (Qwen3–8B)	9 / 12 / 4	+0.30	0.27
Opus (MedCPT)	8 / 14 / 3	+0.30	0.23
Sonnet (Qwen3–8B)	9 / 9 / 7	−0.04	0.80
Nemotron 3 Super (Qwen3–8B)	9 / 10 / 6	+0.02	0.61
Gemma 4 (MedCPT)	5 / 16 / 4	+0.08	1.00
Cross-encoder, same model and queries (Opus)			
Qwen3–8B vs MedCPT, rule-based	4 / 19 / 2	+0.04	0.69
Qwen3–8B vs MedCPT, own queries	5 / 16 / 4	+0.04	1.00

Table 7.6. Paired comparisons of the overall score over the 25 notes: notes on which A scored higher, ties, and notes on which B scored higher; mean difference between A and B; two-sided sign test, excluding ties.

When the models received the same evidence, through the rule-based queries, they scored almost identically. Opus, Sonnet and Nemotron 3 Super were within 0.1 points of each other, with most notes tied, and Gemma 4 was 0.24 points behind Opus. Differences appeared when each model was also tasked with generating its own queries. In what was arguably a more complex task, requiring synthetization abilities and an understanding of both the content of the note and of the guidelines, Opus scored 0.36 points above Sonnet, 0.38 above Nemotron 3 Super and 0.46 above Gemma 4. The last comparison is to only one to have a p-value below 0.05.

The advantage of Opus lies in the coverage of the advice. With its own queries, Opus covered 84% of the advice items in configuration 1 and 78% in configuration 4, against 66% and 55% with the rule-based queries (Table 7.5). Every other model covered between 58% and 73% whichever queries it used, and writing its own queries changed its score by less than 0.1 points, against a gain of 0.30 for Opus (Figure 7.2). Part of the explanation lies in the queries of the smaller models. Nemotron 3 Super and Gemma 4 wrote a median of five queries per note, against eight for Opus and Sonnet, so their retrieval covered fewer aspects of each case. Gemma's queries were also the shortest, with a median of 13 words against 23 or 24 for the other models, and read more like keywords than like the recommendation-like sentences the prompt asks for. It is possible that, were a hospital to adopt that configuration, some prompt tuning specific to the model could have led to improved results. This is only an hypothesis and has not been tested in practice, since fine tuning or prompt engineering on specific models was avoided as to not introduce bias.

The models also differ in style. Gemma 4, which did not use reasoning, wrote the shortest outputs, and the fewest outputs with useful additions, 16 out of 50. Nemotron 3 Super

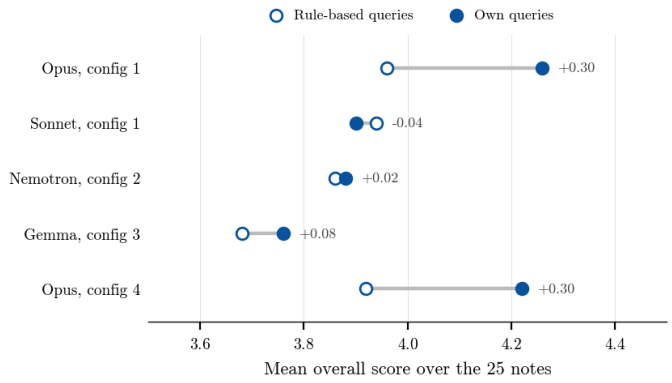


Figure 7.2. Mean overall score over the 25 notes of each generation model with the rule-based queries and with its own queries, showing the difference between the two.

produced the most, 28 of 50, while covering less of the advice: it added guidance beyond what the physician wrote, but missed some of what they did write.

7.4.3 Wrong and Harmful Statements

Over the 250 outputs, the evaluation found 19 wrong statements, 9 of which harmful or potentially harmful. Opus produced none of the harmful ones in its 100 outputs, while Sonnet produced 4 in 50, Gemma 4 produced 3 and Nemotron 3 Super 2. These numbers are too small for a formal comparison, but they follow the same pattern as the overall scores. Errors also weighed heavily on the overall judgement: the nine outputs with a harmful statement scored 1.8 on average, the ten with only harmless errors 3.3, and those without errors 4.05.

7.4.4 Cross-Encoder

Configurations 1 and 4 with Opus differ only in the cross-encoder, and their scores are practically identical, with 16 to 19 ties (depending on the querying mode) out of 25 notes. Qwen3-Reranker-8B led to a higher coverage of the advice, 84% against 78% with the queries of Opus and 66% against 55% with the rule-based ones, but this did not affect the scores by much. Although a third of the excerpts differed between the two cross-encoders (Section 7.3.4), the generator wrote recommendations of equal quality from either set. This suggests that the excerpts on which the two agree carry most of what the recommendations draw on.

7.5 Latency and Cost

A system meant to save time in an emergency department must itself be fast, and its running cost must be sustainable for a hospital. Processing time was measured on eight notes drawn at random, running each configuration on one note at a time, so that no call had to wait for another. Each measurement starts from the clinical note ingestion, in PDF format, and ends with the output, covering parsing, query construction, retrieval with its tables, and generation. It excludes translation, since all runs shared the stored DeepL translations, and

the loading of the models, which happens only once per session. The local components ran on the development laptop.

Cfg.	Model	Queries	Query writing	Retrieval	Generation	Total
1	Opus	rule-based	0.4	4.7	17.3	22.4
1	Opus	own	7.0	3.6	18.2	28.9
1	Sonnet	rule-based	0.4	4.0	12.1	16.5
1	Sonnet	own	4.5	4.1	13.3	21.8
2	Nemotron 3 Super	rule-based	0.4	4.3	107.8	112.6
2	Nemotron 3 Super	own	91.6	3.3	83.9	178.8
3	Gemma 4 31B	rule-based	0.4	1.0	7.2	8.6
3	Gemma 4 31B	own	32.9	0.7	6.1	39.7
4	Opus	rule-based	0.4	1.0	15.3	16.8
4	Opus	own	7.7	0.9	18.2	26.8

Table 7.7. Mean seconds per note over eight notes, from the PDF report to the output, excluding translation and the loading of the models. Parsing takes about 0.01 seconds.

Only configurations 1 and 4 are representative of a real deployment, since their generation model would be accessed through the same API there as well. The times of configurations 2 and 3 instead reflect the hardware and load of the API providers, not the hardware these configurations stand for, so they say little about the speed of a local deployment. They are reported here only for completeness, and are left out of Figure 7.3, which breaks down the times of the two frontier models of configuration 1.

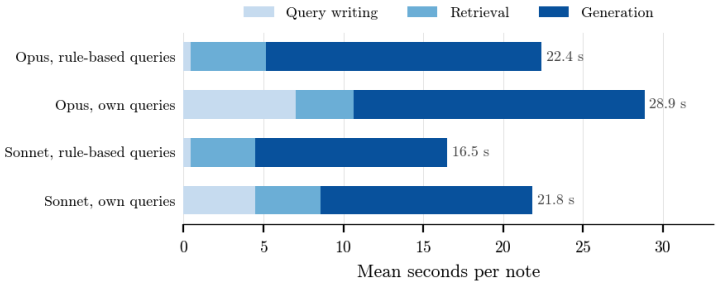


Figure 7.3. Mean seconds per note of Opus and Sonnet in configuration 1, by stage.

Generation takes most of the time in every configuration (Table 7.7), with the exception of API models simulating local development, where query generation was more time consuming. Retrieval took about one second with MedCPT on the laptop GPU and about four with Qwen3-Reranker-8B, where most of the time goes into the calls to the API. The rule-based queries take 0.4 seconds, while a model writing its own queries adds between 4.5 seconds for Sonnet and over 90 for Nemotron 3 Super. With Opus, a note took between 17 and 29 seconds on average, depending on the configuration and the queries. Translating the note with Opus adds about ten seconds more, so the best configuration produces its recommendations in about 40 seconds.

Cost An output written by Opus cost between 6.6 and 7.4 cents through the API, and one written by Sonnet about 3.8. Writing the queries and translating the note add a few cents when they use the same model: translation cost 2.9 cents with Opus and 1.4 with Sonnet, while query writing cost, respectively, 1.7 and 0.9 cents. Therefore, running the whole pipeline with frontier models would cost around 12 cents with Opus, and 6 with Sonnet.

The open-weight models cost a fraction of a cent per output through the API, but in a local deployment their real cost would be that of the hardware rather than of the calls. A detailed breakthrough of the costs can be found in Appendix B.3.

7.6 Discussion

In this section we offer an interpretation of the results displayed in this chapter in light of the research questions established in Section 1.3.1. A discussion on how these results could reflect on system deployment choices follows.

7.6.1 Comparison with the Physicians' Advice

RQ2 asks how the recommendations compare with those written by the physicians themselves. With the frontier model writing its own queries, the answer is largely favourable. These outputs covered more than four fifths of the clinical items in the physicians' advice, added useful guidance in about half of the cases and contained no harmful statement, with a mean score above 4 out of 5. Their readability was judged better than that of the original advice throughout. Actual examples are included in Appendix C.3.

The recommendations do not reproduce the advice, however, and the way they depart from it is consistent. They tend to omit what physicians prescribe out of local practice rather than guidelines, and to add what the guidelines recommend but the physician did not write. This is the behaviour the system was designed for, since every recommendation must be grounded in the retrieved excerpts. It also means that the quality of the recommendations depends on the corpus as much as on the model. Where the corpus covers the diagnosis, these outputs reached a mean score of 4.4, while for the diagnoses it does not cover they fell to about 3.5, as the system had little evidence to draw on.

These results should also be read with the nature of the score in mind. The overall score closely followed the coverage of the advice (Section 7.4.1), so it measures agreement with the physician at least as much as clinical correctness. Where guidelines and local practice diverge, an output can follow the guidelines and still score low, as in the case of acute bronchitis, where the system did not recommend the therapy the physician had prescribed because the guideline does not support it.

The divergence between guidelines and local practice is not an accident of this evaluation. Physicians do not consult a guideline each time they discharge a patient, but rely mostly on their own knowledge and on the practice of their department, and adherence to guidelines in everyday practice can be partial[198]. The corpus used here was built to be as general as possible (Chapter 5), whereas a region or a hospital may follow protocols of its own, and grounding the system in those might have brought its outputs closer to the physicians' advice. For this reason, the corpus is not fixed: the system accepts any set of guidelines as input, so a hospital can extend or replace the corpus with the documents its physicians actually follow.

7.6.2 From Frontier to Hosted Models

RQ3 asks how quality changes when the frontier model is replaced by models a hospital could host on its own hardware. Quality does decrease, but less than might be expected, and not where it might be expected. Given the same evidence, Nemotron 3 Super wrote recommendations as good as those of the Claude models, and Gemma 4 was only slightly behind. Writing recommendations from a given set of excerpts is therefore within reach of open-weight models of the size tested. Likewise, translation quality did not significantly decay in smaller models.

Instead, the losses come from the queries. Opus gained clearly from writing its own queries, while the open-weight models gained nothing, and the best local configurations remained about 0.4 points below Opus with its own queries, mostly because they covered less of the advice and made more errors. Deciding what to look up for a patient is a different task from writing recommendations once the evidence is given, and it appears to demand more of a model. The open-weight models wrote fewer and shorter queries, so their retrieval covered fewer aspects of each case. Quantity is not the whole story, though, since Sonnet wrote as many queries as Opus, of similar length, and gained nothing from them either. For a local deployment, the rule-based queries are therefore the better choice: the queries written by the local models were no better, while the rules build theirs in less than a second, without an additional call to the model.

The cross-encoder, conversely, does not matter as much. With Opus as the generator, MedCPT, which weighs 0.2 GB, led to recommendations as good as those obtained with Qwen3-Reranker-8B, weighing approximately 80 times as much. This equivalence was only tested with Opus, however, and a weaker generator might depend more on the precision of the evidence it receives. Furthermore, the coverage results indicate that Qwen3-Reranker-8B produced better retrieval results.

Smaller models that fit the development laptop or a slightly more powerful configuration, namely Mistral 7B[199], MedGemma 4B[200] and Qwen 3.5 9B[201], were tried during development and produced outputs that could not serve as discharge documents, so they were not included in the evaluation.

Opus vs. Sonnet Within configuration 1, the comparison between Opus and Sonnet answers a narrower question, namely whether the more expensive model is worth its price. On the same evidence the two were equivalent, with most notes tied. With their own queries, Opus scored better on 12 notes and worse on 4, and across all its runs it made no harmful statement, against four for Sonnet. While neither evidence is conclusive on 25 notes, the evidence therefore seems to favour Opus despite the higher cost per note. An hybrid setup could also be considered, though it has not been tested, with Opus writing the queries and the cheaper Sonnet translating the note and writing the recommendations.

7.6.3 Deployment Recommendations

The best setup depends above all on whether the clinical note may leave the hospital. If it may be sent to an external API, under the appropriate agreements, the best results came from Opus writing its own queries. The same model should also translate the note, as Section 7.2 suggests. The cross-encoder, instead, can run locally and still produce satisfactory results. This setup produces its recommendations in slighly more than half a minute, at a cost in the

order of a few cents per note.

If the note must stay within the hospital, the system should use the rule-based queries and MedCPT, and the hardware available decides the generation model. A server with about 160 GB of memory can host Nemotron 3 Super, although its long reasoning makes it slow, and the results were only marginally better than the smaller Gemma-4. Other similar models may perform better and could be integrated seamlessly into the system, but they were not tested. A single 24 GB workstation can host Gemma 4, only slightly weaker but much lighter. Nemotron 3 Super was evaluated only with Qwen3-Reranker-8B, so its pairing with MedCPT, though consistent with the results above, remains untested. Translation would need to be handled by the generation model, since DeepL would send the note outside the hospital.

In every scenario, the outputs remain drafts for a physician to review rather than documents to sign unread. Even the best runs contained wrong statements, and every open-weight configuration produced potentially harmful ones in at least one of its runs, so the value of the system lies in providing a complete, well-written draft that the physician can easily correct instead of writing from scratch.

Chapter 8

Limitations

8.1 Guideline Corpus

Since the generator is instructed to rely exclusively on the retrieved excerpts and the clinical note, the guideline corpus sets an upper bound on what the system can recommend. When a topic is missing from the corpus, the prompt directs the model to defer instructions to the physician. This is the safer failure, but it still produces a less useful output for a physician utlising the system. The evaluation made this cost visible: outputs for the notes whose diagnosis the corpus does not cover scored 3.29 on average, against 4.06 for those it covers (Section 7.4.1). Section 5.7.4 briefly introduced the weaknesses of the corpus; this section examines them in more detail, together with the ways they could be addressed.

The most structural limitation concerns what the scraper is allowed to collect. In fact, each filter applied during the search keeps the corpus focused, but it also excludes a specific category of documents. Restricting the search to PubMed records tagged with the Guideline or Consensus Statement publication type means that documents not indexed in PubMed, or indexed under a different publication type and ambiguous naming, are never retrieved. The restriction to English, in turn, excludes national guidelines written in other languages, since the current model configuration is not able to use them. This is a significant limitation for a tool created for usage in Italian Emergency Departments, since it requires the translation of Italian Guidelines to work.

Beyond the search filters, the corpus also depends on which documents can actually be downloaded. The automatic download can only fetch freely available PDFs, so guidelines behind a paywall must be retrieved through the assisted manual step, which relies on the user's institutional subscriptions. Even with the access provided by the University of Trento, 26 of the selected guidelines could not be obtained, as the University did not subscribe to the publisher, and were left out of the corpus. A hospital or any other third party running the scraper would face the same constraint, and since subscriptions differ between institutions, the corpus obtained from the same configuration also depends on who runs it.

A second group of limitations is the price paid for automation, since the scraper relies on an LLM at various stages. By design, neither stage involves manual review, so that a third party can run the scraper without assistance. The drawback is that errors propagate directly into the corpus: a poorly chosen search term silently leaves a topic uncovered, while an incorrect selection adds a guideline that is off-topic or has since been superseded, while

potentially ignoring an informative one. Moreover, LLM outputs are not guaranteed to be identical across runs, so running the scraper again with the same configuration may yield a slightly different corpus, limiting the reproducibility of the exact corpus used in this thesis.

The evaluation of the corpus presented in Chapter 5 is itself limited in scope. Recall was measured on five large European societies (ESC, ERC, ESO, ESGE and ERS), but it could be lower for smaller societies, whose guidelines may be harder to locate. Many of those societies do not publish a complete list of their publications to use as a check, and a comprehensive list of societies publishing guidelines useful for Emergency Medicine does not exist, so a complete recall test is not possible.

The coverage check, in turn, only considered the diagnoses of the twenty-five evaluation notes, which represent a small fraction of the conditions encountered in emergency care.

Finally, even a complete corpus would not be a perfect fit for the task, since guidelines are written to cover the entire course of a disease rather than solely the acute phase managed in the ED. A substantial part of their content concerns long-term treatment and specialist diagnostic pathways, and while reranking filters most of it out, some of this material still reaches the prompt.

8.2 System Design and Deployment

The pipeline is a chain of components, and a mistake made in an early stage can potentially propagate through the remaining stages. By the time the recommendations reach the physician, its origin is no longer visible. This section discusses the design choices that could expose the pipeline to such errors, and the constraints that separate the evaluated configurations from one the partner department could adopt.

The first dependency is the input. The parser relies on the vocabulary and layout of the department's report. It is designed to fail closed, refusing document structure not resembling that of a clinical note or a provisional template. Since it relies on vocabulary rather than a fixed structure, it could potentially adapt to documents from different hospitals than that of the partner department. However, it is also possible that parsing would fail on such documents, requiring an adaptation of the system. Translation adds a second dependency, since every later component operates on English text. The evaluation showed that this risk is concrete, as DeepL inverted the sex of two patients and changed the route of administration of a drug (Section 7.2), and any brand name missing from the glossary remains invisible to retrieval.

In the rule-based mode, query construction is the stage most exposed to silent errors. An entity missed by the NER model, or wrongly discarded as negated, produces no query, while a wrong choice of the guidelines associated with the diagnosis removes the relevant one from several queries at once. Since the cross-encoder can only reorder what was retrieved, nothing downstream can recover this content. The LLM-based mode avoids these failure points, but it shift the issue to the ability of the language model to convey information efficiently and securely. In practice, as shown in Section 7.4.2, rule-based queries performed as well as those produced by non-frontier models.

Grounding, finally, rests on instructions rather than on verification. The prompt requires

every recommendation to cite its source or to declare that it has none, and asks the model to check its own answer, but these checks are performed by the same model that writes the recommendations. Despite numerous reiteration of this instruction, there is no absolute guarantees that the LLM of choice does not deviate from it. No independent step verifies that a cited excerpt actually supports its item, and the burden of this verification is left to the reviewing physician.

Hardware constraints shaped both development and evaluation. All local components had to fit on a 6 GB laptop GPU, which restricted the embedder, the NER model and the cross-encoder to compact models, and the local deployment tiers had to be simulated through OpenRouter. This gives a faithful picture of output quality, but not of speed, since the measured latencies reflect the providers' infrastructure rather than the hardware a hospital would use.

The most significant gap, however, lies between the evaluated configurations and what the partner department could deploy. The department ideally requires any deployment to be able to run without a dedicated GPU, at no cost, and with all data kept on site. The best configuration violates the last two conditions, as it sends the note to a commercial API at about twelve cents per note, while those that keep the note within the hospital require a 24 GB workstation GPU. None of the evaluated configurations can therefore currently be deployed there. Closing this gap would require the hospital to invest in local hardware, to accept sending anonymised notes to an external provider under appropriate agreements (Section 8.4), or to wait for the development of open-weight models that run without a GPU while matching the quality and speed of today's larger ones. Until then, the results describe what the approach can achieve rather than what the department could adopt in its current state.

8.3 Threats to Validity

The evaluation in Chapter 7 was designed around the constraints that surround clinical data, and several of its choices, while necessary, limit how far its results can be generalised. In this section, we review them in the order in which they enter the evaluation: the data, the reference, the judges and the procedure.

The first threat concerns the notes themselves. Since real notes cannot leave the hospital without the approval of an ethics committee, the evaluation relies on 25 synthetic notes written by physicians of the partner department. They imitate real reports closely, down to their abbreviations and typing errors, but they remain cases written for an evaluation by a single department, and may not capture the full variety of real emergency visits. With so few cases, almost no paired comparison reached statistical significance. The trends discussed in Section 7.6 should therefore be read as indications rather than established differences.

The second threat lies in the reference. The physicians' advice was meant as a reference rather than a gold standard, yet the overall score followed its coverage closely, as indicated by a Spearman correlation of 0.70. The score therefore measures agreement with the physician who wrote the note, and an output that follows the guidelines where local practice differs is penalised, as in the case of acute bronchitis. The outputs were scored by the author with the aid of a single emergency physician, and no second, independent rating was collected, so the reliability of the scores, which remain a subjective measuring, cannot be estimated. Retrieval,

in turn, was judged by a language model. Its verdicts proved stable, with a κ of 0.68 between repeated judgements, but stability is not accuracy: its agreement with a physician was never measured, so the judge could be consistently wrong in ways the evaluation cannot detect.

Finally, two further limitations concern how the evaluation was run. First, while the system was tuned and constructed on separate development notes, the comparisons that selected the embedder, the search depth and the cross-encoder were run on the same 25 notes used for the end-to-end evaluation. Second, each configuration produced a single output per note. Language models served through an API do not always return the same output for the same input, and since the variance between runs was not measured, some of the smaller differences between configurations may fall within it.

8.4 Ethical and Regulatory Considerations

The system processes some of the most sensitive data a hospital holds, and produces documents that patients will act upon, so its limitations are not only technical.

The privacy measures described in Section 4.1.3 reduce exposure, but do not legally eliminate it. Anonymisation removes the identifiers the hospital software prints in known places, together with titled names and formatted identifiers, but it cannot catch those it was not designed for, such as a relative named without a title (Section 6.3.2). Even a note stripped of every name still combines age, sex and a diagnosis, which together may be enough to single out a patient. Under the GDPR, data are anonymous only when the person can no longer be identified by any means reasonably likely to be used, and health data form a special category of personal data[17]. This matters for the configurations that rely on external services, since sending a note to DeepL or to any model served through an API means sharing it with third party servers. Doing so would require a legal basis, agreements governing their role as data processors and, for providers outside the European Union, compliance with the rules on international transfers. Only a fully local configuration avoids these obligations towards third parties, which makes the deployment gap discussed in Section 8.2 an ethical issue as well as a technical one.

The second concern is the role of the physician. Every output is a draft, meant to be reviewed and signed by a physician who remains responsible for its content, but this safeguard only works if the review is genuine. Users of decision support systems tend to over-rely on their output, a phenomenon known as automation bias, and its effect grows with workload and time pressure[202], which are precisely the conditions of an emergency department. This phenomenon appears to be particularly prominent in the usage of LLM assistants, whose confident tone lead users to easily trust their outputs[12]. Despite the system being void of any tone, the inherent trust we place on language models is still a major factor. This creates a tension at the core of the system: its value lies in saving time, but the more time it saves, the less attention its outputs are likely to receive. The risk is concrete, since the evaluation itself showed that the models can produce wrong statements (Section 7.4.3). The design mitigates it in part, as every item points to its source and unsupported items must be declared as such, but these measures only help if they are read.

Finally, the system would fall within the scope of European regulation as soon as it left the research setting. Under the Medical Device Regulation[203], software intended to support

diagnostic or therapeutic decisions for individual patients can qualify as a medical device, and a tool proposing therapies and follow-up for a specific patient falls within this definition. Under the AI Act[204], an AI system that is itself a product of this kind is classified as high-risk, and must meet strict requirements on risk management, data governance, documentation, transparency and human oversight. Article 14 in particular explicitly requires that the people overseeing such a system remain aware of automation bias, which turns the concern discussed above into a legal obligation. Since none of these regulations applies to research and development carried out before a system is placed on the market, they do not concern the prototype of this thesis, but any future potential deployment would have to address them.

8.5 Future Work

We believe there are five broader directions that would bring the system closer to everyday use and actual deployment in a clinical context.

Anonymisation Anonymisation decides where the system can run, and could represent the biggest step towards an effective deployment. As mentioned above, the current procedure is a mitigation rather than a guarantee (Section 8.4), and a stronger approach could make use of third party technologies to ensure full data anonimity. This could follow the approach of the eCREAM project[190], where notes first had every reference to the patient and the clinical context removed, and then passed through certified anonymisation software that caught residual references to third parties, such as the names and contact details of relatives, which are precisely the identifiers the current rules can miss. This process allowed the project to obtain ethics approval for the transfer of real emergency notes in most of the participating centres, although after long deliberations. The eCREAM pipeline also removed diagnoses and dispositions, which this system cannot do, since the diagnosis anchors retrieval and the outcome of the visit decides its route. The remaining quasi-identifiers can nonetheless be removed at little cost: the exact age could be replaced by the age group that retrieval already relies on, while sex, which currently affects the choice of drugs, might influence retrieval if removed. On the test notes, leaving it out did not affect the outcomes. A procedure robust enough to satisfy ethics committees and data protection officers could make external models a viable option for hospitals without the hardware to run them locally.

Multilingual Corpus. Since the system accepts any set of guidelines, the corpus could be extended with, or substituted by, guidelines written in Italian and with the regional and departmental protocols that physicians actually follow, bringing the outputs closer to local practice where it diverges from international guidelines. Doing so would require the pipeline to work beyond English. The embedding model is already multilingual, but the NER model, the negation rules and the cross-encoder are not (though multilingual cross-encoders were tested and performed similarly to the adopted model), and replacing them with multilingual counterparts would allow Italian and English documents to be searched together, directly from the Italian note. This would also remove translation from the retrieval stage, together with the errors it introduces. However, the performance of the multilingual models on the Italian language has not been tested and might be of lesser quality.

Prospective Evaluation. With ethics approval in place, physicians could use the system on real patients during their shifts, while measuring what this thesis could not: the time

needed to review and correct each output compared with writing the advice from scratch, how often and how substantially the outputs are edited, and whether patients understand the resulting recommendations better. This would also give access to a larger sample for a more statistically significant evaluation process.

Closing the Query Gap. The evaluation showed that open-weight models write recommendations nearly as well as Opus when given the same evidence, and that the gap between frontier and local models lies mostly in the queries they write (Section 7.6.2). This makes query writing a natural task for a smaller model trained specifically for it. Since a set of queries written by Opus costs less than two cents per note, a training set could be collected cheaply on synthetic or anonymised notes, and used to fine-tune an open-weight model to reproduce them, an approach known as sequence-level knowledge distillation[205].
Prompts tailored to each model, deliberately avoided in the evaluation to keep the comparison fair, are a lighter alternative worth testing first.

Integration. The system would also benefit from closer integration with the department's software. Today the physician has to export the provisional report from the information system, open it in a separate piece of software and only then read the recommendations. Integrating the tool in the existing software, so that it reads the note directly and shows its output next to it, would remove this detour and the time it costs on every case.

Chapter 9

Conclusions

The discharge recommendations of an Italian Emergency Department take up little space in the report, but they carry a great deal of responsibility. They are the main written channel through which patients and their general practitioners learn what should happen after the visit, yet they are written under the same time pressure that shapes the rest of the visit. As a result, they are often brief, dense with abbreviations and incomplete, and the secondary guidance on lifestyle, follow-up and warning signs are often left out. As discussed in Chapter 1, general-purpose language models can already rewrite such text clearly, and physicians have started to use them for this purpose. However, these models draw on knowledge that cannot be traced to a source, cite references that often fail to support their claims, and require health data to leave the hospital without any prior privacy measures in place. The premise of this thesis was therefore that the missing piece is not the ability to write, but control over where the clinical content comes from, a way to verify it, and a deployment compatible with the handling of health data.

To meet these requirements, this thesis designed, implemented and evaluated a Retrieval-Augmented System that drafts discharge recommendations from the Italian clinical note of an ED visit. The system reads the incomplete clinical note produced by the department's information system, anonymises it, translates it into English and recomposes it into a set of focused queries, built either by rules or by the language model itself. These queries retrieve excerpts and tables from a corpus of European clinical guidelines, and the language model writes the recommendations from the note and this evidence alone, linking each of them to the excerpts that support it. The result is a draft for the physician to review, rather than a document to sign unread. The diagnosis and the prognosis remain the physician's, and the references exist precisely so that each recommendation can be checked without searching for its source. Since the system can only recommend what its corpus contains, the thesis also built the corpus itself, through an automated procedure that collects European guidelines across the whole Emergency Medicine discipline.

Building such a corpus by hand would be slow, and it would still lack a point of reference, since no complete list of the guidelines relevant to emergency medicine exists. **RQ1** asked whether the collection could be automated instead, and the results of Chapter 5 indicate that it can. Starting from the chapters of a reference textbook and from the topics of a model of Emergency Medicine practice, the scraper searches PubMed for European guidelines and lets a Large Language Model select among the results, without the need for any manual review

of its choices. Its final run selected 457 distinct guidelines in about 45 minutes, at a cost of roughly $24 in model calls. The guidelines it picks are almost always the right ones, as nearly every sampled document proved to be a sound, current guideline on the topic it had been chosen for. Recall was similarly high on the five European societies examined, with 41 of their 44 relevant guidelines entering the corpus. Since all 44 had reached the candidates of the search, the three misses came from the preference of the selection for broad guidelines over narrow ones, rather than from the search itself. Where the corpus failed to cover the diagnosis of an evaluation note, which happened for four notes out of 25, a manual search found no suitable guideline either, so we conclude that these gaps lie in the literature rather than in the procedure. What automation cannot overcome, instead, is access. Guidelines behind a paywall depend on the subscriptions of whoever runs the procedure, and 26 of the selected guidelines could not be obtained even with the institutional access of the University of Trento. The restriction to English, in turn, leaves out the national and regional protocols that an Italian department may follow.

The central question of the thesis, **RQ2**, asked how the generated recommendations compare with those written by physicians. It was answered on 25 synthetic but realistic notes written by physicians of the partner department, whose outputs were scored blind by the author with the aid of an emergency physician (Chapter 7). With Claude Opus 5.5 writing its own queries, the comparison is largely favourable to the system. Its outputs contained 84% of the clinical items in the physicians' advice and added useful guidance in about half of the cases, without a single harmful statement. Their mean score was 4.26 out of 5. In every configuration that was tested, every output was more readable than the original advice, and every configuration consistently supplied instructions that physicians had almost always left out, namely when to return to the ED and when to see the general practitioner. They also added useful indications relating to lifestyle habits and secondary care, which the physicians often omitted to save on time. These are precisely the kinds of omission that motivated this work. Where the outputs departed from the advice, they did so in a consistent direction, following the instructions guidelines recommended but the physician had not written. This is the behaviour that grounding is meant to produce, but it also means that the quality of the recommendations depends on the corpus as much as on the model. In the same configuration, the notes whose diagnosis the corpus covers scored about 4.4, against about 3.5 for those it does not. These findings come from a small set of notes from a single department, and few of the differences between configurations reached statistical significance. They should therefore be read as consistent indications rather than as established results (Section 8.3).

Moving away from the established frontier models towards a realistic local deployment was the subject of **RQ3**, which asked how the quality of the recommendations changes when open-weight models that a hospital could host on hardware are used instead. Quality does decrease, but less than expected, and mainly in a specific stage of the pipeline. Given the same evidence, Nemotron 3 Super wrote recommendations as good as those of the Claude models, and Gemma 4 31B, which fits on a single 24 GB workstation GPU, was only slightly behind. The gap opened when each model was also tasked with writing its own retrieval queries. Opus clearly gained from doing so, while Sonnet and the open-weight models gained nothing compared to the faster and cheaper rule-based queries. In fact, deciding what to look up for a patient appears to demand more of a model than writing recommendations once the evidence is given. For a local deployment, the rule-based queries are the better choice, since they performed at least as well as those written by the local models and are built in under a second. The cross-encoder, by contrast, mattered little: with Opus as the generator,

MedCPT led to recommendations almost as good as those obtained with a reranker about 80 times its size. When deciding which models to fit in a limited amount of memory, a larger language model is to be prioritized over the reranker. The open-weight models did, however, occasionally produce potentially harmful statements regardless of configuration, which Opus never did. Models small enough for the development laptop, finally, could not produce usable recommendations at all.

Taken together, these results suggest that the limits of the approach lie less in the language model than in what surrounds it. The corpus sets an upper bound on what can be recommended, the queries decide how much of it reaches the model, and data protection limits which models can be used in the first place. This last constraint is currently the most binding one. The best configuration produces its recommendations in about 40 seconds and for about 12 cents per note, but it sends the note to an external provider. The configurations that keep the note within the hospital avoid this completely, but require at least a moderately powerful dedicated GPU. Neither option meets the conditions of the standard Italian Emergency Department, which would require a deployment without a GPU, without running costs and with all data kept on site (Section 4.4). The results of this thesis therefore describe what the approach can achieve, rather than what a department could adopt today.

Some of the directions outlined in Section 8.5 would narrow this distance. A certified anonymisation procedure could make external models acceptable to ethics committees and data protection officers, while utlising the queries of a frontier model to train a smaller one could close the gap exactly where local models fall short. A prospective evaluation on real patients would measure what this thesis could not, namely whether reviewing a draft is actually faster than writing the advice from scratch, and whether patients understand the result better.

This thesis started from the observation that ED physicians know what their patients should do, but lack the time to write it down completely and clearly. The system presented here does not replace their judgement, since even its best outputs require review. It does show, however, that a complete, readable and traceable draft can be produced from official guidelines in less than a minute, leaving the physician to correct it rather than to write it.

Appendix A

Guideline Corpus

A.1 Prompts

Title terms

Prompt used to produce 1–6 terms describing the content of a chapter from Tintinalli's *Emergency Medicine* based on its title.

```
Below are chapters of Tintinalli's Emergency Medicine, as "number | section | title".
For each chapter, give:
- terms: 1-6 phrases that the title of a clinical practice guideline on the chapter's core
    topic would use.
- population: "pediatric", "pregnancy" or "elderly" if the chapter (its title or section)
    is about that patient group and the terms don't already name it; otherwise null.

Rules:
- Only the chapter's core topic. No broader categories and no neighbouring conditions (e.g.
     "Accidental Hypothermia" must not include therapeutic hypothermia).
- No generic words: guideline, management, treatment, diagnosis, recommendation.
- Plain phrases only: no quotes, operators, parentheses or wildcards.

Return only JSON: {"<number>": {"terms": [...], "population": ...}}
Examples:
{"273": {"terms": ["hip fracture", "femoral fracture", "proximal femur fracture"], "
    population": null},
 "121": {"terms": ["otitis media", "otitis externa", "mastoiditis"], "population": "
    pediatric"},
 "118": {"terms": ["sudden infant death syndrome", "sudden unexpected infant death"], "
    population": null}}
```

Reference topics

Produces up to three terms per chapter describing its subtopics based of the references.

```
Below are chapters of Tintinalli's Emergency Medicine, each with the guideline search terms
     already taken from its title and the MeSH topics its own references cite most, as "
    topic (references in this chapter, chapters citing it)".
For each chapter, pick up to 3 reference topics that are core subjects of the chapter the
    title terms miss, so the guideline search should also cover them. Skip topics a title
    term already covers (same condition, synonym, subtype or parent), topics that are only
    complications, comorbidities, differential diagnoses or treatment side effects, and
    generic topics cited across many chapters. Fewer than 3, or none, is fine.
```

```
Return only JSON: {"<number>": ["<topic exactly as listed>", ...]}
```

EM item names

Names each item of the EM Model by the condition it stands for, so that its name can be checked against the chapters' topics and, when no chapter covers it, searched in PubMed.

```
Below are items of the Model of the Clinical Practice of Emergency Medicine (ABEM, 2022),
     as "number | section | headings -> item".
For each item, give the condition or problem it stands for, named as the title of a
     clinical practice guideline on it would name it:
- Read the item under its headings: "Acid" under "Toxic effects of caustic agents" is "acid
      ingestion", "Tension" under "Pneumothorax" is "tension pneumothorax".
- One name per condition: an item joining conditions with "/" gets one for each ("
     Epididymitis/Orchitis": "epididymitis", "orchitis"). No synonyms, no broader or
     narrower conditions.
- Plain phrases only: no quotes, operators, parentheses or wildcards.

Return only JSON: {"<number>": ["<name>", ...]}
Examples:
{"2.2.2.3.1": ["acid ingestion"], "16.2.6.2": ["tension pneumothorax"], "15.5.3.2": ["
     epididymitis", "orchitis"],
 "17.1.11.3": ["beta-blocker poisoning"]}
{items}
```

Selection

Chooses, for each chapter of the textbook, the smallest set of guidelines among its PubMed candidates that covers the chapter's topics. The `{qualifies}` and `{outsiders}` clauses are filled by the European or the fallback rules below.

```
Chapter of Tintinalli's Emergency Medicine: "{title}" (section: {section}).
Topics of this chapter: {terms}
PubMed records, newest first, as "PMID | date | journal | publication types | title":
{records}

Select guidelines for a corpus of European clinical guidelines, following these rules.

Purpose: the corpus helps emergency physicians write discharge letters, so a pick must
     guide how an emergency physician assesses, treats or discharges a patient with the
     topic's condition.

{qualifies}

Reject: {outsiders}; guidelines on a different condition; guidelines for a different
     population than the chapter (e.g. pediatric-only for an adult chapter); studies of
     guideline adherence or implementation; reviews and commentaries on a guideline;
     versions republished in another journal or language when the original is listed; pocket
      or summary versions when the full guideline is listed; older versions when a newer
     version of the same guideline is listed; documents limited to imaging, laboratory
     methods, classification criteria, training or surgical technique; systematic reviews
     that issue no recommendations.

How many: the smallest set of guidelines that together cover the topics.
- Each topic is covered by at most one guideline: the one that covers it best; among equals,
      the most recent.
- For the same topic, prefer a clinical practice guideline to a consensus statement or
     position paper.
```

```
- Prefer one guideline that covers several topics over several narrower ones.
- Add a guideline only if it covers a topic that no other selected guideline covers.
- A topic may stay uncovered. Do not cover it with a guideline on a neighbouring condition.
- If no record qualifies, return an empty list.

Answer with only JSON:
{"picks": [{"pmid": "<PMID>", "covers": ["<topic>", ...], "name": "<short CamelCase name of
      the guideline's topic, e.g. AcutePulmonaryEmbolism>"}]}
```

European tiers:

```
qualifies: Qualifies: a clinical practice guideline, consensus statement or position paper
     issued by a European medical society (ESC, ERC, ESO, EAACI, ESCMID, ESAIC, ERS, EAU,
     EASL, UEG, ESPGHAN, ...) whose subject is one of the topics above. A national society
     of a European country qualifies only for a topic that no Europe-wide guideline in the
     list covers.
outsiders: non-European bodies
```

Fallback tier, for chapters left without a European pick:

```
qualifies: Qualifies: a clinical practice guideline, consensus statement or position paper
     whose subject is one of the topics above, issued by a European medical society, by
     medical societies of European Union countries (one or several), by the United Kingdom's
      NICE, by the World Health Organization, or by the Clinical Toxicology Recommendations
     Collaborative (led by the European EAPCCT with AACT and APAMT). For the same topic,
     prefer a Europe-wide guideline to a national one (NICE's included), and an Italian
     society's to another country's.
outsiders: bodies of countries outside the European Union other than NICE (the United
     Kingdom's BTS or SIGN included) and other international bodies
```

EM topics, in place of the chapter wording above when the topic is an EM Model section:

```
Section of the Model of the Clinical Practice of Emergency Medicine (ABEM, 2022): "{title}"
      (section: {section}).
Topics of this section: {terms}
PubMed records, newest first, as "PMID | date | journal | publication types | title":
{records}

Select guidelines for a corpus of European clinical guidelines, following these rules.

Purpose: the corpus helps emergency physicians write discharge letters, so a pick must
     guide how an emergency physician assesses, treats or discharges a patient with the
     topic's condition.

{qualifies}

Reject: {outsiders}; guidelines on a different condition; guidelines for a different
     population than the topics (e.g. pediatric-only for a topic of all ages); studies of
     guideline adherence or implementation; reviews and commentaries on a guideline;
     versions republished in another journal or language when the original is listed; pocket
      or summary versions when the full guideline is listed; older versions when a newer
     version of the same guideline is listed; documents limited to imaging, laboratory
     methods, classification criteria, training or surgical technique; systematic reviews
     that issue no recommendations.

How many: the smallest set of guidelines that together cover the topics.
- Each topic is covered by at most one guideline: the one that covers it best; among equals,
      the most recent.
- For the same topic, prefer a clinical practice guideline to a consensus statement or
     position paper.
- Prefer one guideline that covers several topics over several narrower ones.
- Add a guideline only if it covers a topic that no other selected guideline covers.
```

```
- A topic may stay uncovered. Do not cover it with a guideline on a neighbouring condition.
- If no record qualifies, return an empty list.

Answer with only JSON:
{"picks": [{"pmid": "<PMID>", "covers": ["<topic>", ...], "name": "<short CamelCase name of
      the guideline's topic, e.g. AcutePulmonaryEmbolism>"}]}
```

Consolidation

Decides whether two picks, or a pick and a newer record with a similar title, are the same document, two versions of one guideline, or different documents, so that only the right copy is kept.

```
Below are numbered pairs of records with alike titles: a guideline picked for a corpus and
     either another pick or a newer record, each as "PMID | date | journal | title". Give a
     verdict for every pair:
- "same": the same document in two journals or translated, or a summary, pocket or
     practical-instruction version of the other; keep the full original, in the society's
     own journal;
- "newer": two versions of one guideline (same society, same topic; an "update of" a
     guideline is a newer version); keep the newest, if it is the full guideline (not a
     summary of it or a commentary on it). A guideline from another society (a national one
     included), or a practical, pocket or summary version, is never a newer version;
- "different": different documents: other topics or populations, separate parts of one
     guideline, a focused update (titled so) and the guideline it updates, companion
     documents; keep both.
Answer with only JSON: {"verdicts": [{"pair": <number>, "verdict": "same|newer|different",
     "keep": "<PMID to keep, for same and newer>"}, ...]}, one per pair.

{pairs}
```

PubMed search

Each chapter's search combines its topic query, the guideline publication types of the first tier (the second tier when the first finds fewer than three records) and the European filter. The example is the pediatric pneumonia chapter. Long lists are abbreviated by dots. Its topic query is:

```
((community-acquired pneumonia) OR (pneumonia)) AND (child OR infant OR pediatric OR
     adolescent)
```

The search sent to PubMed is:

```
(((community-acquired pneumonia) OR (pneumonia)) AND (child OR infant OR pediatric OR
     adolescent))
AND (guideline[pt] OR practice guideline[pt] OR guideline*[ti] OR consensus statement[pt])
AND ((europe*[ti] OR europe*[cn] OR europe*[ad] OR europe*[ta] OR "the europe*"[ta]
  OR ESC[ti] OR ESMO[ti] OR EAACI[ti] OR ESTRO[ti] OR ESPGHAN[ti] OR ESPEN[ti] OR EULAR[ti]
      OR EHRA[ti] OR ...
  OR ESCMID[ti] OR ... OR EU[ti] OR ...)
  AND english[la] AND 2010:3000[dp] NOT (comment[pt] OR editorial[pt] OR letter[pt] OR
     published erratum[pt]))
```

A.2 Tables

Section of the textbook	Chapters with a pick
Resuscitation	11/11
Resuscitative Procedures	1/1
Analgesia, Anesthesia, and Procedural Sedation	4/4
Wound Management	4/9
Cardiovascular Disease	14/14
Pulmonary Disorders	8/9
Gastrointestinal Disorders	17/17
Renal and Genitourinary Disorders	8/8
Obstetrics and Gynecology	10/10
Pediatrics	43/44
Infectious Diseases	12/14
Neurology	11/11
Toxicology	19/32
Environmental Injuries	10/15
Endocrine Disorders	7/8
Hematologic and Oncologic Disorders	10/10
Eye, Ear, Nose, Throat, and Oral Disorders	7/7
Dermatology	6/6
Trauma	10/13
Orthopedics	9/12
Musculoskeletal Disorders	7/7
Psychosocial Disorders	7/7
Abuse and Assault	3/3
Special Situations	5/5
All sections	243/277

Table A.1. Chapters with at least one pick, by section of the textbook, as named on the companion site.

Category of the EM Model	EM topics with a pick
2 Abdominal and gastrointestinal disorders	21/27
3 Cardiovascular disorders	7/11
4 Cutaneous disorders	9/11
5 Endocrine, metabolic, and nutritional disorders	5/8
6 Environmental disorders	1/4
7 Head, ear, eye, nose, throat disorders	6/13
8 Hematologic and oncologic disorders	5/8
9 Immune system disorders	5/5
10 Systemic infectious disorders	8/9
11 Musculoskeletal disorders (nontraumatic)	8/9
12 Nervous system disorders	8/9
13 Obstetrics and gynecology	12/14
14 Psychobehavioral disorders	9/12
15 Renal and urogenital disorders	8/11
16 Thoracic-respiratory disorders	8/11
17 Toxicologic disorders	2/10
18 Traumatic disorders	11/26
All categories	133/198

Table A.2. EM topics with at least one pick, by category of the Model of the Clinical Practice of Emergency Medicine (2–18).

Appendix B

Experimental Materials

B.1 Prompts

Translation of the clinical note

Translates the Italian clinical note into English when an LLM, rather than DeepL, does the translation. The section labels are replaced with their English names in code before the call, and the line of sex and age is written in code after it. `{glossary}` holds only the entries of the DeepL glossary whose Italian term occurs in the note, as "Italian, tab, English".

```
Translate the Italian clinical note below into English.

- Translate the whole note into English, literally and completely: every sentence of the
    note appears once in the output, in English, and the Italian text is never copied. Do
    not add, remove, summarise, reorder, rephrase,
interpret or explain anything, and do not improve the style: a telegraphic or clumsy
    sentence stays telegraphic or clumsy.
- Do not correct anything, even what looks like an error or a typo, and do not complete
    what is left unfinished.
- Keep the layout exactly: line breaks, blank lines, bullets, punctuation and
    capitalisation. Headings and labels are translated like the rest of the text (ANAMNESI
    GENERALE -> GENERAL MEDICAL HISTORY, Descrizione: -> Description:); the section labels
    already in English stay exactly as they are, each followed by its colon.
- Numbers, values, units, doses, dates and scores stay exactly as written. An Italian
    abbreviation becomes its standard English equivalent (e.g. FA -> AF, e.v. -> IV) or,
    where there is none, stays as written.
- Use standard English medical terminology. A term in the glossary below always takes the
    glossary translation, and no other.
- A drug brand name in the glossary is always written as the generic name the glossary
    gives, with its dose unchanged (e.g. Clexane 4000 UI -> enoxaparin 4000 IU). This is
    part of the translation, not a correction or an interpretation.

Glossary (Italian, tab, English):
{glossary}

Output only the translation, with no preamble, notes or comments.

Clinical note:
{note}
```

Search queries

Writes the retrieval queries for a translated note, one per topic worth retrieving. `{purpose}` and `{topics}` are filled by the discharge or the admission clauses below, chosen from the note's prognosis.

```
You write search queries for a retrieval system over a corpus of official clinical
     guidelines. The passages it retrieves are the only source for this patient's
     recommendations. {purpose}

How the search works: each query is embedded and compared with guideline passages, then a
     cross-encoder scores the passages it found against that query. The passages that score
     best are the ones that read like the query. So:
- Write each query as the guideline recommendation you expect to find, in the guideline's
     own wording: a declarative sentence such as "<intervention> is recommended in patients
     with <condition> [and <patient factor>] to <aim>". Not a question, not a heading, not a
      list of keywords.
- One topic per query. A query that lists several topics matches none of them well; if two
     things matter, write two queries.
- Name the condition as the guidelines name it (e.g. the formal disease name, its type or
     stage), not with the ward shorthand of the note. Leave out the side of the body and
     words like "suspected"; mention sex or age only when they change management (pregnancy,
      children, frailty).
- Be specific where this patient is specific: the drug classes and agents, doses or
     thresholds, scores, comorbidities, current medications and contraindications that
     change what the guideline says for this patient. Use only what the note states as
     present: skip conditions that are denied, past or resolved.
- Stay within what a guideline on this condition would say. Generic ward care (antiemetics,
      routine analgesia, standard perioperative thromboprophylaxis) is rarely in it: include
      such a topic only when the note gives a specific reason.
- Do not search for the diagnostic work-up the note has already completed; search for what
     comes next.
- Keep each query under 35 words.

Topics to consider, skipping those that do not apply to this patient and splitting one when
      two distinct things matter:
{topics}

How many: one query per distinct topic, usually 5 to 8 and never more than 10. Every query
     adds retrieval time and only 12 passages reach the recommendations, so do not pad or
     split a topic into near-duplicates.

Output only the queries, one per line, in English, with no numbering, headings or other
     text.

Clinical note:
{note}
```

Discharge:

```
purpose: The patient is being discharged home. The recommendations tell the patient and
     their GP what to take (drug, dose, duration), how to manage the patient's other
     conditions and current medications alongside this diagnosis, what to change in
     lifestyle, when to be seen again, and which symptoms mean coming back.
topics:
- the definitive treatment of the condition and its drug therapy: agent, dose, duration
- the patient's other conditions and current medications, where they change that treatment
- follow-up, monitoring and reassessment after discharge
- lifestyle and self-care, only for a risk factor the patient actually has
- complications to watch for
```

Admission:

```
purpose: The patient is being admitted to hospital. The recommendations hand the patient
     over to the admitting team: the definitive treatment to start, the monitoring and
     investigations still needed, the complications to anticipate, and how the patient's
     other conditions and current medications affect that management.
topics:
- the definitive treatment (drug protocol or procedure) and its timing
- drug therapy: agent, dose, duration, and adjustment for this patient
- the patient's other conditions and current medications, where they change that treatment
- the monitoring and investigations still needed, and the criteria for escalation
- the complications to anticipate
```

Generation

Both prompts open with the same preamble: the role, the clinical note and the retrieved excerpts, each labelled with its document and pages, followed by the retrieved tables when tables are used. The example shows one excerpt and one table.

```
ROLE:
{role}

CLINICAL NOTE:

{note}

GUIDELINE EXCERPTS:

[E1] ({document}, p.{first}-{last})

{excerpt}

GUIDELINE TABLES:

[T1] ({document}, p.{first}-{last}) {caption}

{table}
```

Discharge, for a patient sent home. The role is:

```
You are an emergency department physician writing discharge recommendations for a patient.
     The document will be read by the patient and their general practitioner.
```

The preamble is followed by the task and the rules below, shown with tables; without them, “and tables” and “or tables” are left out, and so is the item on doses taken from a table.

```
TASK:
Write the discharge recommendations: exactly the two sections below, in plain text, with
     nothing before, between or after them.
Base the recommendations on the excerpts and tables that concern this patient's diagnosis
     or a condition treated in this visit (the treatment, measures and follow-up they
     recommend after discharge, and any home medication they say to stop), together with
     what the clinical note records (therapy begun in the emergency department, specialist
     advice). The excerpts were retrieved automatically and some may concern other
     conditions: ignore those.

SECTION 1 - DISCHARGE RECOMMENDATIONS
A numbered list of instructions for the patient. Go through these groups in this order, one
      instruction per item, and skip every group where nothing applies:
a. Medications to start, including a therapy begun in the emergency department that goes on
      at home: "Take [drug] [dose] [route] [frequency]." or "Continue [drug] [dose] [route]
     [frequency]."
```

```
b. Medications to stop or change: "Stop [drug]." or "Stop [drug] and take [drug] [dose] [
     route] [frequency] instead."
c. Other treatments and self-care measures, as direct instructions, including the care of
     anything applied in the emergency department (cast, splint, dressing).
d. Lifestyle changes (smoking, alcohol, diet, exercise): "[Change] is recommended." Only
     what needs to change or start; say nothing about what the patient already does right.
     State the change in general terms, with no figures or targets: "Losing weight is
     recommended.", never a percentage, an amount or a duration.
e. Follow-up visits, tests and specialist referrals: "A prescription is issued for [
     specialist type] evaluation for [clinical purpose]." or "A prescription is issued for [
     test] for [clinical purpose]." Take the specialist type from the clinical note when it
     names one. Blood tests never get an item of their own: they go in item g.
f. Always: "Go to the nearest emergency department immediately if you experience [warning
     symptoms for this diagnosis]."
g. Always, last: "You are referred back to your general practitioner for any further
     diagnostic or therapeutic needs and ongoing care." When the patient needs blood tests,
     write instead: "You are referred back to your general practitioner for any further
     diagnostic or therapeutic needs, blood tests prescription and ongoing care."

Medications:
- Never mention a medication the patient already takes and keeps taking unchanged: the
     patient goes on with it without being told. Mention a home medication only to stop or
     change it, and only when the clinical note or an excerpt that applies to this visit
     calls for it.
- Give every medication you mention its dose, route and frequency. Take them from the
     excerpts or tables when they state them, otherwise copy them exactly from the clinical
     note; where neither states them, write "as directed by the specialist" in their place.
     Intravenous therapy does not go on at home: give the oral form when the excerpts or
     tables state it.
- A dose may come from a cited table: copy the value or range exactly as written, never
     compute or interpolate. Where a table distinguishes routes, use the oral maintenance
     column and ignore intravenous ones. Before starting a drug from a table, check its
     contraindication or criteria columns against the clinical note; if one applies, do not
     start it.
- Stopping a medication and its replacement is one item. Never put a medication and a non-
     medication instruction in the same item.

Every item:
- One or two sentences. No reasons or explanations (no "because", no "to reduce the risk of
     "), no speculation about future treatment (duration of therapy, drug switches), no
     citations, no bold, headers or sub-bullets.
- If the patient needs a decision that the excerpts do not cover, such as the further work-
     up of the problem of this visit, do not make it: issue the referral of group e instead.
      Refer only for the problem of this visit, never for chronic conditions it did not
     concern.
- Never write the same instruction twice in different words.

SECTION 2 - CLINICAL REFERENCES (for the reviewing physician)
One line for every item of Section 1, with its number, in this format:
[number] - [sources] - [one-sentence clinical justification]
[sources] is always one of:
- the excerpts or tables that support the item. Cite each source by copying the (document,
     pages) shown next to its [E#] or [T#] label, never the label itself, and each distinct
     source once. Cite only a source that states what the item says.
- (Clinical note) when the item rests on the clinical note and no excerpt supports it. When
      both do, write the excerpts and then (Clinical note).
- (No guideline source) when neither an excerpt nor the clinical note supports it.

Example of the format only, for a different patient who takes ramipril and ibuprofen at
     home (ramipril goes on unchanged, so it is not mentioned):
SECTION 1 - DISCHARGE RECOMMENDATIONS
1. Stop ibuprofen and take diclofenac 75 mg orally every 12 hours instead.
```

```
2. Drinking plenty of water is recommended.
3. A prescription is issued for urologist evaluation for management of the ureteral stone.
4. Go to the nearest emergency department immediately if you experience fever, persistent
     vomiting or pain not controlled by the prescribed therapy.
5. You are referred back to your general practitioner for any further diagnostic or
     therapeutic needs and ongoing care.

SECTION 2 - CLINICAL REFERENCES (for the reviewing physician)
1 - (RenalColic, p.6-6)(RenalColic, p.8-9) - Diclofenac is first-line analgesia for renal
     colic; ibuprofen stopped to avoid two NSAIDs.
2 - (RenalColic, p.12-12) - High fluid intake reduces stone recurrence.
3 - (Clinical note) - Urology follow-up requested by the on-call urologist.
4 - (RenalColic, p.4-4) - Fever with an obstructed kidney needs urgent decompression.
5 - (No guideline source) - Standard closing item.

RULES:
- Use ONLY the clinical note and the excerpts that apply to it. Do not rely on prior
     medical knowledge.
- Do NOT invent drug names, doses, frequencies or clinical facts.
- When several excerpts support the same instruction, write one item and cite them all.
- Section 1 must be usable as a discharge document with no editing.
- Before answering, check that no unchanged home medication appears in Section 1 and that
     every line of Section 2 has the number of its item and sources in one of the three
     forms above.
- Last, once both sections are final, revisit Section 1 without adding, removing or
     changing any item: where paracetamol, ibuprofen or a topical anti-inflammatory gel
     taken for pain has "as directed by the specialist" in place of its dose and frequency,
     replace that wording with "as needed for pain" ("Take paracetamol orally as needed for
     pain."). Change nothing else, and no other drug.
```

Admission, for a patient admitted to hospital (tables are never added). The role is:

```
You are an emergency department physician writing recommendations for a patient being
     admitted to hospital. The document will be read by the admitting team, the patient, and
      their family.
```

Task and rules:

```
TASK:
The patient is being admitted to hospital. The ER does not prescribe ongoing therapy or
     follow-up: that is the admitting team's responsibility. Write a brief handover note:
     exactly the two sections below, in plain text, with nothing before, between or after
     them.

SECTION 1 - ADMISSION NOTE
A numbered list of these items, in this order:
1. "The patient is admitted [urgently] to [department] for [planned interventions]." Take
     the department and the planned interventions from the clinical note, and nothing else:
     no age, measurements, test results, therapy given in the emergency department or
     reasons.
2. Only if the patient is admitted for surgery: "The patient has been prepared for surgery
     ."
3. "The patient has been informed of the diagnosis and the planned care pathway." When the
     clinical note records no planned pathway, end the sentence at "diagnosis". Write "The
     patient and family" for a child or when the clinical note says the family was informed.
      Add nothing to this sentence: no expected course, complications, reassessment or
     treatment changes.
4. "The patient is handed over to the care of the [department] team."

Every item is the one sentence above. Do NOT prescribe medications, follow-up visits or
     lifestyle changes, and do not cite sources, use bold, headers or sub-bullets in this
     section.
```

```
SECTION 2 - CLINICAL REFERENCES (for the reviewing physician)
One line for every item of Section 1, with its number, in this format:
[number] - [sources] - [one-sentence clinical justification]
[sources] is the excerpts that support the item, then (Clinical note) when the clinical
     note does; (No guideline source) when neither does. Cite each source by copying the (
     document, pages) shown next to its [E#] or [T#] label, never the label itself, and each
      distinct source once. Cite only a source that states what the item says.
No bold or sub-bullets.

Example of the format only, for a different patient:
SECTION 1 - ADMISSION NOTE
1. The patient is admitted urgently to the Paediatrics department for intravenous
     immunoglobulin therapy, echocardiography with coronary evaluation and clinical-
     laboratory monitoring.
2. The patient and family have been informed of the diagnosis and the planned care pathway.
3. The patient is handed over to the care of the Paediatrics team.

SECTION 2 - CLINICAL REFERENCES (for the reviewing physician)
1 - (KawasakiDisease, p.4-5)(KawasakiDisease, p.9-9)(Clinical note) - IVIG and
     echocardiography are first-line treatment and assessment of Kawasaki disease, to
     prevent coronary artery aneurysms.
2 - (Clinical note) - Diagnosis and plan discussed with the parents.
3 - (No guideline source) - Standard handover item.

RULES:
- Use ONLY the clinical note and the excerpts that apply to it. Do not rely on prior
     medical knowledge.
- Do NOT invent clinical facts not in the clinical note or excerpts.
- Section 1 must be directly usable as a handover note with no editing.
```

Translation of the output

Appended to either generation prompt when the Italian translation is requested in one call:

```
ITALIAN TRANSLATION:
After Section 2, write "TRADUZIONE IN ITALIANO" on a line of its own, then translate into
     Italian everything you wrote above: both sections, with their titles, every numbered
     item and every reference line. This is the only addition allowed after the two sections
     .
- Translate faithfully and completely: the same items in the same order and numbering,
     every sentence in full. Do not add, omit, merge, split, summarise, reword or
     reinterpret anything, and do not correct or complete the content.
- Drugs take their Italian name (enoxaparin -> enoxaparina). Numbers, doses and units keep
     their exact value, written the Italian way (8,000 IU -> 8.000 UI; 2.5 mg -> 2,5 mg).
     The source citations stay exactly as written.
- Use standard Italian medical terminology.
```

The same translation as a second call on the finished output:

```
Translate into Italian the document below: both sections, with their titles, every numbered
      item and every reference line.
- Translate faithfully and completely: the same items in the same order and numbering,
     every sentence in full. Do not add, omit, merge, split, summarise, reword or
     reinterpret anything, and do not correct or complete the content.
- Drugs take their Italian name (enoxaparin -> enoxaparina). Numbers, doses and units keep
     their exact value, written the Italian way (8,000 IU -> 8.000 UI; 2.5 mg -> 2,5 mg).
     The source citations stay exactly as written.
- Use standard Italian medical terminology.
Output only the translation, with no preamble, notes or comments.
```

```
Document:
{output}
```

B.2 Storage of Tables

Below is an example of how the tables are stored in the system. Table 12 of the ESC atrial fibrillation guideline (*Drugs for rate control in AF*) is stored as one record, `AtrialFibrillation:19`, in the table store (`Weights/tables.pkl`). Its table surrogate is embedded in the table index (`Weights/tables.index`). The full table is not reproduced here, to avoid a wall of text.

Record fields Metadata and pre-computed values kept for each table, used to identify it, cite it and match it against a query.

Field	Value
`table_id`	`AtrialFibrillation:19`
`part / parts`	1 / 1
`doc_id`	`AtrialFibrillation`
`page_start-page_end`	40–41
`caption`	Table 12 Drugs for rate control in AF
`header`	Agent | Intravenous administration | Usual range for oral maintenance dose | Contraindicated
`markdown`	the table rendered as markdown (next paragraph)
`rows`	13 row records (cells, group, row surrogate, token sets)

Markdown (what the generator sees) Each table is kept in two forms: the markdown string, shown here, which is the only form given to the generator, and the structured row records described afterwards, used for retrieval and ranking. Figure B.1 shows the source table as printed in the guideline. Group headers become a leading *Group* column. A cell that spans several rows in the PDF is repeated in every row record of the store. In the markdown it is written once, and the rows below show *same as above*.

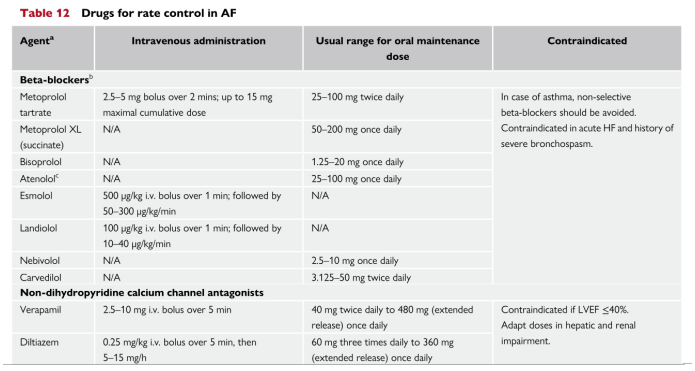


Table 12 Drugs for rate control in AF

Agent[a]	Intravenous administration	Usual range for oral maintenance dose	Contraindicated
Beta-blockers[b]			
Metoprolol tartrate	2.5–5 mg bolus over 2 mins; up to 15 mg maximal cumulative dose	25–100 mg twice daily	In case of asthma, non-selective beta-blockers should be avoided. Contraindicated in acute HF and history of severe bronchospasm.
Metoprolol XL (succinate)	N/A	50–200 mg once daily	
Bisoprolol	N/A	1.25–20 mg once daily	
Atenolol[c]	N/A	25–100 mg once daily	
Esmolol	500 µg/kg i.v. bolus over 1 min; followed by 50–300 µg/kg/min	N/A	
Landiolol	100 µg/kg i.v. bolus over 1 min; followed by 10–40 µg/kg/min	N/A	
Nebivolol	N/A	2.5–10 mg once daily	
Carvedilol	N/A	3.125–50 mg twice daily	
Non-dihydropyridine calcium channel antagonists			
Verapamil	2.5–10 mg i.v. bolus over 5 min	40 mg twice daily to 480 mg (extended release) once daily	Contraindicated if LVEF ≤40%. Adapt doses in hepatic and renal impairment.
Diltiazem	0.25 mg/kg i.v. bolus over 5 min, then 5–15 mg/h	60 mg three times daily to 360 mg (extended release) once daily	

Figure B.1. Table 12 of the ESC atrial fibrillation guideline as printed in the PDF (cropped). The *Contraindicated* cell spans all the beta-blocker rows.

```
| Group | Agent | Intravenous administration | Usual range for oral maintenance dose
↪  | Contraindicated |
```

```
| --- | --- | --- | --- | --- |
| Beta-blockers | Metoprolol tartrate | 2.5-5 mg bolus over 2 mins; up to 15 mg
↪  maximal cumulative dose | 25-100 mg twice daily | In case of asthma,
↪  non-selective beta-blockers should be avoided. Contraindicated in acute HF and
↪  history of severe bronchospasm. |
| Beta-blockers | Metoprolol XL (succinate) | N/A | 50-200 mg once daily | same as
↪  above |
[...]
| Non-dihydropyridine calcium channel antagonists | Verapamil | 2.5-10 mg i.v. bolus
↪  over 5 min | 40 mg twice daily to 480 mg (extended release) once daily |
↪  Contraindicated if LVEF <40%. Adapt doses in hepatic and renal impairment. |
```

Table surrogate (embedded) The table surrogate is a short text summary of the whole table, embedded in the table index and used to retrieve candidate tables for a query. Four parts joined by full stops: the caption, the header, the group labels and the key-column cells.

```
Table 12 Drugs for rate control in AF\@. Agent, Intravenous administration, Usual
↪  range for oral maintenance dose, Contraindicated. Beta-blockers,
↪  Non-dihydropyridine calcium channel antagonists, Digitalis glycosides, Other.
↪  Metoprolol tartrate, Metoprolol XL (succinate), Bisoprolol, Atenolol, Esmolol,
↪  Landiolol, Nebivolol, Carvedilol, Verapamil, Diltiazem, Digoxin, Digitoxin
```

Row surrogate (scored by the cross-encoder) The row surrogate is a text version of a single row, used to rank the rows of the retrieved tables against the query. Built as `caption | group | header: cell; ...`. Empty cells are skipped and the whole string is capped at 400 characters. First row:

```
Table 12 Drugs for rate control in AF | Beta-blockers | Agent: Metoprolol tartrate;
↪  Intravenous administration: 2.5-5 mg bolus over 2 mins; up to 15 mg maximal
↪  cumulative dose; Usual range for oral maintenance dose: 25-100 mg twice daily;
↪  Contraindicated: In case of asthma, non-selective beta-blockers should be
↪  avoided. Contraindicated in acute HF and history of severe bronchospasm.
```

B.3 Table of Costs

Below is a table outlining the mean API cost per note of each stage, in US cents.

Recommendations are averaged over all outputs of each model, query writing and translation over the 25 notes, except the translations, which were measured on five notes. Reranking with Qwen3-Reranker-8B applies to configurations 1 and 2 (Section 7.1); configurations 3 and 4 rerank locally with MedCPT at no API cost, so Opus in configuration 4 costs a mean 11.58 cents per note.

In a local deployment, the cost of the open-weight models would be that of the hardware rather than of the calls.

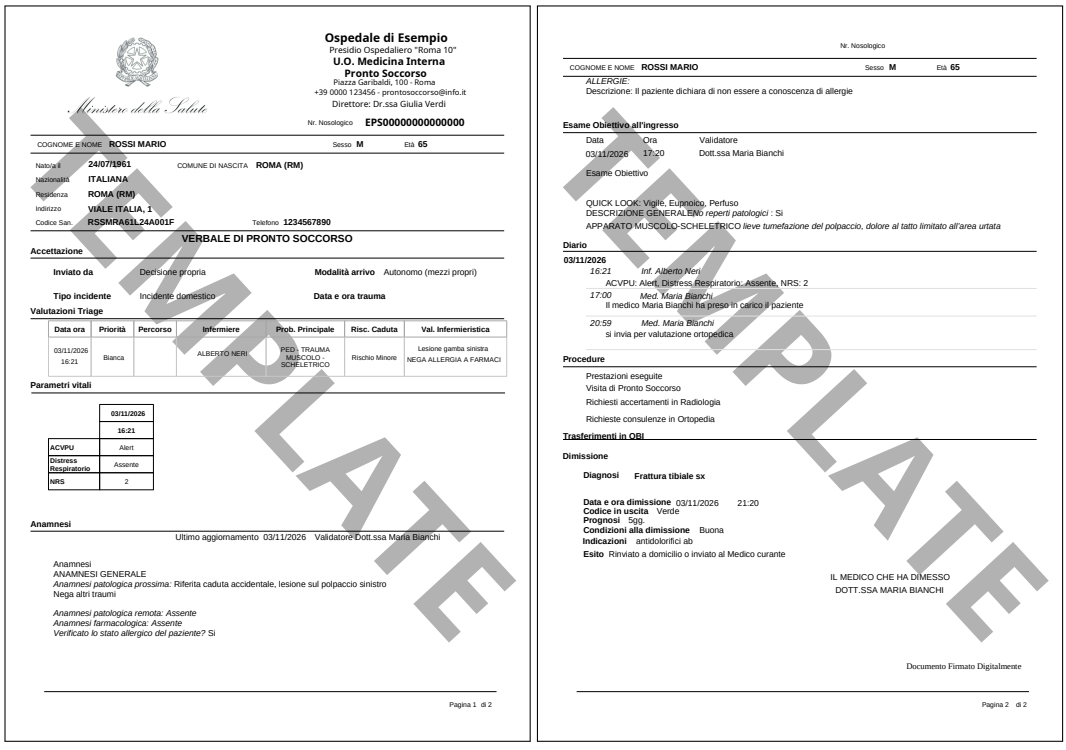
Cents per note	**Opus 5.5**	**Sonnet 5.5**	**Nemotron 3 Super**	**Gemma 4 31B**
Translation	2.86	1.44	0.83	0.25
Query writing	1.68	0.87	0.13	0.07
Recommendations	7.04	3.74	0.30	0.40
Reranking	0.62	0.62	0.57	—
Whole note	**¢12.20**	**¢6.67**	**¢1.83**	**¢0.72**

Appendix C

Output and Evaluation

C.1 Clinical Note Template

Figure C.1 shows the Emergency Department report used as a running example throughout this appendix. All data contained in it (patient, staff, hospital and identifiers) are completely fictitious, and the clinical case described is not realistic: it is used only to illustrate the process and is not part of the 25 notes used for evaluation.



Ministero della Salute

Ospedale di Esempio
Presidio Ospedaliero "Roma 10"
U.O. Medicina Interna
Pronto Soccorso
Piazza Garibaldi, 100 - Roma
+39 0000 123456 - prontosoccorso@info.it
Direttore: Dr.ssa Giulia Verdi

Nr. Nosologico **EPS00000000000000**

COGNOME E NOME **ROSSI MARIO** Sesso **M** Età **65**

Nato/a il **24/07/1961** COMUNE DI NASCITA **ROMA (RM)**
Nazionalità **ITALIANA**
Residenza **ROMA (RM)**
Indirizzo **VIALE ITALIA, 1**
Codice San. **RSSMRA61L24A001F** Telefono **1234567890**

VERBALE DI PRONTO SOCCORSO

Accettazione

Inviato da Decisione propria **Modalità arrivo** Autonomo (mezzi propri)

Tipo incidente Incidente domestico **Data e ora trauma**

Valutazioni Triage

Data ora	Priorità	Percorso	Infermiere	Prob. Principale	Risc. Caduta	Val. Infermieristica
03/11/2026 16:21	Bianca		ALBERTO NERI	PED - TRAUMA MUSCOLO - SCHELETRICO	Rischio Minore	Lesione gamba sinistra NEGA ALLERGIA A FARMACI

Parametri vitali

	03/11/2026 16:21
ACVPU	Alert
Distress Respiratorio	Assente
NRS	2

Anamnesi

Ultimo aggiornamento 03/11/2026 Validatore Dott.ssa Maria Bianchi

Anamnesi
ANAMNESI GENERALE
Anamnesi patologica prossima: Riferita caduta accidentale, lesione sul polpaccio sinistro
Nega altri traumi

Anamnesi patologica remota: Assente
Anamnesi farmacologica: Assente
Verificato lo stato allergico del paziente? Si

Pagina 1 di 2

Nr. Nosologico

COGNOME E NOME **ROSSI MARIO** Sesso **M** Età **65**

ALLERGIE:
Descrizione: Il paziente dichiara di non essere a conoscenza di allergie

Esame Obiettivo all'ingresso

Data Ora Validatore
03/11/2026 17:20 Dott.ssa Maria Bianchi

Esame Obiettivo

QUICK LOOK: Vigile, Eupnoico, Perfuso
DESCRIZIONE GENERALE*No reperti patologici* : Si
APPARATO MUSCOLO-SCHELETRICO *lieve tumefazione del polpaccio, dolore al tatto limitato all'area urtata*

Diario

03/11/2026

16:21 *Inf. Alberto Neri*
ACVPU: Alert, Distress Respiratorio: Assente, NRS: 2

17:00 *Med. Maria Bianchi*
Il medico Maria Bianchi ha preso in carico il paziente

20:59 *Med. Maria Bianchi*
si invia per valutazione ortopedica.

Procedure

Prestazioni eseguite
Visita di Pronto Soccorso
Richiesti accertamenti in Radiologia

Richieste consulenze in Ortopedia

Trasferimenti in OBI

Dimissione

Diagnosi **Frattura tibiale sx**

Data e ora dimissione 03/11/2026 21:20
Codice in uscita Verde
Prognosi 5gg.
Condizioni alla dimissione Buona
Indicazioni antidolorifici ab
Esito Rinviato a domicilio o inviato al Medico curante

IL MEDICO CHE HA DIMESSO
DOTT.SSA MARIA BIANCHI

Documento Firmato Digitalmente

Pagina 2 di 2

TEMPLATE

Figure C.1. Fictitious Emergency Department report used as reference (left: page 1, right: page 2).

Anonymized Note

After the automatic anonymisation step, the content of the note that is sent for translation:

```
Maschio, 65 anni
Anamnesi:
ANAMNESI GENERALE
Patologica prossima: Riferita caduta accidentale, lesione sul polpaccio sinistro
Nega altri traumi
Patologica remota: Assente
Anamnesi farmacologica: Assente
Verificato lo stato allergico del paziente? Si
ALLERGIE:
Descrizione: Il paziente dichiara di non essere a conoscenza di allergie

Esame obiettivo:
QUICK LOOK: Vigile, Eupnoico, Perfuso
DESCRIZIONE GENERALE No reperti patologici : Si
APPARATO MUSCOLO-SCHELETRICO lieve tumefazione del polpaccio, dolore al tatto limitato all'
    area urtata

Procedure:
Richieste consulenze in Ortopedia
```

C.2 Queries

This section reports the queries generated from the note of Appendix C.1 by the LLM-based approach (Claude Opus 5.5) and by the rule-based approach. Each query is preceded by a header with the floor applied, the guideline categories it was restricted to (if any), and the identifiers of the guideline recommendations retrieved and scored (numbers correspond to the cross-encoder final ranking); "filled floor" indicates that the minimum number of results was reached by adding the listed recommendations.

LLM-based (Opus 5.5)

```
--- Query 1 ---
[floor 1] retrieved 6 | scored 6 | filled floor with 6
Non-operative management with immobilisation in a cast or brace is recommended for adults
    with a closed, non-displaced tibial shaft fracture, with orthopaedic fracture clinic
    follow-up.

--- Query 2 ---
[floor 1] retrieved 2 | scored 2 | filled floor with 2
Oral analgesia with paracetamol and NSAIDs is recommended for adults with an acute closed
    tibial fracture managed as outpatients, avoiding prolonged opioid use.

--- Query 3 ---
[floor 1] retrieved 3 | scored 3 | filled floor with 3
Pharmacological thromboprophylaxis with low molecular weight heparin should be considered
    for adults with lower limb immobilisation in a cast after a tibial fracture, based on
    VTE risk assessment.

--- Query 4 ---
[floor 1] retrieved 5 | scored 5 | filled floor with 5
Patients with a tibial fracture should be advised to seek urgent care for increasing pain,
    tense swelling, numbness or pain on passive stretch indicating acute compartment
    syndrome.
```

```
--- Query 5 ---
[floor 1] retrieved 8 | scored 8
Patients with lower limb immobilisation should be informed about symptoms of deep vein
     thrombosis and pulmonary embolism, such as calf swelling, chest pain and breathlessness
     , and return urgently.

--- Query 6 ---
[floor 1] retrieved 7 | scored 7
Follow-up radiographs in a fracture clinic within one to two weeks are recommended in
     tibial fractures to assess alignment and fracture healing.

--- Query 7 ---
[floor 1] retrieved 4 | scored 4 | filled floor with 4
Weight-bearing as tolerated, limb elevation and early physiotherapy are recommended after a
      stable tibial fracture to restore function and prevent stiffness.

--- Query 8 ---
[floor 1] retrieved 1, 9, 10, 11, 12 | scored 1, 9, 10, 11, 12 | filled floor with 1
Assessment for osteoporosis and fracture risk is recommended in adults aged 65 and over
     presenting with a fragility fracture after a fall, together with a falls risk
     assessment.
```

Rule-based

```
--- Query 1 ---
[floor 0 | in ComplexFractures, NonComplexFractures] retrieved 2, 3, 5, 9, 11 | scored 2, 3,
      5, 11
Recommended treatment for patients with Fracture of the tibia.

--- Query 2 ---
[floor 0 | in ComplexFractures, NonComplexFractures] retrieved 3, 4, 8, 9, 10 | scored 4, 8,
      10
Recommendations for drug therapy and dosage in patients with Fracture of the tibia.

--- Query 3 ---
[floor 0 | in ComplexFractures, NonComplexFractures] retrieved 1, 2, 5, 9, 12 | scored 1,
     12
Recommendations for follow-up of patients with Fracture of the tibia.

--- Query 4 ---
[floor 0 | in ComplexFractures, NonComplexFractures] retrieved 1, 9 | scored none
Recommendations for lifestyle and risk factor management in patients with Fracture of the
     tibia.

--- Query 5 ---
[floor 0 | in ComplexFractures, NonComplexFractures] retrieved 1, 2, 6, 7, 9, 11 | scored 6,
      7, 9
Recommendations for complications and warning signs in patients with Fracture of the tibia.
```

Observations

- **Number of queries.** The rule-based approach produced only 5 queries, against the 8 of the LLM. This is due to the scarcity of the note: the rules fire only on the information actually present, which is minimal here to fit in only two pages. On longer and richer notes the same approach produced up to 22 queries.

- **Specificity.** Rule-based queries are generic templates instantiated with the diagnosis only ("Recommendations for ... in patients with Fracture of the tibia"). The LLM queries are full clinical statements that also include details such as closed and non-displaced fracture, adult patient, cast immobilisation or outpatient management, and cover specific topics (thromboprophylaxis, compartment syndrome, DVT/PE warning signs, osteoporosis).
- **Retrieval constraints.** The rule-based queries are restricted to the categories ComplexFractures and NonComplexFractures with no floor, so a query may end up with no contributions of its own. Query 4 retrieved recommendations 1 and 9, but both were scored under the queries where they ranked best (Queries 3 and 5). The LLM queries are not restricted by category and use a floor of 1, so each of them is guaranteed at least one recommendation.
- **Overlap.** The LLM queries retrieve mostly distinct recommendations, whereas the rule-based queries retrieve heavily overlapping sets (recommendation 9 is retrieved by all five queries and recommendations 1 and 2 by three).

C.3 Output Comparison with the Physician's Advice

This section compares the recommendations generated for the note of Appendix C.1 with the advice written by a physician at discharge. Since the note is fictitious, no physician's advice exists for it, so the comparison uses that of a very similar case from the realistic synthetic notes: an adult with a fracture, discharged home with pain relief and referred to the cast room for orthopaedic evaluation the following day. The two are therefore not a one-to-one comparison, but the clinical picture and the expected management are the same, so they illustrate the differences observed in the evaluation (Section 7.6.1).

Physician's Advice

Figure C.2 reports the discharge section of the real note as it appears in the document.

Condizioni alla dimissione antidolorifici ab , si invia presso sala gessi per valutazione ortopedica

Esito Rinviato a domicilio o inviato al Medico curante

Figure C.2. Discharge section written by the physician for a case of fracture. "Condizioni alla dimissione": painkillers as needed ("ab", *al bisogno*), the patient is sent to the cast room for orthopaedic evaluation. "Esito": sent home or referred to the general practitioner.

Generated Recommendations

The output of the system (Figure 6.4), reported verbatim. Section 1 is the text intended for the patient, Section 2 the rationale for the reviewing physician, with the guideline cited for each item (NCF for *Non-Complex Fractures*, CF for *Complex Fractures*).

```
SEZIONE 1 -- RACCOMANDAZIONI ALLA DIMISSIONE
1. Assumere paracetamolo per via orale al bisogno in caso di dolore.
2. Non appoggi il peso sulla gamba fratturata fino all'applicazione del gesso.
```

```
3. Si rilascia impegnativa per visita ortopedica per la gestione della frattura della tibia
      sinistra.
4. Recarsi immediatamente al pronto soccorso più vicino in caso di peggioramento del dolore
      alla gamba sinistra non controllato dalla terapia prescritta, aumento del gonfiore o
     della tensione del polpaccio, intorpidimento o formicolio della gamba o del piede,
     oppure piede pallido o freddo.
5. Si rimanda al medico di medicina generale per ogni ulteriore necessità diagnostica o
     terapeutica e per la presa in carico continuativa.

SEZIONE 2 -- RIFERIMENTI CLINICI (per il medico revisore)
1 - Negli adulti il paracetamolo per via orale è raccomandato per il dolore lieve da
     sospetta frattura tibiale, e nei pazienti anziani con fratture i FANS vengono evitati.
     [NCF] [NCF]
2 - Proteggere la frattura dal carico è la priorità in attesa del gesso. [NCF]
3 - (Clinical note) - In pronto soccorso è stata richiesta una consulenza ortopedica per la
      frattura della tibia sinistra.
4 - Alle persone con fratture tibiali deve essere spiegato come automonitorare i sintomi
     della sindrome compartimentale dopo aver lasciato l'ospedale. [CF]
5 - (No guideline source) - Voce di chiusura standard.
```

Observations

- **Coverage.** Both clinical items in the physician's advice, pain relief and orthopaedic referral, are present in the generated recommendations (items 1 and 3), as is the referral to the general practitioner that the physician's note only records as the outcome of the visit (item 5).

- **Readability.** The physician's advice consists of two sentence fragments and uses the ambniguous abbreviation "ab" (*al bisogno*, as needed). The generated recommendations are complete sentences without abbreviations, and each one states a single, clear action.

- **Warning signs.** The physician's advice contains no instruction on when to return. Item 4 lists the symptoms of compartment syndrome in lay terms (pain not controlled by therapy, swelling or tension of the calf, numbness or tingling, pale or cold foot) and tells the patient to go to the nearest emergency department if they appear, as required by the cited guideline.

- **Behavioural advice.** Item 2 asks the patient not to bear weight on the injured leg until the cast is applied. This kind of instruction is almost certainly given orally by the physician, but it is not in the written advice and could be forgotten by the time the patient is home.

- **Traceability.** Every item in Section 1 is matched to a line in Section 2 that states its source: a guideline excerpt, with a link to the page it comes from (clickable from the User Interface). The reviewing physician can therefore check each recommendation against its origin and distinguish what is grounded in the guidelines from what is not.

- **Omissions.** The generated recommendations do not mention the timing of the orthopaedic visit nor the cast room specifically, since neither is in the note or, obviously, the guidelines. These are details that the physician would add during review.

C.4 Scores per Note

Figure C.3 shows how much the overall score of each note varies across the ten runs of the end-to-end evaluation (Section 7.4.1), and how it relates to the coverage of the corpus.

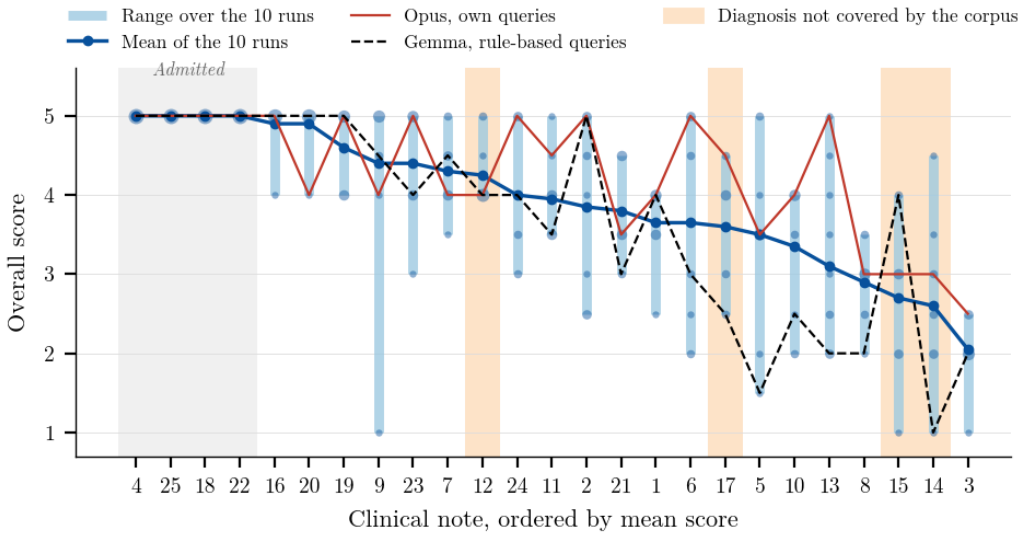


Figure C.3. Overall score of each note across the ten runs, with the notes ordered by their mean score. Bars span the range of the ten scores. The runs with the highest and lowest mean, Opus with its own queries in configuration 1 and Gemma 4 with the rule-based queries, are drawn as lines. Shaded in orange are the notes whose diagnosis the corpus does not cover, and in grey the admitted patients.

Bibliography

[1] Health Level Seven Italia. *HL7 CDA R2 Implementation Guide: Emergency Department Summary*. URL: https://www.hl7.it/realm-italiano/verbale-di-pronto-soccorso-vps/.

[2] Gianni Rastelli, Mario Cavazza, and Gianfranco Cervellin. "Sovraffollamento in Pronto Soccorso". In: *Emergency Care Journal* 4.2 (2010), pp. 25–35.

[3] Fabiola Signorini et al. "Measuring the crowding of emergency departments: an assessment of the NEDOCS in Lombardy, Italy, and the development of a new objective indicator based on the waiting time for the first clinical assessment". In: *BMC Emergency Medicine* 24.1 (2024), p. 196.

[4] Steven L Bernstein et al. "The effect of emergency department crowding on clinically oriented outcomes". In: *Academic emergency medicine* 16.1 (2009), pp. 1–10.

[5] Kirsten G Engel et al. "Patient comprehension of emergency department care and instructions: are patients aware of when they do not understand?" In: *Annals of emergency medicine* 53.4 (2009), pp. 454–461.

[6] Margaret E Samuels-Kalow, Anne M Stack, and Stephen C Porter. "Effective discharge communication in the emergency department". In: *Annals of emergency medicine* 60.2 (2012), pp. 152–159.

[7] Charlotte Blease et al. "General practitioners' adoption of generative artificial intelligence in clinical practice in the UK: An updated online survey". In: *Digital Health* 11 (2025), p. 20552076251394287.

[8] Mohamad-Hani Temsah et al. "ChatGPT and the future of digital health: a study on healthcare workers' perceptions and expectations". In: *Healthcare*. Vol. 11. 13. MDPI. 2023, p. 1812.

[9] Jonah Zaretsky et al. "Generative artificial intelligence to transform inpatient discharge summaries to patient-friendly language and format". In: *JAMA network open* 7.3 (2024), e240357.

[10] Alaa Dalky et al. "Global research trends, hotspots, impacts, and emergence of artificial intelligence and machine learning in health and medicine: a 25-year bibliometric analysis". In: *Healthcare*. Vol. 13. 8. MDPI. 2025, p. 892.

[11] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. "Med-halt: Medical domain hallucination test for large language models". In: *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL)*. 2023, pp. 314–334.

[12] Neil Rathi, Dan Jurafsky, and Kaitlyn Zhou. "Humans overrely on overconfident language models, across languages". In: *arXiv preprint arXiv:2507.06306* (2025).

[13] Christopher YK Williams et al. "Evaluating large language models for drafting emergency department encounter summaries". In: *PLOS digital health* 4.6 (2025), e0000899.

[14] Kevin Wu et al. "An automated framework for assessing how well LLMs cite relevant medical references". In: *Nature Communications* 16.1 (2025), p. 3615.

[15] Ivan Vykopal et al. "Assessing Web Search Credibility and Response Groundedness in Chat Assistants". In: *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2026, pp. 2539–2560.

[16] Mowafak Allaham and Nicholas Diakopoulos. "Synthetic Sources?: Auditing Generative Search Engine Citations for Evidence of AI-Generated Sources". In: *arXiv preprint arXiv:2605.23684* (2026).

[17] General Data Protection Regulation. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. https://gdpr-info.eu/. 2016.

[18] Judith E Tintinalli et al. "Tintinalli. Emergency Medicine". In: (2024).

[19] Michael S Beeson et al. "The 2022 model of the clinical practice of emergency medicine". In: *The Journal of Emergency Medicine* 64.6 (2023), pp. 659–695.

[20] Christine Sinsky et al. "Allocation of physician time in ambulatory practice: a time and motion study in 4 specialties". In: *Annals of internal medicine* 165.11 (2016), pp. 753–760.

[21] Karan Singhal et al. “Large language models encode clinical knowledge”. In: *Nature* 620.7972 (2023), pp. 172–180.

[22] Dave Van Veen et al. “Adapted large language models can outperform medical experts in clinical text summarization”. In: *Nature medicine* 30.4 (2024), pp. 1134–1142.

[23] Christopher YK Williams et al. “Physician-and large language model–generated hospital discharge summaries”. In: *JAMA Internal Medicine* 185.7 (2025), pp. 818–825.

[24] Simon Ellershaw et al. “Automated generation of hospital discharge summaries using clinical guidelines and large language models”. In: *AAAI 2024 Spring Symposium on Clinical Foundation Models*. 2024.

[25] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *Advances in neural information processing systems* 33 (2020), pp. 9459–9474.

[26] Hyunjae Kim et al. “Rethinking retrieval-augmented generation for medicine: A large-scale, systematic expert evaluation and practical insights”. In: *arXiv preprint arXiv:2511.06738* (2025).

[27] Guangzhi Xiong et al. “Benchmarking retrieval-augmented generation for medicine”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024, pp. 6233–6251.

[28] Cyril Zakka et al. “Almanac—retrieval-augmented language models for clinical medicine”. In: *Nejm ai* 1.2 (2024), AIoa2300068.

[29] Simone Kresevic et al. “Optimization of hepatological clinical guidelines interpretation by large language models: a retrieval augmented generation-based framework”. In: *NPJ digital medicine* 7.1 (2024), p. 102.

[30] Yu He Ke et al. “Retrieval augmented generation for 10 large language models and its generalizability in assessing medical fitness”. In: *npj Digital Medicine* 8.1 (2025), p. 187.

[31] Roberts Kirk et al. “Overview of the TREC 2016 clinical decision support track”. In: *Proceedings of the 15th text retrieval conference*. 2016.

[32] Qiao Jin et al. “Matching patients to clinical trials with large language models”. In: *Nature communications* 15.1 (2024), p. 9074.

[33] Ivan Lopez et al. “Clinical entity augmented retrieval for clinical information extraction”. In: *NPJ digital medicine* 8.1 (2025), p. 45.

[34] Orion Weller, Dawn Lawrie, and Benjamin Van Durme. “Nevir: Negation in neural information retrieval”. In: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 2274–2287.

[35] Wendy W Chapman et al. “A simple algorithm for identifying negated findings and diseases in discharge summaries”. In: *Journal of biomedical informatics* 34.5 (2001), pp. 301–310.

[36] Luyu Gao et al. “Precise zero-shot dense retrieval without relevance labels”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2023, pp. 1762–1777.

[37] Liang Wang, Nan Yang, and Furu Wei. “Query2doc: Query expansion with large language models”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, pp. 9414–9423.

[38] Félix Gaschi et al. “Multilingual clinical NER: Translation or cross-lingual transfer?” In: *Proceedings of the 5th Clinical Natural Language Processing Workshop*. 2023, pp. 289–311.

[39] Elaine C Khoong et al. “Assessing the use of Google Translate for Spanish and Chinese translations of emergency department discharge instructions”. In: *JAMA internal medicine* 179.4 (2019), pp. 580–582.

[40] Marianna Kong et al. “Evaluation of the accuracy and safety of machine translation of patient-specific discharge instructions: a comparative analysis”. In: *BMJ quality & safety* 35.3 (2026), pp. 150–158.

[41] Dipankar Srirag et al. “From Triage to Discharge: A Survey of NLP Tasks, Methods, and Open Challenges in the Emergency Department”. In: *arXiv preprint arXiv:2608.23627* (2026).

[42] Ricardo Rei et al. “COMET: A neural framework for MT evaluation”. In: *Proceedings of the 2020 conference on empirical methods in natural language processing (emnlp)*. 2020, pp. 2685–2702.

[43] Scott L Fleming et al. “Medalign: A clinician-generated dataset for instruction following with electronic medical records”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 20. 2024, pp. 22021–22030.

[44] Jean Feng et al. “Expert Evaluation of Clinical AI Tools on Real Point-of-Care Clinical Queries”. In: *arXiv preprint arXiv:2606.28960* (2026).

[45] Lianmin Zheng et al. “Judging llm-as-a-judge with mt-bench and chatbot arena”. In: *Advances in neural information processing systems* 36 (2023), pp. 46595–46623.

[46] William Philipp et al. “Clinician-Level Agreement Without Clinical Caution: LLM Evaluator Limits in Medical AI Benchmarking”. In: *arXiv preprint arXiv:2607.01103* (2026).

[47] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).

[48] Surafel Melaku Lakew, Mauro Cettolo, and Marcello Federico. “A comparison of transformer and recurrent neural networks on multilingual neural machine translation”. In: *Proceedings of the 27th international conference on computational linguistics*. 2018, pp. 641–652.

[49] Shigeki Karita et al. “A comparative study on transformer vs rnn in speech applications”. In: *2019 IEEE automatic speech recognition and understanding workshop (ASRU)*. IEEE. 2019, pp. 449–456.

[50] Thomas Wolf et al. “Transformers: State-of-the-art natural language processing”. In: *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 2020, pp. 38–45.

[51] Rico Sennrich, Barry Haddow, and Alexandra Birch. “Neural machine translation of rare words with subword units”. In: *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*. 2016, pp. 1715–1725.

[52] Kawin Ethayarajh. “How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings”. In: *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 2019, pp. 55–65.

[53] Kevin Clark et al. “What does BERT look at? an analysis of BERT’s attention”. In: *Proceedings of the 2019 ACL workshop BlackboxNLP: analyzing and interpreting neural networks for NLP*. 2019, pp. 276–286.

[54] Yi Tay et al. “Efficient transformers: A survey”. In: *ACM Computing Surveys* 55.6 (2022), pp. 1–28.

[55] Colin Raffel et al. “Exploring the limits of transfer learning with a unified text-to-text transformer”. In: *Journal of machine learning research* 21.140 (2020), pp. 1–67.

[56] Jacob Devlin et al. “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[57] Alec Radford et al. “Improving language understanding by generative pre-training”. In: (2018).

[58] *Encoders and Decoders in Transformer Models*. 2025. URL: https://machinelearningmastery.com/encoders-and-decoders-in-transformer-models/.

[59] Anna Rogers, Olga Kovaleva, and Anna Rumshisky. “A primer in BERTology: What we know about how BERT works”. In: *Transactions of the association for computational linguistics* 8 (2020), pp. 842–866.

[60] Benjamin Warner et al. “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference”. In: *Proceedings of the 63rd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 2025, pp. 2526–2547.

[61] Jinhyuk Lee et al. “BioBERT: a pre-trained biomedical language representation model for biomedical text mining”. In: *Bioinformatics* 36.4 (2020), pp. 1234–1240.

[62] Yu Gu et al. “Domain-specific language model pretraining for biomedical natural language processing”. In: *ACM Transactions on Computing for Healthcare (HEALTH)* 3.1 (2021), pp. 1–23.

[63] Emily Alsentzer et al. “Publicly available clinical BERT embeddings”. In: *Proceedings of the 2nd clinical natural language processing workshop*. 2019, pp. 72–78.

[64] Eric Lehman et al. “Do we still need clinical language models?” In: *Conference on health, inference, and learning*. PMLR. 2023, pp. 578–597.

[65] Wayne Xin Zhao et al. “A survey of large language models”. In: *Frontiers of Computer Science* 20.12 (2026), p. 2012627.

[66] Jared Kaplan et al. “Scaling laws for neural language models”. In: *arXiv preprint arXiv:2001.08361* (2020).

[67] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[68] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.

[69] Zeming Chen et al. “Meditron-70b: Scaling medical pretraining for large language models”. In: *arXiv preprint arXiv:2311.16079* (2023).

[70] Daniel P Jeong et al. “Medical adaptation of large language and vision-language models: Are we making progress?” In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 12143–12170.

[71] Sander Schulhoff et al. “The prompt report: a systematic survey of prompt engineering techniques”. In: *arXiv preprint arXiv:2406.06608* (2024).

[72] Melanie Sclar et al. “Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting”. In: *International Conference on Learning Representations*. Vol. 2024. 2024, pp. 25055–25083.

[73] Nelson F Liu et al. “Lost in the middle: How language models use long contexts”. In: *Transactions of the association for computational linguistics* 12 (2024), pp. 157–173.

[74] Woosuk Kwon et al. “Efficient memory management for large language model serving with pagedattention”. In: *Proceedings of the 29th symposium on operating systems principles*. 2023, pp. 611–626.

[75] Tim Dettmers et al. “Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale”. In: *Advances in neural information processing systems* 35 (2022), pp. 30318–30332.

[76] Elias Frantar et al. “Gptq: Accurate post-training quantization for generative pre-trained transformers”. In: *arXiv preprint arXiv:2210.17323* (2022).

[77] Tim Dettmers and Luke Zettlemoyer. “The case for 4-bit precision: k-bit inference scaling laws”. In: *International Conference on Machine Learning*. PMLR. 2023, pp. 7750–7774.

[78] Lei Huang et al. "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions". In: *ACM transactions on information systems* 43.2 (2025), pp. 1–55.

[79] Joshua Maynez et al. "On faithfulness and factuality in abstractive summarization". In: *Proceedings of the 58th annual meeting of the association for computational linguistics*. 2020, pp. 1906–1919.

[80] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. "Hallucination is inevitable: An innate limitation of large language models". In: *arXiv preprint arXiv:2401.11817* (2024).

[81] Adam Tauman Kalai et al. "Why language models hallucinate". In: *arXiv preprint arXiv:2509.04664* (2025).

[82] Ori Ram et al. "In-context retrieval-augmented language models". In: *Transactions of the Association for Computational Linguistics* 11 (2023), pp. 1316–1331.

[83] Yunfan Gao. "Retrieval-Augmented Generation for Large Language Models: A Survey". In: *arXiv preprint arXiv:2312.10997* (2023).

[84] Oded Ovadia et al. "Fine-tuning or retrieval? comparing knowledge injection in llms". In: *Proceedings of the 2024 conference on empirical methods in natural language processing*. 2024, pp. 237–250.

[85] Paul G Shekelle et al. "Validity of the Agency for Healthcare Research and Quality clinical practice guidelines: how quickly do guidelines become outdated?" In: *Jama* 286.12 (2001), pp. 1461–1467.

[86] Tianyu Gao et al. "Enabling large language models to generate text with citations". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, pp. 6465–6488.

[87] Kurt Shuster et al. "Retrieval augmentation reduces hallucination in conversation". In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 3784–3803.

[88] Manish Gupta and Michael Bendersky. "Information retrieval with verbose queries". In: *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 2015, pp. 1121–1124.

[89] Claudio Carpineto and Giovanni Romano. "A survey of automatic query expansion in information retrieval". In: *Acm Computing Surveys (CSUR)* 44.1 (2012), pp. 1–50.

[90] Nicholas J. Belkin et al. "Combining the evidence of multiple query representations for information retrieval". In: *Information Processing & Management* 31.3 (1995), pp. 431–448.

[91] Xinbei Ma et al. "Query rewriting in retrieval-augmented large language models". In: *Proceedings of the 2023 conference on empirical methods in natural language processing*. 2023, pp. 5303–5315.

[92] Zackary Rackauckas. "Rag-fusion: a new take on retrieval-augmented generation". In: *arXiv preprint arXiv:2402.03367* (2024).

[93] Wenhan Xiong et al. "Answering complex open-domain questions with multi-hop dense retrieval". In: *arXiv preprint arXiv:2009.12756* (2020).

[94] Wenhu Chen et al. "Open question answering over tables and text". In: *arXiv preprint arXiv:2010.10439* (2020).

[95] Cheng Niu et al. "Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 10862–10878.

[96] Nelson F Liu, Tianyi Zhang, and Percy Liang. "Evaluating verifiability in generative search engines". In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023, pp. 7001–7025.

[97] Shayne Longpre et al. "Entity-based knowledge conflicts in question answering". In: *Proceedings of the 2021 conference on empirical methods in natural language processing*. 2021, pp. 7052–7063.

[98] Jian Xie et al. "Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts". In: *International Conference on Learning Representations*. Vol. 2024. 2024, pp. 35623–35646.

[99] Wenxuan Zhou et al. "Context-faithful prompting for large language models". In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023, pp. 14544–14556.

[100] Freda Shi et al. "Large language models can be easily distracted by irrelevant context". In: *International conference on machine learning*. PMLR. 2023, pp. 31210–31227.

[101] Hannah Bast and Claudius Korzen. "A benchmark and evaluation for text extraction from PDF". In: *2017 ACM/IEEE joint conference on digital libraries (JCDL)*. IEEE. 2017, pp. 1–10.

[102] Galal M Binmakhashen and Sabri A Mahmoud. "Document layout analysis: a comprehensive survey". In: *ACM Computing Surveys (CSUR)* 52.6 (2019), pp. 1–36.

[103] Brandon Smock, Rohith Pesala, and Robin Abraham. "PubTables-1M: Towards comprehensive table extraction from unstructured documents". In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE. 2022, pp. 4624–4632.

[104] Denis Griffis et al. "A quantitative and qualitative evaluation of sentence boundary detection for the clinical domain". In: *AMIA summits on translational science proceedings* 2016 (2016), p. 88.

[105] Tibor Kiss and Jan Strunk. "Unsupervised multilingual sentence boundary detection". In: *Computational linguistics* 32.4 (2006), pp. 485–525.

[106] Jonathon Read et al. "Sentence boundary detection: A long solved problem?" In: *Proceedings of COLING 2012: Posters*. 2012, pp. 985–994.

[107] Tong Chen et al. “Dense x retrieval: What retrieval granularity should we use?” In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 15159–15177.

[108] Renyi Qu, Ruixuan Tu, and Forrest Bao. “Is semantic chunking worth the computational cost?” In: *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025, pp. 2155–2177.

[109] Yanshan Wang et al. “Clinical information extraction applications: a literature review”. In: *Journal of biomedical informatics* 77 (2018), pp. 34–49.

[110] Aurélie Névéol et al. “Clinical natural language processing in languages other than English: opportunities and challenges”. In: *Journal of biomedical semantics* 9.1 (2018), p. 12.

[111] Behrang Mohit. “Named entity recognition”. In: *Natural language processing of semitic languages*. Springer, 2014, pp. 221–245.

[112] Özlem Uzuner et al. “2010 i2b2/VA challenge on concepts, assertions, and relations in clinical text”. In: *Journal of the American Medical Informatics Association* 18.5 (2011), pp. 552–556.

[113] John Lafferty, Andrew McCallum, and Fernando CN Pereira. “Conditional random fields: Probabilistic models for segmenting and labeling sequence data”. In: (2001).

[114] Guillaume Lample et al. “Neural architectures for named entity recognition”. In: *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*. 2016, pp. 260–270.

[115] Jing Li et al. “A survey on deep learning for named entity recognition”. In: *IEEE transactions on knowledge and data engineering* 34.1 (2020), pp. 50–70.

[116] Henk Harkema et al. “ConText: an algorithm for determining negation, experiencer, and temporal status from clinical reports”. In: *Journal of biomedical informatics* 42.5 (2009), pp. 839–851.

[117] Yi Luan et al. “Sparse, dense, and attentional representations for text retrieval”. In: *Transactions of the Association for Computational Linguistics* 9 (2021), pp. 329–345.

[118] Gerard Salton, Anita Wong, and Chung-Shu Yang. “A vector space model for automatic indexing”. In: *Communications of the ACM* 18.11 (1975), pp. 613–620.

[119] Justin Zobel and Alistair Moffat. “Inverted files for text search engines”. In: *ACM computing surveys (CSUR)* 38.2 (2006), 6–es.

[120] Karen Sparck Jones. “A statistical interpretation of term specificity and its application in retrieval”. In: *Journal of documentation* 28.1 (1972), pp. 11–21.

[121] Gerard Salton and Christopher Buckley. “Term-weighting approaches in automatic text retrieval”. In: *Information processing & management* 24.5 (1988), pp. 513–523.

[122] Stephen Robertson and Hugo Zaragoza. “The probabilistic relevance framework: BM25 and beyond”. In: *Foundations and trends® in information retrieval* 4.1-2 (2009), pp. 1–174.

[123] Nandan Thakur et al. “Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models”. In: *arXiv preprint arXiv:2104.08663* (2021).

[124] Olivier Bodenreider. “The unified medical language system (UMLS): integrating biomedical terminology”. In: *Nucleic acids research* 32.suppl_1 (2004), pp. D267–D270.

[125] Zellig S Harris. “Distributional structure”. In: *Word* 10.2-3 (1954), pp. 146–162.

[126] Tomas Mikolov et al. “Efficient estimation of word representations in vector space”. In: *arXiv preprint arXiv:1301.3781* (2013).

[127] Wayne Xin Zhao et al. “Dense text retrieval based on pretrained language models: A survey”. In: *ACM Transactions on Information Systems* 42.4 (2024), pp. 1–60.

[128] Vladimir Karpukhin et al. “Dense passage retrieval for open-domain question answering”. In: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*. 2020, pp. 6769–6781.

[129] Nils Reimers and Iryna Gurevych. “Sentence-bert: Sentence embeddings using siamese bert-networks”. In: *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.

[130] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. “Representation learning with contrastive predictive coding”. In: *arXiv preprint arXiv:1807.03748* (2018).

[131] Qiao Jin et al. “Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval”. In: *Bioinformatics* 39.11 (2023), btad651.

[132] Christopher Sciavolino et al. “Simple entity-centric questions challenge dense retrievers”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021, pp. 6138–6148.

[133] Niklas Muennighoff et al. “Mteb: Massive text embedding benchmark”. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, pp. 2014–2037.

[134] Alfirna Rizqi Lahitani, Adhistya Erna Permanasari, and Noor Akhmad Setiawan. “Cosine similarity to determine similarity measure: Study case in online essay assessment”. In: *2016 4th International conference on cyber and IT service management*. IEEE. 2016, pp. 1–6.

[135] Anshumali Shrivastava and Ping Li. “Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS)”. In: *Advances in neural information processing systems* 27 (2014).

[136] Harald Steck, Chaitanya Ekanadham, and Nathan Kallus. "Is cosine-similarity of embeddings really about similarity?" In: *Companion Proceedings of the ACM Web Conference 2024*. 2024, pp. 887–890.

[137] Jeff Johnson, Matthijs Douze, and Hervé Jégou. "Billion-scale similarity search with GPUs". In: *IEEE transactions on big data* 7.3 (2019), pp. 535–547.

[138] Roger Weber, Hans-Jörg Schek, Stephen Blott, et al. "A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces". In: *VLDB*. Vol. 98. 1998, pp. 194–205.

[139] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms". In: *Information Systems* 87 (2020), p. 101374.

[140] Herve Jegou, Matthijs Douze, and Cordelia Schmid. "Product quantization for nearest neighbor search". In: *IEEE transactions on pattern analysis and machine intelligence* 33.1 (2010), pp. 117–128.

[141] Matthijs Douze et al. "The faiss library". In: *IEEE Transactions on Big Data* (2025).

[142] Florin Cuconasu et al. "The power of noise: Redefining retrieval for rag systems". In: *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 2024, pp. 719–729.

[143] Rodrigo Nogueira et al. "Multi-stage document ranking with BERT". In: *arXiv preprint arXiv:1910.14424* (2019).

[144] Joseph A Shaw and Edward A Fox. "Combination of multiple searches". In: *NIST special publication SP* (1995), pp. 105–105.

[145] Joon Ho Lee. "Analyses of multiple evidence combination". In: *Proceedings of the 20th annual international ACM SIGIR conference on Research and development in information retrieval*. 1997, pp. 267–276.

[146] Mark Montague and Javed A Aslam. "Relevance score normalization for metasearch". In: *Proceedings of the tenth international conference on Information and knowledge management*. 2001, pp. 427–433.

[147] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. "Reciprocal rank fusion outperforms condorcet and individual rank learning methods". In: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 2009, pp. 758–759.

[148] Jaime G Carbonell and Jade Goldstein. "The use of MMR, diversity-based reranking for reordering documents and producing summaries." In: *SIGIR*. Vol. 98. 10.1145. 1998, pp. 290941–291025.

[149] Rodrigo Nogueira and Kyunghyun Cho. "Passage Re-ranking with BERT". In: *arXiv preprint arXiv:1901.04085* (2019).

[150] Samuel Humeau et al. "Poly-encoders: Transformer architectures and pre-training strategies for fast and accurate multi-sentence scoring". In: *arXiv preprint arXiv:1905.01969* (2019).

[151] hackerllama. *Sentence Embeddings. Cross-encoders and Re-ranking*. URL: https://osanseviero.github.io/hackerllama/blog/posts/sentence_embeddings2/.

[152] Rodrigo Nogueira et al. "Document ranking with a pretrained sequence-to-sequence model". In: *Findings of the association for computational linguistics: EMNLP 2020*. 2020, pp. 708–718.

[153] Weiwei Sun et al. "Is ChatGPT good at search? investigating large language models as re-ranking agents". In: *Proceedings of the 2023 conference on empirical methods in natural language processing*. 2023, pp. 14918–14937.

[154] Scott Barnett et al. "Seven failure points when engineering a retrieval augmented generation system". In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 2024, pp. 194–199.

[155] Earl Steinberg et al. *Clinical practice guidelines we can trust*. national academies press, 2011.

[156] Paul K Whelton et al. "Harmonization of the American College of Cardiology/American Heart Association and European Society of Cardiology/European Society of Hypertension blood pressure/hypertension guidelines: comparisons, reflections, and recommendations". In: *European heart journal* 43.35 (2022), pp. 3302–3311.

[157] *Disposizioni in materia di sicurezza delle cure e della persona assistita, nonche' in materia di responsabilita' professionale degli esercenti le professioni sanitarie*. 2017. URL: https://www.gazzettaufficiale.it/atto/stampa/serie_generale/originario.

[158] Primiano Iannone. *Il nuovo SNLG: standard metodologici per la elaborazione e la valutazione delle linee guida*. Presentation, Centro Nazionale per l'Eccellenza Clinica,Istituto Superiore di Sanità. Rome, May 2018. URL: https://www.lice.it/pdf/Presentazione_SNLG.pdf.

[159] Shuyin Ouyang et al. "An empirical study of the non-determinism of chatgpt in code generation". In: *ACM Transactions on Software Engineering and Methodology* 34.2 (2025), pp. 1–28.

[160] Berk Atil et al. "Non-determinism of" deterministic" llm settings". In: *arXiv preprint arXiv:2408.04667* (2024).

[161] Carl Preiksaitis et al. "Weighing the gold standard: The breadth of emergency medicine core content covered by textbooks". In: *AEM Education and Training* 8.3 (2024), e10994.

[162] Carolyn E Lipscomb. "Medical subject headings (MeSH)". In: *Bulletin of the Medical Library Association* 88.3 (2000), p. 265.

[163] National Library of Medicine. *About PubMed*. URL: https://pubmed.ncbi.nlm.nih.gov/about/ (visited on 10/08/2026).

[164] Nicolas Fiorini et al. "Best match: new relevance search for PubMed". In: *PLoS biology* 16.8 (2018), e2005343.

[165] *Anthropic: Claude Opus 5.5 Pricing*. URL: https://openrouter.ai/anthropic/claude-opus-5.5 (visited on 10/08/2026).

[166] *Anthropic: Claude Sonnet 5.5 Pricing*. URL: https://openrouter.ai/anthropic/claude-sonnet-5.5 (visited on 10/08/2026).

[167] National Library of Medicine. *Accessing PMC Article Datasets Using Amazon Web Services*. URL: https://pmc.ncbi.nlm.nih.gov/tools/pmcaws/ (visited on 10/08/2026).

[168] OpenAlex. *OpenAlex: Unlock the world's research*. URL: https://openalex.org (visited on 10/08/2026).

[169] Edwin B Wilson. "Probable inference, the law of succession, and statistical inference". In: *Journal of the American Statistical Association* 22.158 (1927), pp. 209–212.

[170] Lawrence D Brown, T Tony Cai, and Anirban DasGupta. "Interval estimation for a binomial proportion". In: *Statistical science* (2001), pp. 101–117.

[171] DeepL SE. *DeepL API Documentation*. URL: https://developers.deepl.com/docs (visited on 10/10/2026).

[172] OpenRouter. *OpenRouter: The Unified Interface for LLMs*. URL: https://openrouter.ai (visited on 10/10/2026).

[173] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019, pp. 8024–8035.

[174] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3982–3992.

[175] Matthijs Douze et al. "The Faiss library". In: *arXiv preprint arXiv:2401.08281* (2024).

[176] Artifex Software. *PyMuPDF Documentation*. URL: https://pymupdf.readthedocs.io (visited on 10/10/2026).

[177] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O'Reilly Media, 2009.

[178] Fabian Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[179] Snowflake Inc. *Streamlit Documentation*. URL: https://docs.streamlit.io (visited on 10/10/2026).

[180] lokeshch19. *ModernPubMedBERT*. URL: https://huggingface.co/lokeshch19/ModernPubMedBERT (visited on 10/11/2026).

[181] NVIDIA. *Nemotron-3-Embed-1B Model Card*. URL: https://huggingface.co/nvidia/Nemotron-3-Embed-1B-BF16 (visited on 10/11/2026).

[182] Jörg Tiedemann and Santhosh Thottingal. "OPUS-MT–Building open translation services for the World". In: *Annual Conference of the European Association for Machine Translation*. European Association for Machine Translation. 2020, pp. 479–480.

[183] Sneha Kudugunta et al. "Madlad-400: A multilingual and document-level large audited dataset". In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 67284–67296.

[184] Clinical-AI-Apollo. *Medical-NER*. URL: https://huggingface.co/Clinical-AI-Apollo/Medical-NER (visited on 10/12/2026).

[185] Congying Xia et al. "FOFO: A benchmark to evaluate LLMs' format-following capability". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 680–699.

[186] Chaofan Li et al. *Making Large Language Models A Better Foundation For Dense Retrieval*. 2023. arXiv: 2312.15503 [cs.CL].

[187] Yanzhao Zhang et al. "Qwen3 embedding: Advancing text embedding and reranking through foundation models". In: *arXiv preprint arXiv:2506.05176* (2025).

[188] NVIDIA. *llama-nemotron-rerank-1b Model Card*. URL: https://huggingface.co/nvidia/llama-nemotron-rerank-1b-v2 (visited on 10/12/2026).

[189] ggml-org. *llama.cpp: LLM inference in C/C++*. 2023. URL: https://github.com/ggml-org/llama.cpp (visited on 10/04/2026).

[190] Tiziano Labruna et al. *eCREAM-MedCorpus A Large-Scale Corpus of Clinical Notes for Italian*. 2026. arXiv: 2606.12569 [cs.CL]. URL: https://arxiv.org/abs/2606.12569.

[191] Unsloth AI. *NVIDIA-Nemotron-3-Super-120B-A12B-GGUF*. 2026. URL: https://huggingface.co/unsloth/NVIDIA-Nemotron-3-Super-120B-A12B-GGUF (visited on 10/12/2026).

[192] Qwen Team. *Qwen3-Reranker-8B*. 2025. URL: https://huggingface.co/Qwen/Qwen3-Reranker-8B (visited on 10/12/2026).

[193] Unsloth AI. *Gemma 4 QAT*. 2026. URL: https://unsloth.ai/docs/models/gemma-4/qat (visited on 10/12/2026).

[194] Kishore Papineni et al. "Bleu: a method for automatic evaluation of machine translation". In: *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 2002, pp. 311–318.

[195] Chin-Yew Lin. "Rouge: A package for automatic evaluation of summaries". In: *Text summarization branches out*. 2004, pp. 74–81.

[196] Wilfrid J Dixon and Alexander M Mood. "The statistical sign test". In: *Journal of the American Statistical Association* 41.236 (1946), pp. 557–566.

[197] Charles Spearman. "The proof and measurement of association between two things." In: (1961).

[198] Michael D Cabana et al. "Why don't physicians follow clinical practice guidelines?: A framework for improvement". In: *Jama* 282.15 (1999), pp. 1458–1465.

[199] Mistral AI. *Mistral-7B-Instruct-v0.3*. 2024. URL: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3 (visited on 10/13/2026).

[200] Google. *MedGemma 4B IT*. 2025. URL: https://huggingface.co/google/medgemma-4b-it (visited on 10/13/2026).

[201] Qwen Team. *Qwen3.5-9B*. 2026. URL: https://huggingface.co/Qwen/Qwen3.5-9B (visited on 10/13/2026).

[202] Kate Goddard, Abdul Roudsari, and Jeremy C Wyatt. "Automation bias: a systematic review of frequency, effect mediators, and mitigators". In: *Journal of the American Medical Informatics Association* 19.1 (2012), pp. 121–127.

[203] European Parliament and Council of the European Union. *Regulation (EU) 2017/745 on medical devices*. 2017. URL: https://eur-lex.europa.eu/eli/reg/2017/745/oj/eng.

[204] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng.

[205] Yoon Kim and Alexander M. Rush. "Sequence-Level Knowledge Distillation". In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 2016.

List of Figures

1.1 Illustrative examples of recommendation sections as typically written under time pressure. Abbreviations (s.c., die, fl, ab), brief and ambiguous instructions, misspellings, sentence fragments and generic prescriptions ("antidolorifici", painkillers). 2

3.1 The transformer architecture, with the encoder stack on the left and the decoder stack on the right[47]. 18

3.2 Comparison between a bi-encoder (left) and a cross-encoder (right)[151]. 31

4.1 Overview of the system. The ingestion pipeline runs offline and writes the stored files; the query processing and generation pipeline runs once per clinical note and reads them. 34

4.2 The guideline ingestion pipeline. Each page is split into a table stream and a running-text stream, which are processed separately. 36

4.3 The query processing and generation pipeline. The note is anonymised and translated, decomposed into queries, and used to retrieve text excerpts and then tables, which ground the generated recommendation. 39

5.1 The procedure that builds the corpus. Shaded steps involve the judgement of an LLM. 46

5.2 Chapters per section of the reference textbook by the tier of their picks. The fraction next to each bar is the share of chapters with at least one pick. Sections whose chapters were all excluded from the scope (Table 5.1) are not shown. 61

6.1 The procedure of the retrieval of guideline excerpts for a clinical note. 77

6.2 The interface after uploading a report: the anonymised note in an editable box, the titles of the excluded sections below it, and the diagnosis and prognosis fields. The report shown is the template of Appendix C.1, filled in for display only. 87

6.3 The upper section of the interface in the full mode, with the option controls above the file selector and the index maintenance panel at the bottom. The excerpt selection option is hidden, since it only applies to the prompt-only mode. The interface is in Italian, the language of the department. 88

6.4 Recommendations generated for the note of Figure 6.2. The first section can be copied into the letter as it is, while each line of the second ends with links to the cited guidelines, labelled with their initials (NCF for *Non-Complex Fractures*, CF for *Complex Fractures*). Two links on the same line point to two excerpts, which come from different pages when the initials are the same. 90

7.1 Overall score (left) and coverage of the physician's advice (right) of each note in the ten runs. R and O stand for rule-based and own queries, and Opus* for Opus in configuration 4. Notes the corpus does not cover are in italics. Coverage cells report the advice items found in the output over those in the advice. 100
7.2 Mean overall score over the 25 notes of each generation model with the rule-based queries and with its own queries, showing the difference between the two. 102
7.3 Mean seconds per note of Opus and Sonnet in configuration 1, by stage. . . . 103

B.1 Table 12 of the ESC atrial fibrillation guideline as printed in the PDF (cropped). The *Contraindicated* cell spans all the beta-blocker rows. 129

C.1 Fictitious Emergency Department report used as reference (left: page 1, right: page 2). 131
C.2 Discharge section written by the physician for a case of fracture. "Condizioni alla dimissione": painkillers as needed ("ab", *al bisogno*), the patient is sent to the cast room for orthopaedic evaluation. "Esito": sent home or referred to the general practitioner. 134
C.3 Overall score of each note across the ten runs, with the notes ordered by their mean score. Bars span the range of the ten scores. The runs with the highest and lowest mean, Opus with its own queries in configuration 1 and Gemma 4 with the rule-based queries, are drawn as lines. Shaded in orange are the notes whose diagnosis the corpus does not cover, and in grey the admitted patients. 136

List of Tables

4.1 Options for the configurable components and their effect on cost, privacy and performance. All other components run locally in every configuration. 43

5.1 Chapters of the reference textbook excluded from the scope. 49
5.2 The PubMed searches of the final run, European sources only. The fallback tier is reported in Section 5.4. 55
5.3 The fallback tier in the final run. 57
5.4 Retrieval of full texts in the final run. 59
5.5 The corpus produced by the final run. 60
5.6 Results of the evaluation of the corpus, with Wilson score intervals. 64

6.1 Main code modules and their responsibilities. 66
6.2 Main libraries and services used in the implementation. 67
6.3 Embedding models considered for the system. 71
6.4 Stored files produced by the ingestion pipeline. 72
6.5 Main parameters of query construction and retrieval. 83
6.6 Options of the user interface and the mode in which each of them is available. 89

7.1 Configurations of the end-to-end evaluation. Nemotron-3-Embed-1B used as the embedding model. 92
7.2 Criteria used to score each output. 94
7.3 Useful excerpts over the 25 notes with each embedding model, out of 300 with 12 excerpts per note and 375 with 15; p from a two-sided sign test; bge-reranker-large[186] as the cross-encoder. 97
7.4 Cross-encoders compared on the same candidates. Tables were counted over the 21 patients going home. 98
7.5 Results of the ten runs over the 25 notes. Coverage is the share of the 37 items of the physicians' advice that the outputs contain; wrong and harmful statements are totals over the 25 outputs; additions is the number of outputs with useful additions compared to the original recommendations; score is the mean overall score (subjective), from 1 to 5. 99
7.6 Paired comparisons of the overall score over the 25 notes: notes on which A scored higher, ties, and notes on which B scored higher; mean difference between A and B; two-sided sign test, excluding ties. 101
7.7 Mean seconds per note over eight notes, from the PDF report to the output, excluding translation and the loading of the models. Parsing takes about 0.01 seconds. 103

A.1 Chapters with at least one pick, by section of the textbook, as named on the companion site. 121
A.2 EM topics with at least one pick, by category of the Model of the Clinical Practice of Emergency Medicine (2–18). 122